

JAVA SDE-2 INTERVIEW PREPARATION SERIES

LEARNING STEP 1 - VOLUME 3 OF 18

Java Foundations for Problem Solving

Java Fundamentals for DSA and SDE-2 Interviews

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Restart from the first Java program, master language and collection behavior, then prove interview readiness through executable examples, traps, debugging, and focused practice.

JAVA BASICS | TYPES | OBJECTS | COLLECTIONS | TRAPS | INTERVIEWS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to the **Absolute Beginner route in the series index**. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume is the canonical Java language foundation and the recommended first book in the series. It starts with source-to-execution and program structure, builds through types, control flow, methods, arrays, strings, objects, core libraries, exceptions, generics, and usage-first collections, then converts that knowledge into interview-quality implementation and debugging skill.

Readiness map

Decision	Evidence
START HERE	A correct algorithm is failing because of Java semantics; Primitive, reference, equality, or pass-by-value behavior is unclear; Core List, Set, Map, Queue, Deque, or PriorityQueue usage is not automatic; The interviewer asks for cleaner, safer, or more testable Java.
PREPARE FIRST	No prior Java is required; A Java 21 JDK is useful for running the examples; Curiosity and willingness to dry-run small programs.
MOVE ON WHEN	Explain Java program execution and write compiling Java from the first program onward; Use types, control flow, methods, arrays, strings, objects, exceptions, generics, and core collections precisely; Diagnose common Java interview traps and write safe comparators and numeric expressions; Continue to Time and Space Complexity with enough Java fluency to analyze real code.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

START OF SERIES	YOU ARE HERE 03 - Java Problem-Solving Foundations	02 - Time and Space Complexity
-----------------	--	--------------------------------

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	5
Chapter 1 - Java and Program Foundations	5
Chapter 2 - Variables, Primitive Types, Literals, and Numeric Semantics	10
Chapter 3 - Operators, Expressions, and Control Flow	18
Chapter 4 - Methods, Overloading, Varargs, and Pass-by-Value	26
Chapter 5 - Arrays, Strings, Text Blocks, and Unicode	34
Chapter 6 - Practice Lab A - Language and Execution	42
Chapter 7 - Classes, Objects, and Object-Oriented Foundations	48
Chapter 8 - Practice Lab B - Arrays, Strings, and Objects	56
Chapter 9 - Wrappers, Generics, Enums, Collections, I/O, and Utilities	62
Chapter 10 - Java Collections Basics for SDE-2 Interviews	69
Chapter 11 - Exceptions and Resource Management	79
Chapter 12 - Collections Framework Architecture	87
Chapter 13 - Comparable, Comparator, Sorting, and Selection	96
Chapter 14 - Practice Lab C - Wrappers, Generics, Collections, Exceptions, I/O	106
Chapter 15 - The Java Coding Interview Playbook	112
Chapter 16 - Forty Java Interview Traps	124
Chapter 17 - Practice Lab D - Traps, Refactoring, and Readiness	136
Chapter 18 - Solution Studio A - Language and Execution	142
Chapter 19 - Solution Studio B - Arrays, Strings, and Objects	148

CONTENTS NAVIGATION

Chapter 20 - Solution Studio C - Wrappers, Generics, Collections, Exceptions, I/O	153
Chapter 21 - Solution Studio D - Traps and Readiness	158
Chapter 22 - Java Syntax and Language Quick Reference	163
Chapter 23 - Seventy Compiling Java Fundamentals Examples	174
Part II - Practice and Handoff	195
Practice Ladder and Next Steps	195

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Java and Program Foundations

Learning objectives

By the end of this chapter, you can explain why Java is useful in coding interviews, trace a source file through compilation and execution, read the smallest complete program, and distinguish compile-time, runtime, and logical failures.

Why Java works well for DSA interviews

Java makes many contracts visible. A variable has a declared type, array indexes are checked, and the standard library provides lists, sets, maps, queues, deques, heaps, sorting, parsing, and numeric utilities. Automatic memory management removes manual deallocation from ordinary interview code. These properties let a candidate concentrate on the invariant and the algorithm while still writing code that is explicit enough to review.

The trade-off is ceremony. Java solutions are usually longer than equivalent Python solutions, primitive and reference behavior must be understood, and a careless choice such as `int` instead of `long` can silently overflow. Good interview Java is intentionally plain: familiar APIs, descriptive names, small helpers, explicit boundary handling, and no abstraction that does not clarify the solution.

Java is not "always faster," and platform independence is not absolute. `javac` normally produces platform-neutral class files, and a compatible JVM translates their operations for a host. Native libraries, file systems, default charsets, environment variables, timing, and resource limits can still make a program environment-dependent.

Source, bytecode, and execution

TEXT

```
Hello.java source
      |
      | javac Hello.java
      v
Hello.class bytecode
      |
      | java Hello
      v
JVM execution on the host machine
```

- The **JDK** contains development tools such as `javac`, the `java` launcher, and a runtime.
- The **JVM** is the execution engine for class files.

- **JRE** remains useful as the conceptual name for a JVM plus the libraries and support files needed to run Java. Modern deployments often ship a JDK or a custom runtime image rather than a separately branded JRE.

Keep this mental model high level here. Class loading, JIT compilation, garbage collectors, and stack-frame internals belong in the JVM volume.

The first complete program

File: `HelloInterview.java`

JAVA

```
public class HelloInterview {  
    public static void main(String[] args) {  
        System.out.println("Ready for Java");  
    }  
}
```

Expected output:

TEXT

Ready for Java

Compile and run:

BASH

```
javac HelloInterview.java  
java HelloInterview
```

Read it from the outside inward:

- 1 `public class HelloInterview` declares a class named `HelloInterview`. A public top-level class normally lives in a file with the same name.
- 2 `{` and `}` delimit the class body and method body.
- 3 `main` is the entry method used by the launcher.
- 4 `public` lets the launcher access it.
- 5 `static` means the launcher does not need to construct `HelloInterview` first.
- 6 `void` means the method returns no value.
- 7 `String[] args` receives command-line arguments as an array of strings.
- 8 `System.out.println(...)` calls a method that writes a line.
- 9 The semicolon terminates the statement.

Do not memorize the signature as unexplained punctuation. Read it as: "an accessible class method named main, returning nothing, that accepts an array of text arguments."

Statements, blocks, and comments

A **statement** performs an action. A **block** groups statements and creates a scope. Java ignores most whitespace, but consistent indentation exposes structure.

JAVA

```
int score = 72;           // end-of-line comment
if (score >= 70) {        // block begins
    System.out.println("pass");
}

/* A block comment can span lines. */
```

Use comments to explain a decision or invariant, not to translate obvious syntax. In interviews, `sum += number; // add number to sum` adds noise; `// sum equals numbers[0..index)` records useful reasoning.

Three failure stages

Failure	When found	Example	Repair direction
Compile-time error	before a class file is produced	missing semicolon, incompatible type, uninitialized local	repair Java syntax or type rules
Runtime error	while the program executes	null dereference, invalid index, division by zero	validate state or correct the execution path
Logical error	program runs but result is wrong	loop skips the last element	repair the invariant, condition, or formula

Compile-time-error demonstration:

JAVA

```
int answer;
System.out.println(answer); // local variable answer might not have been initialized
```

Corrected version:

JAVA

```
int answer = 0;
System.out.println(answer);
```

Fields receive specified default values when an object or class is initialized. Local variables do not; a local must be definitely assigned before it is read.

TRACE IT | A beginner execution dry run

JAVA

```
public class SumArguments {  
    public static void main(String[] args) {  
        int sum = 0;  
        for (String argument : args) {  
            sum += Integer.parseInt(argument);  
        }  
        System.out.println(sum);  
    }  
}
```

For `java SumArguments 4 7 9`:

- 1 `args` refers to `{"4", "7", "9"}`.
- 2 `sum` starts at 0.
- 3 Each string is parsed as an `int`; `sum` becomes 4, then 11, then 20.
- 4 The loop ends and the program prints 20.

`Integer.parseInt` can throw `NumberFormatException`; later chapters define parsing and exception contracts. The point here is the execution sequence: initialize, repeat, update state, produce output.

WATCH OUT | Interview angle and common mistakes

- Saying "write once, run anywhere" without the compatible-JVM and environment qualification.
- Calling bytecode machine code.
- Saying the JDK, JRE, and JVM are interchangeable names.
- Omitting the filename/class-name relationship for a public top-level class.
- Claiming local variables automatically start at zero.
- Copying a long `main` signature from memory without being able to explain each token.
- Adding console-input code when the interview platform already provides a method signature.

An interviewer may ask where a failure occurs. Answer with evidence: the compiler rejects an incompatible assignment; the JVM throws an exception on a reached bad operation; a wrong answer with no error is a logical defect.

Quick check

- 1 What artifact does `javac` normally produce?
- 2 Why is `main` static?
- 3 Does platform independence guarantee identical operating-system behavior?
- 4 Why can a field be read at its default value while an unassigned local cannot?
- 5 Which failure category describes an off-by-one result with no exception?

Small practice

Foundation: Write `Greeting.java` so `java Greeting Vinay` prints `Hello, Vinay`. Handle an empty `args` array.

Interview Core: Predict which of four lines in a supplied program fail at compilation, runtime, or logically, and justify the stage.

SDE-2 Follow-up: Explain what a build using JDK 21 and `javac --release 17` does and does not guarantee.

Cross-book boundary

Use this chapter's execution model for the rest of Volume 03. Continue to the JVM volume for class loading, JIT, garbage collection, stack frames, and memory diagnostics. Continue to Advanced Java for modules, packaging, and language evolution in depth.

RECAP | Chapter summary

Java interview code is valuable because types and library contracts are explicit. Source is compiled to class files and executed by a compatible JVM; that portability has environmental limits. A complete program is a class containing an accessible static `main`, and failures must be located at compile time, runtime, or in logic before they can be repaired.

SDE-2 CORE CHAPTER

Chapter 2 - Variables, Primitive Types, Literals, and Numeric Semantics

Learning objectives

By the end of this chapter, you should be able to:

- distinguish a variable, a value, a type, and an object reference;
- choose an appropriate primitive type and write unambiguous literals;
- predict integer promotion, floating-point behavior, narrowing, and overflow;
- explain local-variable initialization, `final`, `var`, boxing, and unboxing; and
- design numeric code whose domain assumptions are explicit and testable.

WHY IT WORKS | Why this matters at SDE-2

Many production failures are not algorithm failures. They are representation failures: an order total overflows an `int`, a nullable `Integer` is unboxed, a monetary comparison uses `double`, or a shift silently masks its distance. An SDE-2 engineer must see these risks during review, not after an incident.

Interviewers also use short numeric expressions to test whether a candidate reasons from Java rules instead of intuition borrowed from another language. The goal is not memorizing puzzles. It is building a reliable model for conversions and evaluation.

Learning-path note: Build the Java language model in this chapter first. After Java Foundations and Time and Space Complexity, continue to Number Systems Volume 01 for digit algorithms, base conversion, overflow-safe arithmetic, large-number strings, GCD/LCM, modular arithmetic, powers, roots, and number-oriented practice.

WHY IT WORKS | First-principles model

A Java variable is a named storage location with a compile-time type. A primitive variable contains a primitive value. A reference variable contains either `null` or a reference to an object or array; it does not contain the object itself.

Java has eight primitive types: `boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, and `double`. Except for `boolean`, these participate in numeric operations. Integral types use fixed-width two's-complement representations in modern Java's specified numeric model. Floating-point types follow IEEE 754 binary floating-point semantics.

A literal is source syntax for a value. Its spelling affects its type: `1` is an `int`, `1L` is a `long`, `1.0` is a `double`, and `1.0F` is a `float`. A conversion may preserve the mathematical value, approximate it, wrap it, or be rejected.

Specification boundary: Java specifies primitive ranges, conversions, and expression results. It does not expose a source-level memory address for a reference or guarantee that a local primitive occupies a particular number of physical bytes in a stack frame.

Core terminology

- **Declaration:** introduces a variable and its type, such as `long count;`.
- **Initialization:** supplies its first value.
- **Assignment conversion:** conversion allowed when storing an expression into a variable.
- **Widening conversion:** moves to a type with a broader representable domain, although `long` to `float` can lose precision.
- **Narrowing conversion:** explicitly converts to a smaller or different domain and may discard information.
- **Numeric promotion:** converts operands before unary or binary arithmetic.
- **Overflow:** a result outside an integral type's range; ordinary Java integer arithmetic wraps modulo the type's width.
- **Boxing/unboxing:** conversion between a primitive and its wrapper, such as `int` and `Integer`.
- **Compile-time constant:** a restricted expression of primitive or `String` type whose value the compiler can determine.

Detailed mechanics

Types, ranges, and defaults

`byte` is signed 8-bit, `short` signed 16-bit, `int` signed 32-bit, and `long` signed 64-bit. `char` is an unsigned 16-bit UTF-16 code unit, not necessarily a complete Unicode character. `float` is 32-bit binary floating point and `double` is 64-bit.

Fields and array elements receive default values: numeric zero, `false`, or `null`. Local variables do not. Java's definite-assignment analysis rejects a path that might read an uninitialized local.

JAVA

```
public class InitializationDemo {
    private int field;           // default 0

    public int choose(boolean useField) {
        int local;
        if (useField) {
            local = field;
        } else {
            local = 10;
        }
        return local;           // assigned on every reachable path
    }
}
```

`final` means a variable can be assigned only once. For a reference, it freezes the reference, not the referenced object's state. A blank `final` field must be assigned by every constructor.

`var`, available for local variables since Java 10, asks the compiler to infer a static type from the initializer. It is not dynamic typing, cannot be used for fields or method parameters, and requires an initializer. Prefer it when the inferred type remains obvious.

Literals

Integral literals can be decimal, hexadecimal (`0xFF`), binary (`0b1111`), or octal (`017`). Underscores may group digits but cannot occur at the literal's beginning, end, or next to a radix prefix or decimal point. An integer literal without `L` has type `int` and must satisfy the `int` literal range rules; use uppercase `L` for `long` because lowercase `l` resembles `1`.

JAVA

```
long nanos = 1_000_000_000L;
int permissions = 0b110_100_100;
int mask = 0xFF;
double ratio = 6.25e-2;           // 0.0625
float sample = 3.5F;
char newline = '\n';
char omegaCodeUnit = '\u03A9';
```

A special rule lets a constant `int` expression be assigned to `byte`, `short`, or `char` if the value fits. The same value held in a non-constant variable requires a cast.

JAVA

```
byte a = 100;           // constant fits
final int fixed = 100;
byte b = fixed;         // constant variable fits
int runtime = 100;
// byte c = runtime;    // compile-time error
byte c = (byte) runtime;
```

Promotion and evaluation

Unary numeric promotion changes `byte`, `short`, and `char` to `int`. Binary numeric promotion normally chooses `double`, then `float`, then `long`, otherwise `int`. Thus two `byte` operands add as `int`.

JAVA

```
byte x = 10;
byte y = 20;
int sum = x + y;
// byte bad = x + y;
x += y;           // compound assignment includes an implicit cast
```

Compound assignment `E1 op= E2` behaves roughly like `E1 = (T)(E1 op E2)`, while evaluating the left side once. That implicit narrowing can hide wraparound.

Integer division truncates toward zero. The remainder has the dividend's sign. Division by integral zero throws `ArithmeticException`, but floating-point division by zero produces infinity or NaN under IEEE 754 rules.

JAVA

```
System.out.println(-7 / 3);    // -2
System.out.println(-7 % 3);    // -1
System.out.println(1.0 / 0.0); // Infinity
System.out.println(0.0 / 0.0); // NaN
```

Overflow and exact arithmetic

Ordinary integer overflow does not throw. `Integer.MAX_VALUE + 1` becomes `Integer.MIN_VALUE`. Use `Math.addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, or conversion helpers such as `Math.toIntExact` when overflow is an invalid domain event.

Floating-point values model a finite subset of binary fractions. Decimal `0.1` is not exactly representable, so repeated addition can accumulate visible error. `BigDecimal` is appropriate for decimal business arithmetic, but construction matters: prefer a string or `BigDecimal.valueOf(double)` over `new BigDecimal(0.1)`.

NaN is unordered: every relational comparison with NaN is false, and `NaN == NaN` is false. Positive and negative zero compare equal with `==`, although some library operations distinguish their bit patterns.

Boxing and unboxing

Wrapper objects enable primitives in generic APIs. Boxing may allocate, but the language does not promise object identity. `Integer` commonly caches a small range, so `==` can appear to work and then fail. Compare wrapper values with `equals` or `Objects.equals`.

Unboxing `null` throws `NullPointerException` at the conversion site. A conditional expression or overloaded call can trigger unboxing less visibly than a direct assignment.

TRACE IT | Worked Java example

This invoice example treats cents as exact integral units and rejects overflow.

JAVA

```
import java.math.BigDecimal;

public class InvoiceMath {
    static long lineTotalCents(long unitCents, int quantity) {
        if (unitCents < 0 || quantity < 0) {
            throw new IllegalArgumentException("negative price or quantity");
        }
        return Math.multiplyExact(unitCents, (long) quantity);
    }

    static BigDecimal dollars(long cents) {
        return BigDecimal.valueOf(cents, 2);
    }

    public static void main(String[] args) {
        long first = lineTotalCents(1_299L, 3);
        long second = lineTotalCents(250L, 4);
        long total = Math.addExact(first, second);
        System.out.println(total);           // 4897
        System.out.println(dollars(total));  // 48.97
    }
}
```

The public unit is explicit in the names. Multiplication promotes quantity to `long` before evaluation, and exact helpers turn an impossible invoice into an exception rather than corrupted data.

TRACE IT | Execution or memory walkthrough

For `lineTotalCents(1_299L, 3)`, the literal `1_299L` is already `long`; `3` is `int`. The method receives copies of both primitive values. The range checks evaluate without conversion surprises. Casting `quantity` to `long` ensures 64-bit multiplication, producing `3_897L`.

The second line produces `1_000L`. `Math.addExact` returns `4_897L`. `BigDecimal.valueOf(4897, 2)` represents an unscaled integer of 4897 with scale 2, so its decimal value is exactly 48.97.

If multiplication exceeded `long`, `multiplyExact` would throw before a wrapped value reached persistence. No wrapper is needed on this hot path. The values may be held in registers or stack-frame slots, but that placement is a JVM implementation decision.

Complexity and performance

Primitive arithmetic is constant time for fixed-width types. Exact arithmetic helpers are also $O(1)$, with a small overflow check. `BigInteger` and `BigDecimal` costs grow with precision; their operations allocate immutable result objects and are unsuitable as automatic replacements for every primitive.

Boxing can create objects and increase garbage-collection pressure. Specialized primitives, primitive streams, or domain objects backed by `long` often help in high-volume paths. Measure before optimizing, because escape analysis and JIT compilation can remove some allocations.

HotSpot note: HotSpot may keep values in registers, scalar-replace non-escaping wrappers, and use cached wrapper instances. None of these optimizations changes Java's observable value semantics or makes wrapper identity safe to depend on.

WATCH OUT | Edge cases and common mistakes

- `long result = intA * intB;` performs `int` multiplication before widening. Cast an operand first.
- `Math.abs(Integer.MIN_VALUE)` is still negative because its positive magnitude does not fit. `Math.absExact` throws instead.
- A shift distance is masked: for `int`, only the low five bits are used; `x << 32` is effectively `x << 0`.
- `>>` sign-extends; `>>>` shifts zero bits into the high end.
- Narrowing a floating-point value truncates toward zero, clamps out-of-range finite values to the target endpoint, and converts NaN to zero.
- `0.1 + 0.2 == 0.3` is false in binary floating point.
- Autounboxing a nullable wrapper can fail during comparison, arithmetic, or overload selection.
- `char` arithmetic promotes to `int`; a surrogate pair requires two `char` values.
- Never use wrapper `==` for value comparison.

Production engineering notes

Put units in names and APIs: `timeoutMillis`, `amountCents`, and `bytesPerSecond`. Validate at boundaries, use exact helpers where overflow violates the domain, and test minimum, maximum, zero, and just-outside-range cases.

For money, decide whether fixed minor units or `BigDecimal` best fits currency and rounding requirements. Specify scale and `RoundingMode`; do not let defaults leak into accounting behavior. For scientific measurements, `double` is often correct, but define tolerances and NaN handling.

Be cautious when database, JSON, and Java numeric domains differ. A JavaScript client cannot exactly represent every `long` as a number. Database `DECIMAL` scale may reject or round a `BigDecimal`. Treat conversion as part of the contract.

TRY IT | Interview questions and model answers

Why does `long n = 1_000_000 * 1_000_000;` not produce one trillion?

Both operands are `int`, so multiplication occurs in 32 bits and overflows before assignment widens the wrapped result. Make either operand `long`, for example `1_000_000L * 1_000_000`.

Is widening always lossless?

No. Widening integral conversion preserves values except conversions to floating point can lose precision. A sufficiently large `long` cannot be represented exactly by `float` or even by `double`.

What does `final List<String> names` guarantee?

The variable cannot be reassigned after initialization. It does not make the list immutable; callers can still mutate the referenced list unless the object itself prevents mutation.

Why can `Integer a = 100; Integer b = 100; a == b` be true?

Boxing implementations reuse some wrapper instances, and the language requires caching for certain constant values. `==` asks about reference identity, not numeric equality, so code must not rely on that result outside the specified caching case. Use `a.equals(b)` or `Objects.equals`.

When would you choose `BigDecimal` over scaled `long`?

Choose `BigDecimal` when variable decimal scale, large magnitudes, or explicit decimal rounding are central. Scaled `long` is compact and fast when the unit and range are fixed. In either design, encode scale and rounding in the domain API.

TRY IT | Exercises

- 1 Predict the type and value of `byte b = 127; b += 1;`. Rewrite it so overflow throws.
- 2 Implement `average(long a, long b)` without overflowing their sum. Consider negative values.
- 3 Explain every conversion in `double d = 3 + 4L / 2.0F`.
- 4 Write tests for converting an incoming decimal price into cents with a required scale of two.
- 5 Find and fix nullable unboxing in `boolean active = map.get(id);`.
- 6 Compare `new BigDecimal("0.10")` and `new BigDecimal("0.1")` using both `equals` and `compareTo`; explain the different contracts.

RECAP | Chapter summary

Java's numeric rules are deterministic: literals have types, operands are promoted before evaluation, narrowing may discard information, and ordinary integral overflow wraps. Primitive variables hold values while reference variables hold references. `final` restricts assignment, `var` preserves static typing, and wrappers introduce identity and nullability concerns. Robust systems make units, range, precision, scale, and overflow policy visible in their APIs.

TRY IT | Revision checklist

- ☐ I can state all eight primitive types and their roles.
- ☐ I can distinguish default field values from definite assignment of locals.
- ☐ I can trace unary and binary numeric promotion.
- ☐ I cast before arithmetic when the operation needs a wider type.
- ☐ I know how integer division, remainder, shifts, and overflow behave.
- ☐ I can explain NaN, infinity, signed zero, and decimal approximation.
- ☐ I use exact arithmetic or `BigDecimal` when the domain requires it.
- ☐ I avoid wrapper identity comparisons and defend against null unboxing.

SDE-2 CORE CHAPTER

Chapter 3 - Operators, Expressions, and Control Flow

Learning objectives

By the end of this chapter, you should be able to:

- evaluate Java expressions using precedence, associativity, promotion, and short-circuit rules;
- distinguish statement forms from value-producing expressions;
- design readable branching and iteration with explicit invariants;
- use modern `switch` expressions safely; and
- identify control-flow bugs caused by side effects, fall-through, labels, and abrupt completion.

WHY IT WORKS | Why this matters at SDE-2

Control flow is where requirements become executable decisions. At SDE-2 level, correctness includes more than reaching the happy-path result. You must reason about whether a side effect runs, whether a loop terminates, whether cleanup executes, and whether all domain states are represented.

Interview code often compresses these questions into a loop or conditional. Production code spreads them across authorization, retries, state transitions, and pricing rules. A precise expression model makes both settings easier.

Focused series path: Finish this Java foundation first, then study Time and Space Complexity Volume 02. Continue to Number Systems Volume 01 for mathematical modulo and overflow-safe numeric patterns, Volume 04 for full bit manipulation, and Volume 05 for loop, pattern, and index-calculation drills. The stable filenames and recommended order are listed in [Java-SDE2-Interview-Preparation-Series-Index.pdf](#).

WHY IT WORKS | First-principles model

An expression is evaluated to produce a value, a variable, or a side effect. A statement controls execution: declaration, expression statement, block, branch, loop, transfer, or exception. Operators define how operand values are combined, but evaluation order determines when operands and side effects occur.

Java evaluates operand expressions from left to right. Precedence determines grouping, not temporal reordering. Parentheses should communicate intent even when precedence already makes code legal.

Control normally completes a statement and continues to the next one. `break`, `continue`, `return`, and `throw` complete abruptly and transfer control. Loops are best understood through an invariant: a property true before and after each iteration, plus a progress measure that proves termination.

Specification boundary: Java specifies left-to-right operand evaluation and short-circuit behavior. An optimizing JVM may rearrange machine instructions only if the observable Java behavior, including exceptions and synchronization semantics, is preserved.

Core terminology

- **Operand:** input expression to an operator.
- **Precedence:** which operator binds more tightly in source parsing.
- **Associativity:** how equal-precedence operators group.
- **Short-circuiting:** skipping the right operand when the left determines a boolean result.
- **Side effect:** observable state change, I/O, synchronization action, or exception.
- **Statement expression:** an expression allowed as a statement, such as assignment or method invocation.
- **Loop invariant:** claim maintained by every completed iteration.
- **Abrupt completion:** transfer caused by `break`, `continue`, `return`, or `throw`.
- **Exhaustiveness:** every possible selector value is handled by a `switch` expression.

Detailed mechanics

Operator families

Arithmetic operators are `+`, `-`, `*`, `/`, and `%`. Unary `+` and `-` apply numeric promotion. Prefix and postfix `++` and `--` both mutate a variable; postfix yields the old value and prefix yields the new value. Avoid embedding them in larger expressions.

Relational operators (`<`, `<=`, `>`, `>=`) compare numeric values. Equality (`==`, `!=`) compares primitive values or reference identity after permitted conversions. It does not invoke `equals`. Boolean logical operators are `!`, `&&`, `||`, `&`, `|`, and `^`. With booleans, `&` and `|` evaluate both sides; `&&` and `||` short-circuit.

Bitwise `&`, `|`, `^`, and `~` operate on promoted integral values. Shifts are `<<`, `>>`, and `>>>`. Assignment operators include simple `=` and compound forms such as `+=` and `&=`.

The conditional operator `condition ? whenTrue : whenFalse` evaluates exactly one branch. Its result type follows detailed rules involving numeric constants, boxing, reference least-upper-bound analysis, and `null`; do not assume it is simply the type of one textual branch.

Evaluation and short-circuiting

JAVA

```
static boolean valid(String value) {
    return value != null && !value.isBlank();
}
```

If `value` is `null`, `value != null` is false and `isBlank` is never called. Replacing `&&` with `&` would evaluate it and throw. Short-circuit operators also encode ordering, so avoid hiding expensive or state-changing operations in their right operands.

Java evaluates the target and arguments of a method invocation from left to right before calling the method. In `array[index()] += delta()`, the array reference and index are evaluated once, then the current element and right operand are combined.

Branches

An `if` condition must be `boolean` or `Boolean` subject to unboxing; Java does not treat integers or arbitrary references as truthy. Braces prevent misleading indentation and reduce maintenance risk.

Classic `switch` statements use colon labels and can fall through. Arrow labels do not fall through. A `switch` expression produces a value and must be exhaustive.

JAVA

```
enum Tier { FREE, STANDARD, PREMIUM }

static int retentionDays(Tier tier) {
    return switch (tier) {
        case FREE -> 7;
        case STANDARD -> 30;
        case PREMIUM -> 365;
    };
}
```

For an enum selector, covering every declared constant makes this source exhaustive. A compiler-generated defensive path can still throw if separately compiled code supplies a newer enum constant at runtime. `switch` on traditional supported reference selectors throws `NullPointerException` for `null` unless modern pattern-switch labels explicitly handle `null`.

A block arm in a switch expression uses `yield` to provide its value:

JAVA

```
int cost = switch (tier) {
    case FREE -> 0;
    case STANDARD -> {
        audit("standard pricing");
        yield 20;
    }
    case PREMIUM -> 50;
};
```

Loops and transfer

while checks before an iteration. **do-while** runs the body once before checking. A basic **for** has initialization, condition, and update components. Enhanced **for** delegates traversal to an iterator for **Iterable** objects or uses array indexing semantics for arrays.

JAVA

```
static int firstAtLeast(int[] values, int threshold) {
    for (int i = 0; i < values.length; i++) {
        if (values[i] >= threshold) {
            return i;
        }
    }
    return -1;
}
```

break exits the nearest loop or switch statement. **continue** moves to the next loop iteration; in a basic **for**, the update expression still runs. A labeled **break** or **continue** can target an enclosing statement or loop, but extraction into a method is often clearer.

return exits a method, and **throw** propagates an exception. A **finally** block runs during most abrupt transfers, but a **return** or **throw** from **finally** can replace the original outcome and should be avoided.

TRACE IT | Worked Java example

The following retry policy combines an exhaustive switch expression with a loop whose termination is explicit.

JAVA (continued 1/2)

```
import java.time.Duration;

public class RetryPolicy {
    enum Result { SUCCESS, RETRYABLE, PERMANENT_FAILURE }

    interface Operation {
        Result run(int attempt);
    }

    static boolean execute(int maxAttempts, Operation operation) {
        if (maxAttempts < 1) {
            throw new IllegalArgumentException("maxAttempts must be positive");
        }

        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            Result result = operation.run(attempt);
            boolean stop = switch (result) {
                case SUCCESS -> true;
                case PERMANENT_FAILURE -> true;
            };
        }
    }
}
```

JAVA (continued 2/2)

```

        case RETRYABLE -> attempt == maxAttempts;
    };

    if (stop) {
        return result == Result.SUCCESS;
    }
    Duration delay = Duration.ofMillis(100L * attempt);
    System.out.println("retry after " + delay);
}
throw new AssertionError("loop is exhaustive");
}

public static void main(String[] args) {
    boolean ok = execute(3, attempt ->
        attempt < 3 ? Result.RETRYABLE : Result.SUCCESS);
    System.out.println(ok);
}
}

```

The last throw documents an unreachable condition based on the validated positive limit and loop exits. In a real service, sleeping should respect interruption and policy jitter; the example focuses on control flow.

TRACE IT | Execution or memory walkthrough

`maxAttempts` is validated once. Iteration one invokes the operation with `1`, which returns `RETRYABLE`. The switch selects its third arm; `attempt == maxAttempts` is false, so `stop` is false. A 100 millisecond duration is printed.

Iteration two follows the same path and prints 200 milliseconds. The for-update increments `attempt` after each normally completed body. Iteration three returns `SUCCESS`; the first switch arm yields true. The `if` executes and returns `true`, so neither the update nor another condition check runs.

At every loop entry, the invariant is `1 <= attempt <= maxAttempts`. Progress is the increment of `attempt`, and the upper bound proves termination. Only one switch arm executes on each iteration.

Complexity and performance

Expression operators on fixed-width primitives are $O(1)$. A branch is $O(1)$, while loop cost is the iteration count multiplied by body cost. The example makes at most `maxAttempts` operation calls, so it is $O(n)$ time and $O(1)$ extra space, excluding work performed by the operation.

Branch shape can influence CPU prediction, and switch compilation may use tables, lookups, or comparison chains. Such choices are implementation details and rarely justify distorting domain code. First minimize unnecessary work and choose an appropriate algorithm; then profile representative loads.

HotSpot note: HotSpot may inline branches, eliminate unreachable paths, unroll loops, or compile a switch to different machine structures based on profiling. Source semantics and exception behavior remain the contract.

WATCH OUT | Edge cases and common mistakes

- `a < b < c` is illegal; the first comparison is boolean. Write `a < b && b < c`.
- `if (flag = true)` assigns and then tests. Prefer `if (flag)` and enable static analysis.
- `&` and `|` do not short-circuit boolean operands.
- Post-increment returns the previous value: `a[i++] = i` is legal but hard to review.
- A missing `break` in a colon-style switch falls through, sometimes intentionally and often accidentally.
- A semicolon after `if` or `while` creates an empty controlled statement.
- Mutating a collection structurally during enhanced-for traversal usually triggers fail-fast behavior; use the iterator's removal operation or another strategy.
- Comparing references with `==` does not compare object content.
- Floating-point NaN makes all relational comparisons false, including both `x < y` and `x >= y`.
- A `continue` can skip state updates placed in the loop body and create nontermination.
- Deep labels often signal that the body should become a named method.

Production engineering notes

Keep decision tables centralized. An exhaustive enum switch can make a missing business state a compiler concern, but account for enum evolution across independently deployed components. For workflows with many transitions, model legal state transitions explicitly rather than building a maze of conditionals.

Separate predicates from side effects. A name such as `isEligible` should not write to a database. This makes short-circuiting safe to understand and lets tests cover decision logic deterministically.

Loops over external work need budgets: maximum attempts, deadlines, pagination termination, cancellation, and metrics. Check thread interruption when doing blocking or long-running work. In retry code, distinguish an attempt count from a retry count and test off-by-one boundaries.

TRY IT | Interview questions and model answers

What is the difference between precedence and evaluation order?

Precedence decides how tokens group, as multiplication grouping before addition. Java then evaluates operand expressions left to right. Parentheses can change grouping but do not authorize arbitrary operand reordering.

When should `&` be used with booleans?

Only when both boolean operands intentionally must be evaluated, which is uncommon in application predicates. `&&` is usually safer because it encodes conditional evaluation. `&` remains useful as a bitwise operator for integral masks.

How is a switch expression different from a classic switch statement?

A switch expression produces a value, must be exhaustive, and commonly uses non-fall-through arrow arms. A classic colon-style statement controls execution and may fall through. A block expression arm supplies its result with `yield`.

How do you prove a loop terminates?

State an invariant and a bounded progress measure. For an index loop, show that the index begins in range, advances on every continuing path, and is bounded by a finite length. External loops also need a deadline or other finite budget.

Does `finally` always execute?

It executes for normal and most abrupt completion of its `try`, but not if the JVM process terminates or cannot continue, such as `System.exit`, a crash, or forced kill. It is also dangerous for `finally` to return because that replaces an earlier return or exception.

TRY IT | Exercises

- 1 Trace `int x = 1; int y = x++ + ++x;` using left-to-right evaluation. Then rewrite it for clarity.
- 2 Convert a fall-through day-category switch into an exhaustive switch expression.
- 3 State an invariant and progress measure for binary search.
- 4 Implement pagination that stops on an empty page, a repeated cursor, a maximum page count, or interruption.
- 5 Explain why `user != null & user.active()` is unsafe and whether changing to `&&` fully solves the design problem.
- 6 Write a truth table for `&&`, `||`, `^`, and equality on booleans, noting which operands execute.

RECAP | Chapter summary

Java expressions follow fixed grouping, conversion, left-to-right evaluation, and short-circuit rules. Statements organize those expressions into branches, loops, and abrupt transfers. Modern switch expressions make value mapping and exhaustiveness explicit. Reliable iteration requires an invariant, a progress measure, and a finite budget. Readable control flow minimizes hidden side effects and makes every exit path intentional.

TRY IT | Revision checklist

- ☐ I distinguish precedence, associativity, and evaluation order.
- ☐ I know which boolean operators short-circuit.
- ☐ I avoid mutation hidden inside complex expressions.
- ☐ I can use switch expressions, arrow arms, and `yield`.
- ☐ I understand fall-through and null selector behavior.
- ☐ I can state a loop invariant and prove termination.
- ☐ I understand `break`, `continue`, `return`, and `throw` as abrupt completion.
- ☐ I place explicit budgets around retries and external iteration.

SDE-2 CORE CHAPTER

Chapter 4 - Methods, Overloading, Varargs, and Pass-by-Value

Learning objectives

By the end of this chapter, you should be able to:

- explain method declarations, signatures, return contracts, and invocation;
- predict compile-time overload selection and runtime overriding separately;
- use varargs without ambiguity, mutation leaks, or heap-pollution surprises;
- demonstrate Java's pass-by-value rule for primitives and references; and
- design methods with cohesive responsibilities and stable API boundaries.

WHY IT WORKS | Why this matters at SDE-2

Methods are Java's primary unit of behavior and API design. Subtle errors arise when overloaded APIs accept `null`, generic and varargs methods interact, or a developer expects a callee to replace the caller's reference. These issues appear in framework APIs, backward-compatible library evolution, and interviews about dispatch.

At SDE-2 level, a correct method is not enough. Its contract must communicate nullability, mutation, failure, ownership, cost, and concurrency expectations. Overload sets must remain predictable as the codebase evolves.

WHY IT WORKS | First-principles model

A method declaration gives code a name, parameter list, return type, modifiers, optional type parameters, and declared checked exceptions. On invocation, Java evaluates the receiver and arguments, creates a new invocation context, and initializes each parameter with a copy of its corresponding argument value.

For a primitive argument, the copied value is the primitive. For a reference argument, the copied value is the reference. Caller and callee can therefore refer to the same mutable object, but assigning the callee's parameter cannot replace the caller's variable. Java has no pass-by-reference parameter mode.

Invocation has two major decisions. The compiler selects an applicable declaration using the compile-time types and overload rules. If the chosen method is an overridable instance method, runtime dynamic dispatch selects the most specific override for the receiver's actual class.

Specification boundary: Java is always pass-by-value. References are values that can be copied. The JVM's physical calling convention may use registers, stack locations, or optimized inlining, but these do not create source-language pass-by-reference semantics.

Core terminology

- **Formal parameter:** variable declared by a method.
- **Argument:** expression supplied at a call site.
- **Signature:** method name plus type parameters after adaptation and formal parameter types; return type is not part of overload identity.
- **Overloading:** same method name with different parameter lists, resolved at compile time.
- **Overriding:** subclass instance method supplies behavior for an inherited method, dispatched at runtime.
- **Arity:** number of arguments or parameters.
- **Varargs:** variable-arity final parameter declared with `T...` and represented as an array.
- **Applicable method:** overload that can accept the arguments in a particular invocation phase.
- **Covariant return:** override narrows a reference return type.
- **Bridge method:** compiler-generated adapter that preserves polymorphism after type erasure.

Detailed mechanics

Declaration and contracts

JAVA

```
public static <T> T require(T value, String message) {  
    if (value == null) {  
        throw new IllegalArgumentException(message);  
    }  
    return value;  
}
```

`public` `static` are modifiers, `<T>` declares a method type parameter, the next `T` is the return type, and two formal parameters follow. A non-`void` method must not complete normally without returning a compatible value. Returning a reference does not copy the object.

Parameter names are generally not part of binary method identity. Overloading cannot differ only by return type, parameter names, `throws` clauses, or generic type arguments erased to the same signature.

Overload resolution phases

Conceptually, Java searches in phases to preserve compatibility:

- 1 fixed-arity methods applicable without boxing or unboxing;
- 2 fixed-arity methods allowing boxing and unboxing; then
- 3 variable-arity methods.

Within a phase, it chooses a most-specific method. Widening a primitive is preferred over boxing because it succeeds in an earlier phase.

JAVA

```
static String choose(long value) { return "long"; }
static String choose(Integer value) { return "Integer"; }
static String choose(int... values) { return "varargs"; }

System.out.println(choose(5)); // long
```

The `int` literal widens to `long` during strict fixed-arity invocation. Boxing and varargs are considered only if no earlier-phase candidate works.

`null` can make an overload ambiguous when unrelated reference parameter types are equally specific:

JAVA

```
static void send(String value) {}
static void send(StringBuilder value) {}
// send(null); // ambiguous
```

Casting documents the intended overload, but a public API that repeatedly needs such casts probably has a poor overload set.

Static selection and dynamic dispatch

JAVA

```
class Parent {
    String label(Object value) { return "Parent:Object"; }
}
class Child extends Parent {
    @Override String label(Object value) { return "Child:Object"; }
    String label(String value) { return "Child:String"; }
}

Parent p = new Child();
System.out.println(p.label("x")); // Child:Object
```

The compile-time type of `p` exposes only `Parent.label(Object)`, so overload selection chooses that signature. Runtime dispatch then finds `Child`'s override of `label(Object)`. It does not restart overload selection to discover `label(String)`.

Static methods are hidden, not overridden. Private methods are not inherited as overridable members. Constructors are not methods and are not inherited. `final` instance methods cannot be overridden.

Varargs

`void log(String... messages)` is compiled using a `String[]` parameter. A call such as `log("a", "b")` causes the call site to create an array, while `log(existingArray)` can pass the existing array directly. This creates an ownership risk if the callee mutates it.

Only the final formal parameter may be varargs. A method is treated as fixed arity first, which is why passing an existing array does not require another wrapper array.

Generic varargs can create non-reifiable array types. The compiler warns because the runtime array cannot fully check its element type. `@SafeVarargs` is a promise by the author that an eligible method does not perform unsafe operations on or expose its varargs array. It suppresses warnings; it does not make unsafe code safe.

JAVA

```
@SafeVarargs
static <T> java.util.List<T> immutableList(T... values) {
    return java.util.List.copyOf(java.util.Arrays.asList(values));
}
```

This method reads the array and returns an independent unmodifiable list. It does not store incompatible values or expose the array.

Pass-by-value and mutation

JAVA

```
static void change(StringBuilder builder, int number) {
    builder.append("!");
    builder = new StringBuilder("replacement");
    number = 99;
}

StringBuilder text = new StringBuilder("hello");
int count = 1;
change(text, count);
System.out.println(text); // hello!
System.out.println(count); // 1
```

The copied reference lets `change` mutate the original builder. Reassigning the parameter changes only the callee's local copy. The copied integer is similarly independent.

TRACE IT | Worked Java example

This API avoids an ambiguous overload set and defensively handles its variable-arity input.

JAVA

```
import java.time.Instant;
import java.util.List;
import java.util.Objects;

public final class AuditEvent {
    private final String action;
    private final Instant occurredAt;
    private final List<String> tags;

    private AuditEvent(String action, Instant occurredAt, List<String> tags) {
        this.action = Objects.requireNonNull(action);
        this.occurredAt = Objects.requireNonNull(occurredAt);
        this.tags = List.copyOf(tags);
    }

    public static AuditEvent now(String action, String... tags) {
        Objects.requireNonNull(tags, "tags");
        return at(action, Instant.now(), List.of(tags.clone()));
    }

    public static AuditEvent at(
        String action, Instant occurredAt, List<String> tags) {
        return new AuditEvent(action, occurredAt, tags);
    }

    public List<String> tags() {
        return tags;
    }
}
```

Named factories distinguish convenience construction from deterministic construction. The full method takes a collection rather than adding many same-name overloads. `List.copyOf` validates elements and creates an unmodifiable snapshot when necessary.

TRACE IT | Execution or memory walkthrough

For `AuditEvent.now("login", "security", "user")`, the call site constructs a `String[]` containing two references. `now` receives a copy of the array reference. It checks that reference, clones the array, and creates a list view over the clone. The clone prevents a caller-supplied array from being affected by internal processing.

`at` receives copies of three references. The constructor receives another set of copied references. `List.copyOf` rejects null elements and returns an unmodifiable list whose contents cannot be changed through the original list. The final fields receive references once; immutability also depends on the referenced values. `String` and `Instant` are immutable, and the tag list is unmodifiable with immutable string elements.

The JVM may inline these calls and eliminate some intermediate allocations, but semantically every parameter is initialized from a copied value.

Complexity and performance

A direct method invocation is $O(1)$ before considering method work. Virtual dispatch is also constant time conceptually. Varargs array creation is $O(n)$ time and space in the argument count; cloning adds another $O(n)$. `List.copyOf` may copy in $O(n)$, depending on its input.

Tiny methods can be inlined by a JIT, so architectural clarity usually matters more than avoiding calls. Varargs on a high-frequency logging or serialization path can allocate even when the callee does little. Offer collection, builder, or fixed-arity forms only when measurement and API clarity support them.

HotSpot note: HotSpot uses profiling and class-hierarchy knowledge to inline or devirtualize calls and may scalar-replace short-lived varargs arrays. These are optimizations, not guarantees that allocation has no cost.

WATCH OUT | Edge cases and common mistakes

- Return type alone cannot distinguish overloads.
- `null` can be ambiguous or select a surprising most-specific reference overload.
- Widening, boxing, and varargs do not have equal priority.
- Autounboxing during invocation can throw `NullPointerException`.
- Overloading a method in a subclass does not override a different parameter signature.
- A static call is selected using compile-time type, even when invoked through an instance expression.
- Passing an existing array to a varargs method may let the callee mutate caller-owned data.
- Calling a varargs method with plain `null` can mean a null array rather than one null element and may produce a warning. Cast explicitly if unavoidable.
- Generic varargs plus array writes can produce heap pollution and delayed `ClassCastException`.
- A method that returns an internal mutable collection leaks representation.
- Recursive methods need both a base case and progress toward it; large depth can exhaust the thread stack.

Production engineering notes

Prefer cohesive methods named for business intent. Keep validation near boundaries and make ownership explicit: snapshot, view, mutable result, or caller-owned input. Public overloads should share one canonical implementation so fixes do not diverge.

Evolving an API requires source, binary, and behavioral compatibility analysis. Adding an overload can make existing source calls ambiguous or change which declaration recompilation selects. Changing a return type is not overloading; covariant changes are limited to overriding. Framework reflection may also observe bridge, synthetic, or parameter metadata differently.

Avoid boolean parameter clusters such as `send(data, true, false)`. An options object, enum, or separate named operation communicates more. Document nullability and checked failures; do not make callers reverse-engineer them from implementation.

TRY IT | Interview questions and model answers

Is Java pass-by-reference for objects?

No. The argument value is a reference, and Java copies that value into the parameter. Callee and caller can use their copies to reach the same object, so mutation is visible. Parameter reassignment is not visible to the caller.

What is selected at compile time versus runtime?

Overload resolution and static method selection use compile-time types. For a selected overridable instance signature, runtime dispatch chooses the override associated with the actual receiver class.

Why does primitive widening beat boxing?

Invocation resolution first searches fixed-arity methods applicable without boxing. Only if none is suitable does it search with boxing, followed later by varargs. This phased design preserves older method-selection behavior.

What does `@SafeVarargs` guarantee?

It is an assertion by the method author that operations on the generic varargs parameter are type safe. The compiler trusts it for warnings; neither the annotation nor runtime verifies the implementation. Apply it only to eligible non-overridable methods and audit the body.

Can an override throw broader checked exceptions?

No. It may declare the same checked exceptions or narrower subclasses, and it may omit them. Unchecked exceptions are not restricted in the same way. This preserves the caller's compile-time exception contract.

TRY IT | Exercises

- 1 Create overloads for `f(long)`, `f(Integer)`, and `f(int...)`; predict calls with `1`, `Integer.valueOf(1)`, and an `int[]`.
- 2 Trace a base-reference/subclass-object example containing one overload and one override.
- 3 Rewrite a `swap(Object a, Object b)` attempt and explain why it cannot swap caller variables. Design a return-value alternative.
- 4 Audit a generic varargs method for array writes, escaping references, and unsafe casts.
- 5 Refactor a method with four boolean parameters into a clear options type.
- 6 Design tests proving that a method does not retain or mutate a caller's input array.

RECAP | Chapter summary

Methods establish behavioral boundaries. Java initializes every parameter with a copied argument value, including copied references. Overload resolution is compile-time and phased; overriding is runtime dispatch of a previously selected instance signature. Varargs are arrays with allocation, ownership, and generic safety implications. Strong APIs minimize ambiguous overloads, document mutation and failure, and funnel convenience entry points into one canonical implementation.

TRY IT | Revision checklist

- ☐ I can identify what is and is not part of a method signature.
- ☐ I understand the widening, boxing, and varargs resolution phases.
- ☐ I separate overload selection from override dispatch.
- ☐ I can demonstrate pass-by-value using a mutable object and reassignment.
- ☐ I treat varargs arrays as ordinary arrays with ownership concerns.
- ☐ I know why generic varargs can cause heap pollution.
- ☐ I design overload sets that remain clear with `null` and future evolution.
- ☐ I document method contracts for mutation, nullability, cost, and failure.

SDE-2 CORE CHAPTER

Chapter 5 - Arrays, Strings, Text Blocks, and Unicode

Learning objectives

By the end of this chapter, you should be able to:

- model Java arrays as fixed-length, reified reference objects;
- distinguish shallow copying, deep copying, aliasing, and covariance hazards;
- explain `String` immutability, pooling, concatenation, and comparison;
- write and reason about text blocks; and
- process Unicode code points instead of assuming one `char` equals one user-visible character.

WHY IT WORKS | Why this matters at SDE-2

Arrays and strings sit on almost every backend boundary: network buffers, database values, identifiers, JSON, logs, and security checks. Errors here become data corruption, accidental quadratic work, or Unicode vulnerabilities. A senior engineer must recognize ownership and encoding assumptions as part of an API contract.

Interviews use arrays and strings heavily because their syntax is simple while their semantics expose fundamentals: references, mutability, indexing, complexity, equality, and representation.

WHY IT WORKS | First-principles model

An array is an object with a runtime component type and immutable length. Its elements are mutable unless prevented by external design. A variable such as `int[] values` contains a reference, not the elements themselves. Multidimensional array syntax represents arrays whose elements are array references; rows may have different lengths or be null.

A `String` is an immutable sequence of UTF-16 code units. A `char` is one 16-bit code unit. Some Unicode code points use one code unit, others use a surrogate pair, and a visible grapheme can combine several code points. Therefore `length()`, code-point count, and what a user sees are different concepts.

A text block is source syntax for a `String`. It improves multiline readability but does not introduce a different runtime type.

Specification boundary: Java specifies array bounds checks, runtime store checks, `String` behavior, and UTF-16-based APIs. It does not guarantee a particular internal `String` field layout or byte encoding used inside a JVM implementation.

Core terminology

- **Component type:** type of an array's elements.
- **Reified:** runtime retains enough component-type information to check array stores.
- **Covariance:** `Sub[]` is a subtype of `Super[]` when `Sub` is a subtype of `Super`.
- **Aliasing:** multiple references reach the same mutable object.
- **Shallow copy:** copies top-level values or references, not referenced nested objects.
- **Interning:** canonicalizing equal strings in a JVM-managed pool.
- **Code unit:** unit used by an encoding; Java `char` is a UTF-16 code unit.
- **Code point:** Unicode numeric value such as U+1F680.
- **Grapheme cluster:** sequence perceived as one user-visible character.
- **Incidental indentation:** whitespace text blocks remove based on delimiter and content placement.

Detailed mechanics

Array creation and initialization

JAVA

```
int[] counts = new int[4];           // [0, 0, 0, 0]
String[] names = {"Ada", "Linus"};
int[][] matrix = new int[3][];
matrix[0] = new int[] {1, 2};
matrix[1] = new int[] {3};
// matrix[2] remains null
```

Every array element is default-initialized. Length is available through the final-like `length` field and cannot change. Indexes range from zero through `length - 1`; any other index throws `ArrayIndexOutOfBoundsException`.

Array declarations allow brackets after the type or name, but placing them with the type avoids mixed declarations. Generic array creation such as `new List<String>[10]` is illegal because erased generic element types cannot support the array's runtime store check.

Covariance is checked dynamically:

JAVA

```
Number[] numbers = new Integer[2];
numbers[0] = 10;
// numbers[1] = 2.5; // ArrayStoreException
```

The variable permits a `Double` by its static component type, but the actual object is `Integer[]`, so the store fails. Generic collections use invariant static checking and usually make this class of error impossible.

Copying and comparison

Assignment copies an array reference. `clone`, `Arrays.copyOf`, and `System.arraycopy` create or populate top-level arrays, but nested references remain shared.

JAVA

```
int[][] original = {{1, 2}, {3, 4}};
int[][] shallow = original.clone();
shallow[0][0] = 99;
System.out.println(original[0][0]); // 99
```

Arrays inherit identity-based `equals` and `hashCode` from `Object`. Use `Arrays.equals` for one-dimensional content, `Arrays.deepEquals` for nested arrays, and matching hash helpers. Printing an array directly does not format its elements; use `Arrays.toString` or `deepToString`.

String identity, value, and pool

String literals and constant string expressions are interned. Runtime-created strings need not share identity.

JAVA

```
String a = "java";
String b = "ja" + "va";           // compile-time constant
String part = "ja";
String c = part + "va";           // runtime concatenation
System.out.println(a == b);       // true
System.out.println(a == c);       // generally false; do not rely on identity
System.out.println(a.equals(c));  // true
```

Use `equals` for exact content and `equalsIgnoreCase` only when its locale-independent comparison matches the domain. For natural-language case conversion, specify a `Locale`. For protocol identifiers, often use `Locale.ROOT`. Unicode normalization is separate from case folding; visually equivalent strings can have different code-point sequences.

Because strings are immutable, operations such as `substring`, `replace`, and `toUpperCase` return a string and do not modify the receiver. `StringBuilder` provides a mutable sequence for single-threaded assembly. `StringBuffer` synchronizes individual operations but rarely solves a higher-level concurrency design.

Modern compilers use `invokedynamic`-based concatenation strategies for many `+` expressions, so simple one-shot concatenation is readable and efficient. Repeated `result += item` in a loop can still create work proportional to all accumulated content; use a builder or joining collector.

Text blocks

Text blocks became permanent in Java 15 and are available in Java 17 and 21. The opening `"""` must be followed by a line terminator. The compiler removes incidental indentation, normalizes line terminators, and processes escape sequences.

JAVA

```
String json = """
    {
        "name": "Ada",
        "active": true
    }
    """;
```

This value normally ends with a newline because content ends before the closing delimiter. Place the closing delimiter immediately after content, or use a line-continuation escape, when no terminal newline is desired. The `\s` escape preserves a space explicitly. `String.stripIndent()` and `translateEscapes()` expose related runtime transformations, but a compiled text block has already undergone compiler processing.

Never use text blocks as a substitute for SQL parameter binding or HTML/JSON escaping. They improve source presentation, not data safety.

Unicode processing

`String.length()` returns UTF-16 code units. `codePointCount` counts Unicode code points. Iterate safely with code-point-aware methods:

JAVA

```
String value = "A\uD83D\uDE80"; // A plus rocket
System.out.println(value.length()); // 3 code units
System.out.println(value.codePointCount(0, value.length())); // 2

value.codePoints().forEach(cp ->
    System.out.printf("U+%04X%n", cp));
```

`codePointAt` combines a valid surrogate pair. If input contains an unpaired surrogate, it returns that surrogate value; Java strings can contain ill-formed UTF-16 sequences. Encoders must decide whether to replace, report, or ignore malformed input.

Unicode escapes such as `\u000A` are processed very early in Java source translation, even in comments. A Unicode escape that becomes a line terminator can change tokenization or make source illegal. Prefer ordinary escapes like `\n` for controls inside literals.

TRACE IT | Worked Java example

This function reverses code points rather than UTF-16 code units and preserves supplementary characters.

JAVA

```
public class UnicodeReverse {
    static String reverseCodePoints(String input) {
        int[] points = input.codePoints().toArray();
        StringBuilder result = new StringBuilder(input.length());
        for (int i = points.length - 1; i >= 0; i--) {
            result.appendCodePoint(points[i]);
        }
        return result.toString();
    }

    public static void main(String[] args) {
        String input = "A\uD83D\uDE80B";
        System.out.println(reverseCodePoints(input)); // B, rocket, A
    }
}
```

This is code-point correct, not grapheme-cluster correct. Reversing combining marks separately can still produce surprising visible output. User-perceived text segmentation requires a boundary algorithm suitable for the Unicode version and locale.

TRACE IT | Execution or memory walkthrough

The input contains four UTF-16 code units: **A**, a high surrogate, a low surrogate, and **B**. `codePoints()` combines the surrogate pair and emits three `int` values. `toArray` allocates an `int[3]`.

The loop begins at index two and appends **B**. It then calls `appendCodePoint` for the rocket value, which emits two UTF-16 code units, followed by **A**. `toString` creates the immutable result. The original string is unchanged.

No bounds error occurs because the loop starts at `points.length - 1`, continues while nonnegative, and decrements. For an empty input it starts at -1 and performs zero iterations.

Complexity and performance

Array indexing is $O(1)$; traversal and copying are $O(n)$. `System.arraycopy` is still $O(n)$, though JVMs optimize its constant factors. A rectangular r by c traversal is $O(rc)$; a jagged traversal is proportional to actual elements.

Most string searches and transformations are $O(n)$ in code units, with algorithm-specific exceptions. Concatenating into an immutable accumulator inside a loop can be $O(n^2)$ in total copied content. A pre-sized `StringBuilder` usually makes assembly $O(n)$.

The reverse example is $O(n)$ time and $O(n)$ additional space. A more intricate backward code-point traversal can avoid the `int[]`, but clarity is often worth the allocation unless profiling identifies the path.

HotSpot note: Current HotSpot implementations may store strings compactly using a byte array plus an encoding marker when contents permit. This is not a public `String` contract and should not influence correctness assumptions.

WATCH OUT | Edge cases and common mistakes

- Array assignment aliases; it does not copy elements.
- `clone` on a nested or object array is shallow.
- Covariant array stores can fail at runtime.
- `Arrays.asList(primitiveArray)` creates a one-element list containing that array, not a list of boxed primitives.
- `array.length`, `string.length()`, and `collection.size()` use different syntax.
- `==` compares string references; `equals` compares content.
- Calling `toString` on a null reference fails; `String.valueOf` renders it as `"null"`, which may hide missing data.
- Splitting on a regex metacharacter requires regex escaping, and `split` discards trailing empty strings by default unless a negative limit is used.
- A supplementary code point occupies two `char` values.
- Code-point correctness does not guarantee grapheme or locale correctness.
- Default-charset conversions vary by environment; specify `StandardCharsets.UTF_8` at byte boundaries.
- Text blocks still process escapes and normally include a trailing newline based on delimiter placement.

Production engineering notes

Specify ownership for arrays and mutable builders. Defensive-copy untrusted mutable input when retaining it, and do not expose internal arrays directly. For secrets, a mutable `char[]` can be cleared, but copies may still exist in libraries and memory; do not overstate that protection.

Define charset at every byte/text boundary. Validate malformed input intentionally. Normalize only when the domain requires it, and decide where canonicalization occurs so database keys, caches, signatures, and authorization checks agree.

Use locale-aware libraries for human language and locale-neutral rules for protocols. Length limits must state whether they mean bytes, UTF-16 units, code points, or grapheme clusters. Database column limits and API validators often count differently.

TRY IT | Interview questions and model answers

Why are arrays covariant while generics are invariant?

Arrays predate generics and retain runtime component types, so Java permits covariance and protects stores dynamically. Generic type arguments are normally erased and checked statically; invariance prevents inserting the wrong subtype without waiting for a runtime failure.

Does `String.length()` count characters?

It counts UTF-16 code units. It may differ from Unicode code points and from user-perceived grapheme clusters. The correct metric depends on the requirement.

Why is `String` immutable?

Immutability enables safe sharing, stable hashing, pooling, and simpler security boundaries. It does not mean every string operation is allocation-free, and sensitive values can remain in memory beyond application control.

What does a text block produce?

An ordinary `String`. The compiler determines content through indentation removal, line-ending normalization, and escape processing. Delimiter placement controls whether a final newline remains.

Is `StringBuilder` always required for concatenation?

No. One expression using `+` is clear and compilers optimize it effectively. Use a builder when incrementally accumulating in loops or when capacity and append behavior need explicit control.

TRY IT | Exercises

- 1 Deep-copy a jagged `int[][]` while preserving null rows.
- 2 Demonstrate an `ArrayStoreException`, then replace the API with a generic collection.
- 3 Write tests showing trailing-newline and indentation behavior for three text blocks.
- 4 Count code units, code points, and expected grapheme clusters in strings containing an emoji and a combining mark.
- 5 Implement UTF-8 encode/decode with a `CharsetEncoder` configured to report malformed input.
- 6 Replace quadratic loop concatenation with a pre-sized `StringBuilder` and justify the capacity estimate.

RECAP | Chapter summary

Arrays are fixed-length, reified, mutable objects; assignment aliases them, covariance can defer type errors, and top-level copy operations are shallow for reference elements. Strings are immutable UTF-16 code-unit sequences and require value comparison. Text blocks are readable source syntax for ordinary strings. Correct text handling separates bytes, code units, code points, graphemes, normalization, locale, and user-visible length.

TRY IT | Revision checklist

- ☐ I can explain array reification, covariance, and store checks.
- ☐ I distinguish aliasing, shallow copying, and deep copying.
- ☐ I use content helpers to compare and print arrays.
- ☐ I compare strings by value and understand interning boundaries.
- ☐ I know when repeated concatenation becomes expensive.
- ☐ I can predict text-block indentation and final-newline behavior.
- ☐ I distinguish UTF-8 bytes, UTF-16 units, code points, and graphemes.
- ☐ I specify charset, locale, normalization, and length units at boundaries.

SDE-2 CORE CHAPTER

Chapter 6 - Practice Lab A - Language and Execution

Attempt these without the solution guide. Record both your answer and the rule that supports it.

Start here: work in four short passes instead of treating this as one exam. First answer K01-K25 aloud. Next predict O01-O19 before running anything. Then repair D01-D19 in a scratch file and compile each fix. Finish by implementing C01-C19 and discussing F01-F13 with explicit contracts and edge cases.

Keep an error log with three columns: the mistaken prediction, the Java rule that corrects it, and one boundary input that exposes the mistake. A correct answer without the rule is recall; a correct answer with the rule is reusable interview reasoning. Use Solution Studio A only after an honest attempt, then retry missed items the next day without looking.

Knowledge check

- **K01 [Foundation]** Distinguish source code, bytecode, and machine execution.
- **K02 [Foundation]** Define JDK, runtime/JRE, and JVM without treating them as synonyms.
- **K03 [Foundation]** Explain each token in `public static void main(String[] args)`.
- **K04 [Foundation]** When must a public top-level class match its filename?
- **K05 [Foundation]** Distinguish declaration, initialization, assignment, and reassignment.
- **K06 [Foundation]** Why must a local variable be definitely assigned before reading?
- **K07 [Foundation]** List Java's eight primitive types and identify integral versus floating types.
- **K08 [Interview Core]** Why does primitive bit width not prove total object or field-layout size?
- **K09 [Foundation]** Distinguish a primitive value from a reference value.
- **K10 [Interview Core]** Explain why Java is always pass-by-value.
- **K11 [Foundation]** What is the type and range role of `char`?
- **K12 [Interview Core]** Why can `long result = intA * intB` still overflow?
- **K13 [Foundation]** Distinguish widening and narrowing conversion.
- **K14 [Interview Core]** Describe binary numeric promotion for `byte + byte`.
- **K15 [Foundation]** Explain integer division and remainder with negative operands.
- **K16 [Foundation]** Distinguish `&&` from boolean `&`.
- **K17 [Interview Core]** What hidden cast does compound assignment provide?
- **K18 [Foundation]** Distinguish prefix from postfix increment.
- **K19 [Foundation]** When is a guard clause clearer than nested `if` blocks?
- **K20 [Foundation]** Distinguish traditional switch fall-through from a switch expression.
- **K21 [Foundation]** Choose among `for`, `while`, `do-while`, and enhanced-`for`.
- **K22 [Interview Core]** What does `break` affect in nested loops?
- **K23 [Interview Core]** What makes an overload distinct, and why is return type insufficient?
- **K24 [Interview Core]** State the base case and recursive case in a recursive method preview.
- **K25 [SDE-2 Follow-up]** Explain the limits of Java's platform-independence claim.

Predict the output

- **O01 [Foundation]** Predict `System.out.println(5 / 2);`.
- **O02 [Foundation]** Predict `System.out.println(-5 % 2);`.
- **O03 [Interview Core]** Predict `(byte) 130` printed as an integer.
- **O04 [Interview Core]** Predict `long x = Integer.MAX_VALUE + 1; System.out.println(x);`.
- **O05 [Foundation]** Predict `int x=3; System.out.println(x++ + " " + x);`.
- **O06 [Foundation]** Predict `int x=3; System.out.println(++x + " " + x);`.
- **O07 [Interview Core]** Predict `short x=10; x+=5; System.out.println(x);`.
- **O08 [Foundation]** Predict `System.out.println('7' - '0');`.
- **O09 [Interview Core]** Predict `System.out.println('A' + 1);`.
- **O10 [Foundation]** Predict `boolean x=false; System.out.println(x && fail());` assuming `fail` would throw.
- **O11 [Interview Core]** Predict `int x=0; if (true || ++x>0){} System.out.println(x);`.
- **O12 [Foundation]** Predict a loop `for(int i=2;i>=0;i--) print(i)`.
- **O13 [Foundation]** Predict `int i=0; do { i++; } while(false); print(i)`.
- **O14 [Interview Core]** Predict a traditional switch with case 1 printing A and falling into case 2 printing B.
- **O15 [Foundation]** Predict `sum(2, 3)` when the method returns `left + right`.
- **O16 [Interview Core]** Predict a caller's primitive after a method increments only its parameter.
- **O17 [Interview Core]** Predict an object's field after a method mutates it through a copied reference.
- **O18 [Interview Core]** Predict a caller reference after a method reassigns only its parameter.
- **O19 [SDE-2 Follow-up]** Predict the evaluation result of `true ? 1 : 2.0` and state its type.

Debug the code

- **D01 [Foundation]** Repair a missing semicolon after `int count = 3`.
- **D02 [Foundation]** Repair reading an uninitialized local `int total`;
- **D03 [Foundation]** Repair a public class `Main` stored in `Program.java` without renaming the file.
- **D04 [Interview Core]** Repair `long area = width * height` for two arbitrary `int` inputs.
- **D05 [Interview Core]** Repair `short value = value + 1`.
- **D06 [Foundation]** Repair an average computed with integer division.
- **D07 [Interview Core]** Repair a null check that uses `text != null & !text.isEmpty()`.
- **D08 [Foundation]** Repair `if (score = 100)`.
- **D09 [Foundation]** Repair `for (int i=0; i<=array.length; i++)`.
- **D10 [Interview Core]** Repair a reverse loop that starts at `array.length`.
- **D11 [Foundation]** Repair a loop whose update moves away from its termination condition.
- **D12 [Interview Core]** Repair accidental switch fall-through.
- **D13 [Foundation]** Repair an enhanced-for loop that tries to use an undeclared index.
- **D14 [Interview Core]** Repair an overload pair differing only by return type.
- **D15 [Interview Core]** Repair an ambiguous `print(null)` call with `String` and `StringBuilder` overloads.
- **D16 [Foundation]** Repair a method declared `int` that reaches the end without returning.
- **D17 [Interview Core]** Repair recursion with no reachable base case.
- **D18 [Interview Core]** Repair `Math.abs(Integer.MIN_VALUE)` when a non-negative long magnitude is required.
- **D19 [SDE-2 Follow-up]** Repair a deeply nested validator using early returns while preserving behavior.

Small coding tasks

- **C01 [Foundation]** Print a greeting with a default name when no command-line argument is supplied.
- **C02 [Foundation]** Return the larger of two integers without using `Math.max`.
- **C03 [Foundation]** Classify an integer as negative, zero, or positive.
- **C04 [Foundation]** Return whether a year is a leap year.
- **C05 [Foundation]** Sum integers from 1 through `n` with a loop.
- **C06 [Foundation]** Traverse an array forward and print values.
- **C07 [Foundation]** Traverse an array backward.
- **C08 [Foundation]** Count values at even indexes.
- **C09 [Interview Core]** Compute a safe `long` product of two `int` values.
- **C10 [Interview Core]** Implement checked narrowing from `long` to `int`.
- **C11 [Foundation]** Implement a calculator with traditional switch and explicit breaks.
- **C12 [Foundation]** Implement the same calculator with a Java 14+ switch expression.
- **C13 [Foundation]** Write a method with parameters and a return value for rectangle area.
- **C14 [Interview Core]** Overload `clamp` for `int` and `long`.
- **C15 [Interview Core]** Demonstrate primitive pass-by-value with a caller assertion.
- **C16 [Interview Core]** Demonstrate shared object mutation through a parameter.
- **C17 [Interview Core]** Demonstrate why reference reassignment does not escape a method.
- **C18 [Interview Core]** Convert an ASCII digit only after validation.
- **C19 [SDE-2 Follow-up]** Refactor one large input-validation method into guard clauses and small helpers.

Interview follow-ups

- **F01 [Interview Core]** What changes when a compiler accepts source but runtime dependencies are incompatible?
- **F02 [SDE-2 Follow-up]** What does `--release 17` constrain and what does it not prove?
- **F03 [Interview Core]** When should overflow wrap, throw, saturate, or widen?
- **F04 [Interview Core]** Why is memorizing the full precedence table less useful than parentheses?
- **F05 [Interview Core]** Explain `&&` as both a correctness and cost tool.
- **F06 [SDE-2 Follow-up]** Defend a loop termination proof using a decreasing measure.
- **F07 [Interview Core]** When is a labeled break clearer than a helper method with early return?
- **F08 [Interview Core]** How can adding an overload break source compatibility?
- **F09 [Interview Core]** Why is `varargs` represented as an array inside the method?
- **F10 [SDE-2 Follow-up]** Explain `StackOverflowError` without claiming a fixed recursion depth.
- **F11 [Interview Core]** Which input boundaries expose integer-division mistakes?
- **F12 [SDE-2 Follow-up]** What evidence separates a compile-time, runtime, and logical defect?
- **F13 [SDE-2 Follow-up]** Explain a platform-dependent behavior that valid Java bytecode cannot hide.

Cumulative assessment 1

A01: In 35 minutes, write and explain a program that parses command-line integers, validates them, computes a safe `long` sum and product, reports negative/zero/positive counts, and avoids overflow-before-widening. Include six tests and classify every possible failure stage.

SDE-2 CORE CHAPTER

Chapter 7 - Classes, Objects, and Object-Oriented Foundations

Learning objectives

This chapter builds the minimum object model needed for Java interviews: classes, objects, constructors, **this**, **static**, access control, packages, encapsulation, inheritance, polymorphism, abstraction, interfaces, composition, equality, and immutability.

Class, object, state, and behavior

A class defines a type. An object is an instance of that class. Fields hold object state; methods expose behavior. A reference variable holds a reference value that may designate an object or be **null**; it does not contain the entire object.

JAVA

```
final class Student {
    private final int id;
    private String name;

    Student(int id, String name) {
        if (id <= 0) {
            throw new IllegalArgumentException("id must be positive");
        }
        this.id = id;
        this.name = java.util.Objects.requireNonNull(name);
    }

    int id() {
        return id;
    }

    String name() {
        return name;
    }

    void rename(String name) {
        this.name = java.util.Objects.requireNonNull(name);
    }
}
```

`Student first = new Student(1, "Ada");` creates one object and stores a reference value in `first`. `Student alias = first;` copies that reference value; both variables designate the same object. `alias.rename("Grace")` is therefore visible through `first`.

TEXT

```
first ----+
          +----> Student{id=1, name="Grace"}
alias ----+
```

Java still passes arguments by value. Passing `first` copies its reference value. A method can mutate the shared object through its copy, but assigning its parameter to a new object does not change the caller's variable.

Constructors and this

A constructor initializes a new object. Its name matches the class and it has no return type, not even `void`.

JAVA

```
final class Window {
    private final int width;
    private final int height;

    Window() {
        this(800, 600);           // must be the first statement
    }

    Window(int width, int height) {
        if (width <= 0 || height <= 0) {
            throw new IllegalArgumentException("positive dimensions required");
        }
        this.width = width;       // field = parameter
        this.height = height;
    }
}
```

`this` is the current receiver. It disambiguates a field from a parameter and calls another constructor with `this(...)`. A constructor may call `this(...)` or `super(...)` first, never both explicitly. If a class declares no constructor, Java may supply a no-argument default constructor. The moment any constructor is declared, that automatic constructor is not supplied.

Avoid calling overridable methods from constructors: dynamic dispatch can reach subclass code before subclass initialization is complete.

static: class state and behavior

An instance member belongs to an object. A static member belongs to the class.

JAVA

```
final class IdGenerator {
    private static int nextId = 1;

    private IdGenerator() {}

    static int next() {
        return nextId++;
    }
}
```

Call it as `IdGenerator.next()`. A static method has no `this` and cannot directly read an instance field. Static constants such as `private static final int MAX_ATTEMPTS = 3;` are appropriate. Mutable static state is shared across tests, calls, and threads; it makes reasoning and isolation harder and should be used deliberately.

Static methods are hidden, not overridden. Instance methods can be overridden and dynamically dispatched. Fields are also selected by the declared reference type, not polymorphically.

Access modifiers and packages

Modifier	Same class	Same package	Subclass in another package	Unrelated class elsewhere
<code>private</code>	yes	no	no	no
<code>package-private</code>	yes	yes	no	no
<code>protected</code>	yes	yes	yes, through inheritance rules	no
<code>public</code>	yes	yes	yes	yes

Across packages, a subclass's `protected` access is tied to inheritance; it cannot use a subclass reference to reach the protected member through an arbitrary sibling/base instance. Treat `protected` as an extension API, not "public to subclasses everywhere."

The package declaration is first, apart from comments:

JAVA

```
package interview.model;

import java.util.List;
import static java.util.Objects.requireNonNull;
```


An import lets source use a short name. It does not copy the class, change its package, or load it at runtime. Use a fully qualified name to resolve conflicts. Avoid the default package in maintainable multi-file code because named packages cannot import from it cleanly.

WHY IT WORKS | Encapsulation and invariants

Encapsulation means keeping representation decisions and invariants behind a small API. It is not the mechanical presence of getters and setters.

JAVA

```
final class BankAccount {
    private long balanceCents;

    BankAccount(long openingBalanceCents) {
        if (openingBalanceCents < 0) throw new IllegalArgumentException();
        balanceCents = openingBalanceCents;
    }

    void withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) {
            throw new IllegalArgumentException("invalid withdrawal");
        }
        balanceCents -= cents;
    }

    long balanceCents() {
        return balanceCents;
    }
}
```

A public setter for `balanceCents` would destroy the invariant. Expose the domain operation instead.

Inheritance and runtime polymorphism

Inheritance expresses an IS-A relationship and a subtype promise.

JAVA

```
class Notification {
    String format() {
        return "notification";
    }
}

final class EmailNotification extends Notification {
    @Override
    String format() {
        return "email";
    }
}

Notification message = new EmailNotification();
System.out.println(message.format()); // email
```

The declared reference type is **Notification**; the runtime object type is **EmailNotification**. The selected overridable instance method is dispatched to the object type. This is runtime polymorphism. Overloading is compile-time selection among different parameter lists; return type alone cannot create an overload.

An upcast to a base type is safe. A downcast can fail:

JAVA

```
if (message instanceof EmailNotification email) {
    System.out.println(email.format());
}
```

The pattern form is final in Java 16+. In earlier Java, test with **instanceof** and then cast. Do not cast merely to access behavior that should be declared on the base contract.

super(...) invokes a superclass constructor and must be first when explicit. Superclass construction occurs before the subclass constructor body. Java supports multiple interface implementation, not multiple class inheritance.

Abstraction and interfaces

Abstraction exposes what a type does while hiding unnecessary implementation detail. An abstract class can have fields, constructors, concrete methods, and abstract methods.

JAVA

```
abstract class Parser {
    Parser() {}
    final int parseNonNegative(String text) {
        int value = parse(text);
        if (value < 0) throw new IllegalArgumentException();
        return value;
    }
    protected abstract int parse(String text);
}
```

An interface defines a role or contract that unrelated classes may implement.

JAVA

```
interface Formatter {
    String format(int value);

    default String label(int value) {
        return "value=" + format(value);
    }

    static Formatter decimal() {
        return Integer::toString;
    }
}
```

Interfaces may contain abstract methods, default methods, static methods, and, since Java 9, private helper methods. They have no constructors or per-instance fields. Interface fields are constants. A **functional interface** has one abstract method and can be a lambda target; lambdas are introduced here only so library examples remain readable.

Choose	When
Interface	a role, multiple implementations, an open contract, or test seam
Abstract class	closely related types share state, construction, or a template
Concrete class	one complete implementation is the useful concept
Composition	one object uses a replaceable collaborator or HAS-A relationship

Composition before forced inheritance

Composition delegates work to a collaborator.

JAVA

```
interface Clock {
    long nowMillis();
}

final class ExpiringToken {
    private final Clock clock;
    private final long expiresAt;

    ExpiringToken(Clock clock, long expiresAt) {
        this.clock = java.util.Objects.requireNonNull(clock);
        this.expiresAt = expiresAt;
    }

    boolean isExpired() {
        return clock.nowMillis() >= expiresAt;
    }
}
```

ExpiringToken HAS-A Clock; it is not a clock. A test can pass a deterministic clock. Composition keeps the token invariant local and permits replacement without inventing a false subtype.

Equality, hash code, and immutability

`==` compares object identity. `equals` represents semantic equality when a class defines it. Equal objects must have equal hash codes; mutable fields that participate in equality make unsafe hash-map keys because mutation can move the logical key without moving its bucket entry.

An immutable class prevents observable state changes after construction. `final` on a reference prevents reassignment; it does not freeze the referenced object.

JAVA

```
final class Coordinate {
    private final int row;
    private final int column;

    Coordinate(int row, int column) {
        this.row = row;
        this.column = column;
    }

    @Override public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof Coordinate that)) return false;
        return row == that.row && column == that.column;
    }

    @Override public int hashCode() {
        return java.util.Objects.hash(row, column);
    }
}
```

Records can express simple immutable data carriers in Java 16+, but ordinary classes remain essential interview knowledge.

WATCH OUT | Common mistakes and interview angles

- Confusing a reference with the object itself.
- Calling Java pass-by-reference.
- Assuming a declared constructor leaves the compiler-provided no-argument constructor in place.
- Adding `void` to a constructor, accidentally creating a method.
- Calling static members through instances.
- Using inheritance only to reuse code.
- Treating fields or static methods as dynamically dispatched.
- Thinking abstract classes cannot have constructors or interfaces cannot have implemented methods.
- Saying `final` makes an object immutable.
- Overriding `equals` without a consistent `hashCode`.

Quick check and practice

- 1 What is copied by `Student alias = first`?
- 2 When does Java provide a default constructor?
- 3 Why can a static method not use `this`?
- 4 What is selected from the declared reference type, and what is dispatched from the runtime object type?
- 5 When is composition more truthful than inheritance?

Foundation: Model a `Rectangle` with validated dimensions and an `area()` method.

Interview Core: Refactor a class with public mutable fields into an invariant-preserving API.

SDE-2 Follow-up: Explain why a mutable object used as a `HashMap` key can become unreachable by lookup.

Cross-book boundary

This chapter teaches the object model required by DSA code. Continue to Low-Level Design for SOLID, patterns, modeling exercises, and larger collaborations; to Advanced Java for sealed types, records, reflection, and generic variance; and to the Hashing volume for collection-key contracts in depth.

RECAP | Chapter summary

Classes define state and behavior; references designate objects and may alias. Constructors establish invariants, instance methods use receivers, static members belong to the class, and access control shapes APIs. Runtime dispatch applies to overridden instance methods, while composition is often safer than forced inheritance. Equality, hashing, and immutability are behavioral contracts, not keywords to memorize.

SDE-2 CORE CHAPTER

Chapter 8 - Practice Lab B - Arrays, Strings, and Objects

Start here: draw references before writing code. For every array or object question, mark which variables hold primitive values, which hold reference values, and which references designate the same object. For strings, state whether the requirement is identity, UTF-16 content, code points, or user-visible text before selecting an API.

Complete K26-K50 aloud, predict O20-O38 on paper, compile repairs for D20-D38, and implement C20-C38 in small files. Use a mutation table for each task: input alias, state before, operation, state after, and whether the caller observes the change. Check Solution Studio B later and add every missed aliasing, constructor, dispatch, or equality rule to your error log.

Knowledge check

- **K26 [Foundation]** Distinguish array declaration, creation, and initialization.
- **K27 [Foundation]** What default values do array elements receive?
- **K28 [Interview Core]** Explain array aliasing.
- **K29 [Interview Core]** Distinguish shallow from deep copy.
- **K30 [Foundation]** Why does printing an array reference not print its contents?
- **K31 [Interview Core]** Distinguish `Arrays.equals` from `Arrays.deepEquals`.
- **K32 [Foundation]** What makes a Java two-dimensional array jagged-capable?
- **K33 [Foundation]** Why is `String` immutable?
- **K34 [Interview Core]** Distinguish string identity, content, and pooling.
- **K35 [Foundation]** Distinguish empty, blank, and null strings.
- **K36 [Interview Core]** When should repeated concatenation use `StringBuilder`?
- **K37 [Foundation]** What does a Java `char` represent?
- **K38 [Interview Core]** Distinguish code unit, code point, and user-perceived character.
- **K39 [Foundation]** Define class, object, field, method, state, and behavior.
- **K40 [Interview Core]** What does a reference variable store?
- **K41 [Foundation]** When is a default constructor supplied?
- **K42 [Foundation]** List the three common uses of `this` in fundamentals.
- **K43 [Interview Core]** Why is mutable static state hard to test?
- **K44 [Interview Core]** Explain protected access across package boundaries.
- **K45 [Foundation]** What does an import declaration do and not do?
- **K46 [Interview Core]** Why is encapsulation more than getters and setters?
- **K47 [Interview Core]** Distinguish overloading, overriding, field hiding, and static hiding.
- **K48 [Foundation]** Compare interface, abstract class, and concrete class.
- **K49 [Interview Core]** When is composition preferable to inheritance?
- **K50 [SDE-2 Follow-up]** State the `equals/hashCode` requirement for hash keys.

Predict the output

- **O20 [Foundation]** Predict an `int[3]` element before assignment.
- **O21 [Interview Core]** Predict output after mutating an aliased one-dimensional array.
- **O22 [Interview Core]** Predict output after mutating a nested row through `matrix.clone()`.
- **O23 [Foundation]** Predict `Arrays.toString(new int[]{1,2})`.
- **O24 [Interview Core]** Predict `new int[2][ ]` row values.
- **O25 [Foundation]** Predict `"java".substring(1,3)`.
- **O26 [Interview Core]** Predict literal equality for `"a" == "a"` and explain why not to rely on it generally.
- **O27 [Interview Core]** Predict equality for `new String("a") == "a"`.
- **O28 [Foundation]** Predict a string after ignored `toUpperCase()` return value.
- **O29 [Foundation]** Predict a builder after `append`, `deleteCharAt`, and `reverse`.
- **O30 [Interview Core]** Predict the length of one supplementary emoji encoded as UTF-16.
- **O31 [Foundation]** Predict two objects' independent field values after one is renamed.
- **O32 [Interview Core]** Predict a shared object's field after alias mutation.
- **O33 [Foundation]** Predict constructor-chaining print order with `this(...)`.
- **O34 [Interview Core]** Predict static field count after creating three instances.
- **O35 [Interview Core]** Predict field and method outputs through a base reference to a child.
- **O36 [Interview Core]** Predict overloaded method selection through a base reference.
- **O37 [Interview Core]** Predict overridden method dispatch through an interface reference.
- **O38 [SDE-2 Follow-up]** Predict map lookup after mutating an equality-participating key.

Debug the code

- **D20 [Foundation]** Repair array traversal that reads index `length`.
- **D21 [Interview Core]** Repair a method that unintentionally mutates the caller's array.
- **D22 [Interview Core]** Repair a supposed deep copy that clones only the outer array.
- **D23 [Foundation]** Repair `firstArray == secondArray` for content equality.
- **D24 [Foundation]** Repair direct array printing.
- **D25 [Interview Core]** Repair matrix traversal that assumes every row has `matrix[0].length`.
- **D26 [Foundation]** Repair `firstString == secondString` for value equality.
- **D27 [Interview Core]** Repair null-unsafe `text.equals(target)`.
- **D28 [Interview Core]** Repair repeated string concatenation in a large loop.
- **D29 [Foundation]** Repair ASCII digit conversion without validation.
- **D30 [Interview Core]** Repair string reversal that corrupts surrogate pairs under a code-point contract.
- **D31 [Foundation]** Repair a constructor mistakenly declared `void Student(...)`.
- **D32 [Foundation]** Restore a missing no-argument constructor intentionally.
- **D33 [Interview Core]** Repair a static method that directly reads an instance field.
- **D34 [Interview Core]** Repair a public setter that violates an account invariant.
- **D35 [Interview Core]** Repair an intended override with the wrong parameter type.
- **D36 [Interview Core]** Repair unsafe downcasting with a type check or stronger contract.
- **D37 [Interview Core]** Repair an interface implementation that reduces method visibility.
- **D38 [SDE-2 Follow-up]** Repair a mutable list field in an allegedly immutable class.

Small coding tasks

- **C20 [Foundation]** Create and traverse a one-dimensional integer array.
- **C21 [Foundation]** Create and traverse a rectangular matrix.
- **C22 [Foundation]** Create a jagged array with row lengths 1, 2, and 3.
- **C23 [Interview Core]** Demonstrate array aliasing and then repair it with a copy.
- **C24 [Interview Core]** Deep-copy a nullable jagged array.
- **C25 [Foundation]** Compare and print one-dimensional arrays correctly.
- **C26 [Foundation]** Demonstrate string content versus identity comparison.
- **C27 [Interview Core]** Demonstrate literal pooling versus explicit construction.
- **C28 [Foundation]** Build a comma-separated string with `StringBuilder`.
- **C29 [Interview Core]** Convert validated decimal characters with `Character.digit`.
- **C30 [Foundation]** Implement the validated `Student` class from this volume.
- **C31 [Foundation]** Add constructor overloading and chaining to a `Rectangle`.
- **C32 [Interview Core]** Implement a class-level ID generator and discuss shared state.
- **C33 [Interview Core]** Encapsulate a bounded counter with invariant-preserving methods.
- **C34 [Interview Core]** Implement an immutable `Coordinate` with equality and hash code.
- **C35 [Foundation]** Implement a base `Notification` and overriding subtype.
- **C36 [Interview Core]** Demonstrate runtime polymorphism through a list of interface references.
- **C37 [Interview Core]** Implement an abstract parser with a concrete decimal parser.
- **C38 [SDE-2 Follow-up]** Replace a false inheritance relationship with a composed collaborator and test double.

Interview follow-ups

- **F14 [Interview Core]** When should an array method preserve versus mutate its input?
- **F15 [SDE-2 Follow-up]** Define a copy contract for jagged arrays containing null rows.
- **F16 [Interview Core]** Why can `Arrays.binarySearch` be wrong on an unsorted array?
- **F17 [Interview Core]** When is `char` traversal the correct contract despite Unicode limits?
- **F18 [SDE-2 Follow-up]** How would you state string complexity in code units versus code points?
- **F19 [Interview Core]** Why does `final` not prove deep immutability?
- **F20 [Interview Core]** What construction bug can an overridable method expose?
- **F21 [SDE-2 Follow-up]** Explain protected access through a subclass in another package.
- **F22 [Interview Core]** When is an abstract class better than an interface?
- **F23 [Interview Core]** How can a default interface method create an evolution conflict?
- **F24 [SDE-2 Follow-up]** Defend composition for replaceability and testability.
- **F25 [Interview Core]** Why must equal hash keys have equal hash codes?
- **F26 [SDE-2 Follow-up]** Design an immutable class that owns a mutable list safely.

Cumulative assessment 2

A02: In 45 minutes, build a small library checkout model using an enum state, an interface policy, composition, an immutable identifier, a mutable aggregate with guarded transitions, and a collection-free array snapshot. Explain aliases, constructor invariants, dispatch, equality, and five boundary tests.

SDE-2 CORE CHAPTER

Chapter 9 - Wrappers, Generics, Enums, Collections, I/O, and Utilities

Learning objectives

This chapter supplies the library runway used in early DSA solutions: primitive wrappers, boxing, basic generics, enums, collection selection, console I/O, and high-value utility APIs. Internals and advanced type-system mechanics remain in later books.

Wrapper classes and parsing

Generics and collections require reference types, so every primitive has a wrapper: `byte/Byte`, `short/Short`, `int/Integer`, `long/Long`, `float/Float`, `double/Double`, `char/Character`, and `boolean/Boolean`. Wrappers are immutable and provide parsing, comparison, conversion, constants, and utility methods.

JAVA

```
int count = Integer.parseInt("42");
long maximum = Long.MAX_VALUE;
int comparison = Integer.compare(2_000_000_000, -2_000_000_000);
System.out.println(count); // 42
System.out.println(comparison); // 1
```

`parseInt` returns a primitive and throws `NumberFormatException` for invalid or out-of-range input. `Integer.valueOf("42")` returns a wrapper. Use `Integer.compare(left, right)`, not `left - right`, because subtraction can overflow.

Autoboxing and unboxing

Autoboxing converts a primitive to its wrapper where required; unboxing extracts the primitive value.

JAVA

```
java.util.List<Integer> values = new java.util.ArrayList<>();
values.add(7); // boxes int to Integer
int first = values.get(0); // unboxes Integer to int
```

Unboxing `null` throws `NullPointerException`.

JAVA

```
Integer boxed = null;
// int value = boxed; // runtime NullPointerException if reached
```

Wrapper identity is not value equality:

JAVA

```
Integer first = 127;
Integer second = 127;
Integer third = 128;
Integer fourth = 128;
System.out.println(first == second); // commonly true by required small cache
System.out.println(third == fourth); // do not rely on this identity result
System.out.println(third.equals(fourth)); // true
```

The required cache makes some small values share instances, and implementations may cache more. Never turn that optimization into program logic; use `equals` or `Objects.equals` for wrapper values.

Basic generics

A type parameter lets one declaration work safely with multiple reference types.

JAVA

```
final class Box<T> {
    private final T value;
    Box(T value) { this.value = value; }
    T value() { return value; }
}

Box<String> word = new Box<>("java");
String text = word.value();
```

`T` is a type parameter. `Box<String>` prevents inserting an `Integer` and removes a caller-side cast. The diamond `<>` asks the compiler to infer type arguments. Primitives cannot be generic arguments, so use `List<Integer>`, not `List<int>`.

A generic method can declare its own type parameter:

JAVA

```
static <T> T first(java.util.List<T> values) {
    if (values.isEmpty()) throw new IllegalArgumentException("empty list");
    return values.get(0);
}
```

A light bound says which operations are available: `<T extends Comparable<? super T>>`. Wildcards, PECS, capture, erasure, and heap pollution belong in Advanced Java.

Enums model finite state

JAVA

```
enum OrderStatus {
    CREATED,
    PAID,
    SHIPPED,
    DELIVERED,
    CANCELLED;

    boolean isTerminal() {
        return this == DELIVERED || this == CANCELLED;
    }
}
```

Enum constants are typed singleton instances. Enums may have fields, private constructors, and methods. They are safer than magic strings because the compiler checks names and `switch` can cover the declared states. Avoid persisting `ordinal()` as a durable external contract; declaration order can change.

Collection selection for early DSA

Choose by semantics first.

Need	Typical interface/implementation	Core calls
indexed sequence	List / ArrayList	add, get, set, size
membership or deduplication	Set / HashSet	add, contains, remove
key to value or frequency	Map / HashMap	put, get, getOrDefault, merge
FIFO queue	Queue / ArrayDeque	offer, poll, peek
LIFO stack	Deque / ArrayDeque	push, pop, peek
smallest/largest priority	PriorityQueue	offer, poll, peek
insertion order retained	LinkedHashSet, LinkedHashMap	set/map APIs
sorted keys/elements	TreeMap, TreeSet	map/set plus navigation

`LinkedList` is useful when its deque semantics or iterator-position operations fit. It is not universally faster for insertion: locating a position and pointer-heavy memory behavior matter. Prefer `ArrayDeque` over legacy `Stack` in new interview code.

Frequency map:

JAVA

```
Map<String, Integer> frequency = new HashMap<>();
for (String word : List.of("java", "dsa", "java")) {
    frequency.merge(word, 1, Integer::sum);
}
System.out.println(frequency.get("java")); // 2
```

Deduplication while retaining encounter order:

JAVA

```
Set<Integer> unique = new LinkedHashSet<>(List.of(3, 1, 3, 2));
System.out.println(unique); // [3, 1, 2]
```

Queue and stack with [ArrayDeque](#):

JAVA

```
Deque<Integer> deque = new ArrayDeque<>();
deque.offerLast(10);
deque.offerLast(20);
System.out.println(deque.pollFirst()); // queue removes 10

deque.push(30);
System.out.println(deque.pop()); // stack removes 30
```

Min-heap:

JAVA

```
PriorityQueue<Integer> minimums = new PriorityQueue<>();
minimums.offer(9);
minimums.offer(3);
minimums.offer(5);
System.out.println(minimums.poll()); // 3
```

Only repeated `poll` operations reveal priority order. Iteration is not sorted order. Hash-based operations are commonly expected constant time under suitable hashing, but the API does not promise universal $O(1)$ behavior. Continue to the dedicated collection and DSA volumes for costs and internals.

Input and output

Many interviews provide a method signature and do not require console parsing. When input code is required, choose convenience or throughput deliberately.

Scanner is convenient:

JAVA

```
Scanner scanner = new Scanner(System.in);
int length = scanner.nextInt();
int[] numbers = new int[length];
for (int index = 0; index < length; index++) {
    numbers[index] = scanner.nextInt();
}
```

BufferedReader plus parsing is more explicit and often faster for token-heavy input:

JAVA

```
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
int rows = Integer.parseInt(reader.readLine().strip());
int[][] matrix = new int[rows][];
for (int row = 0; row < rows; row++) {
    StringTokenizer tokens = new StringTokenizer(reader.readLine());
    int columns = tokens.countTokens();
    matrix[row] = new int[columns];
    for (int column = 0; column < columns; column++) {
        matrix[row][column] = Integer.parseInt(tokens.nextToken());
    }
}
```

The second example permits jagged rows. In real code, validate token counts and define behavior for missing lines. Use `PrintWriter` to buffer output and flush at the ownership boundary. Try-with-resources is appropriate for resources your code owns; do not casually close `System.in` when another component owns it.

Interview utility APIs

- Math: `min`, `max`, `abs`, `sqrt`, `pow`, `floor`, `ceil`, `addExact`, `subtractExact`, `multiplyExact`.
- Arrays: `sort`, `fill`, `copyOf`, `equals`, `deepEquals`, `binarySearch`.
- Collections: `sort`, `reverse`, `min`, `max`, `frequency`, `binarySearch`.
- Objects: `equals`, `hash`, `requireNonNull`.
- Wrappers: parsing, `compare`, `min/max` constants, `toString(value, radix)`, and `unsigned/base` helpers where needed.
- Character: `isDigit`, `isLetter`, `isWhitespace`, `toLowerCase`, `toUpperCase`, and `digit`.

Important caveats:

JAVA

```
List<String> fixed = Arrays.asList("a", "b");
fixed.set(0, "x");      // allowed
// fixed.add("c");      // UnsupportedOperationException

int[] primitive = {1, 2};
List<int> oneElement = Arrays.asList(primitive);
System.out.println(oneElement.size()); // 1, not 2
```

`List.of(...)` creates an unmodifiable list and rejects null elements.

`Collections.unmodifiableList(existing)` creates an unmodifiable view; changes through another alias to `existing` remain visible. Label the exact contract instead of calling every such result "immutable."

WATCH OUT | Common mistakes and interview angles

- Unboxing a nullable wrapper.
- Comparing wrappers with `==`.
- Using `List<int>` instead of `List<Integer>`.
- Using raw collections and deferring type errors to runtime.
- Using a string where an enum defines the finite state.
- Assuming `HashMap` order or guaranteed $O(1)$ operations.
- Treating `PriorityQueue` iteration as sorted.
- Writing `left.priority - right.priority` in a comparator.
- Using legacy `Stack` for new code.
- Closing a stream the current method does not own.
- Assuming `Arrays.asList(intArray)` produces a list of boxed integers.

Quick check and practice

- 1 What is the difference between parsing, boxing, and unboxing?
- 2 Why can `Integer` identity tests appear inconsistent?
- 3 Why are primitives unavailable as generic arguments?
- 4 Which collection represents FIFO without accepting `null` in its common implementation?
- 5 Why is a priority queue's iterator not a sorted traversal?

Foundation: Count character frequencies with `Map<Character, Integer>`.

Interview Core: Read a jagged integer matrix and print each row sum.

SDE-2 Follow-up: Compare an unmodifiable view with an independently copied unmodifiable list, including aliasing behavior.

Cross-book boundary

Continue to Hashing, Stacks, Queues and Deques, Heaps, and Time and Space Complexity for algorithm patterns and defensible costs. Continue to Java Collections Internals for implementation mechanics, Advanced Java for generics and lambdas in depth, and the I/O volume for files, charsets, channels, and resource ownership.

RECAP | Chapter summary

Wrappers connect primitives to generic APIs but introduce null and identity traps. Generics provide compile-time type safety for reference types. Enums model finite state. Collections should be selected by contract, not folklore. Input code should match the platform boundary, and utility methods should be used with their exact aliasing, ordering, overflow, and mutability behavior understood.

SDE-2 CORE CHAPTER

Chapter 10 - Java Collections Basics for SDE-2 Interviews

Learning objectives

By the end of this chapter, you can create and use the collection types expected in Java coding interviews, choose one from required behavior, iterate and modify safely, explain basic mutability and ordering contracts, and avoid the API traps that commonly break otherwise correct algorithms.

This is the usage chapter. The later Collections Architecture chapter explains views, ownership, optional operations, and design trade-offs more deeply. Hash-table internals, tree balancing, heap mechanics, and concurrent collections remain in their dedicated books.

Start with the problem, not the class name

An array has fixed length. A collection manages a changing group of reference values through an interface. Generic type arguments state what may be stored.

JAVA

```
List<String> names = new ArrayList<>();  
names.add("Ada");  
names.add("Grace");
```

Read the declaration as: `names` follows the `List<String>` contract, and the current implementation is `ArrayList<String>`. Program to the interface unless a concrete implementation exposes behavior the method genuinely needs.

Use this first decision:

Requirement	Start with	Common implementation
ordered sequence, duplicates allowed	<code>List<E></code>	<code>ArrayList<E></code>
uniqueness or membership	<code>Set<E></code>	<code>HashSet<E></code>
key-to-value association	<code>Map<K, V></code>	<code>HashMap<K, V></code>
first-in, first-out work	<code>Queue<E></code> or <code>Deque<E></code>	<code>ArrayDeque<E></code>
last-in, first-out work	<code>Deque<E></code>	<code>ArrayDeque<E></code>
repeatedly remove best priority	<code>PriorityQueue<E></code>	<code>PriorityQueue<E></code>
retain insertion order	<code>List</code> , <code>LinkedHashSet</code> , <code>LinkedHashMap</code>	matching type
keep keys/elements sorted	<code>TreeMap</code> , <code>TreeSet</code>	matching type

Do not select `LinkedList` because an interview slogan says insertion is fast. Insertion is only useful after reaching the position, and node-based storage has real locality and memory costs.

List basics

A list preserves encounter order, permits duplicates, and supports positional access.

JAVA

```
List<Integer> scores = new ArrayList<>();
scores.add(70);           // [70]
scores.add(90);           // [70, 90]
scores.add(1, 80);        // [70, 80, 90]
int first = scores.get(0); // 70
scores.set(0, 75);        // [75, 80, 90]
boolean hasNinety = scores.contains(90);
int size = scores.size();
```

Indexes are zero-based and form `[0, size())`. `get`, `set`, and indexed insertion throw `IndexOutOfBoundsException` for an invalid position.

The overloaded `remove` methods are a classic trap:

JAVA

```
List<Integer> values = new ArrayList<>(List.of(10, 20, 30));
values.remove(1);           // removes index 1 -> value 20
values.remove(Integer.valueOf(30)); // removes the element 30
```

`ArrayList` is the default general-purpose list for interviews. `LinkedList` also implements `List` and `Deque`, but indexed access is linear. Use it only when its exact deque or iterator-position behavior is justified; `ArrayDeque` is normally clearer for queue/stack work.

Set basics

A set stores no duplicate elements according to equality or ordering.

JAVA

```
Set<Integer> seen = new HashSet<>();
System.out.println(seen.add(4)); // true
System.out.println(seen.add(4)); // false
System.out.println(seen.contains(4)); // true
seen.remove(4);
```

`add` returning `false` is often useful while detecting duplicates.

JAVA

```
static boolean hasDuplicate(int[] numbers) {
    Set<Integer> seen = new HashSet<>();
    for (int number : numbers) {
        if (!seen.add(number)) return true;
    }
    return false;
}
```

- `HashSet` does not promise iteration order.
- `LinkedHashSet` preserves insertion encounter order.
- `TreeSet` keeps elements sorted according to natural order or a comparator and supports operations such as `floor`, `ceiling`, `lower`, and `higher`.

Set behavior depends on stable equality/hash or comparison. Mutating a stored element's key fields can make membership behavior incorrect.

Map basics

A map associates unique keys with values. It is not a subtype of `Collection`.

JAVA

```
Map<String, Integer> attempts = new HashMap<>();
attempts.put("java", 1);
attempts.put("java", 2); // replaces the value for the same key

int value = attempts.get("java");
int missingAsZero = attempts.getDefault("dsa", 0);
boolean hasKey = attempts.containsKey("java");
attempts.remove("java");
```

Distinguish a missing key from a key mapped to null when the implementation permits null values: `get` returns null for both, while `containsKey` separates them. Prefer a domain contract that avoids ambiguous null when practical.

Frequency counting:

JAVA

```
static Map<String, Integer> frequencies(List<String> words) {  
    Map<String, Integer> count = new HashMap<>();  
    for (String word : words) {  
        count.merge(word, 1, Integer::sum);  
    }  
    return count;  
}
```

Useful update APIs:

- `putIfAbsent(key, value)` installs only when absent.
- `computeIfAbsent(key, function)` lazily creates a value, useful for grouping.
- `merge(key, value, function)` combines a new value with an existing one.
- `replace` changes only an existing mapping.

Grouping example:

JAVA

```
Map<Character, List<String>> groups = new HashMap<>();  
for (String word : List.of("array", "api", "map")) {  
    groups.computeIfAbsent(word.charAt(0), ignored -> new ArrayList<>())  
        .add(word);  
}
```

Iteration choices:

JAVA

```
for (Map.Entry<String, Integer> entry : attempts.entrySet()) {  
    String key = entry.getKey();  
    int count = entry.getValue();  
}
```

Use `entrySet` when both key and value are needed; it avoids a second lookup and communicates the mapping traversal directly.

- `HashMap` does not promise iteration order.
- `LinkedHashMap` preserves insertion order, or access order when configured for specialized use.
- `TreeMap` keeps keys sorted and supports range/nearest-key navigation.

Queue and deque basics

A queue models FIFO work. A deque supports both ends and can model both a queue and a stack.

Prefer the non-throwing queue method family in interview algorithms:

Operation	Special-value form	Throwing form
insert	<code>offer</code>	<code>add</code>
remove head	<code>poll</code>	<code>remove</code>
inspect head	<code>peek</code>	<code>element</code>

FIFO with `ArrayDeque`:

JAVA

```
Deque<String> queue = new ArrayDeque<>();
queue.offerLast("first");
queue.offerLast("second");
System.out.println(queue.pollFirst()); // first
```

LIFO with `ArrayDeque`:

JAVA

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
System.out.println(stack.pop()); // 20
```

Use consistent ends. Mixing `push`, `offerLast`, `pollFirst`, and `removeLast` without a written convention makes traces difficult. `ArrayDeque` rejects null, which keeps null available as the empty-result signal for `poll` and `peek`.

Avoid legacy `Stack` for new interview code. `ArrayDeque` provides the needed stack contract without inheriting the old synchronized `Vector` API.

PriorityQueue basics

`PriorityQueue` exposes the element with the smallest natural order by default.

JAVA

```
PriorityQueue<Integer> minimums = new PriorityQueue<>();
minimums.offer(9);
minimums.offer(3);
minimums.offer(5);

System.out.println(minimums.peek()); // 3
System.out.println(minimums.poll()); // 3
```

A max-heap reverses the comparator:

JAVA

```
PriorityQueue<Integer> maximums =  
    new PriorityQueue<>(Comparator.reverseOrder());
```

For a custom type, compare without subtraction:

JAVA

```
record Candidate(String name, int score) {}  
  
Comparator<Candidate> byHighestScore =  
    Comparator.comparingInt(Candidate::score)  
        .reversed()  
        .thenComparing(Candidate::name);
```

The queue promises only the head after each operation. Iterating the priority queue is not sorted traversal. Poll a defensive copy when sorted output is required and the original must remain intact.

Iteration and safe removal

Enhanced-for is best when no index or structural mutation is needed:

JAVA

```
for (String name : names) {  
    System.out.println(name);  
}
```

Do not structurally modify an ordinary collection directly inside enhanced-for. Use one of these contracts:

JAVA

```
values.removeIf(value -> value < 0);  
  
for (Iterator<Integer> iterator = values.iterator(); iterator.hasNext();) {  
    if (iterator.next() < 0) iterator.remove();  
}
```

`ConcurrentModificationException` is a best-effort bug signal, not a thread-safety mechanism.

Factories, copies, and mutability

These declarations do not mean the same thing:

JAVA

```
List<String> mutable = new ArrayList<>(List.of("a", "b"));
List<String> unmodifiable = List.of("a", "b");
List<String> fixedSize = Arrays.asList("a", "b");
List<String> snapshot = List.copyOf(mutable);
List<String> readOnlyView = Collections.unmodifiableList(mutable);
```

- `mutable` permits structural and element replacement.
- `List.of` is unmodifiable and rejects null.
- `Arrays.asList` is fixed-size; `set` works but `add` and `remove` do not.
- `List.copyOf` creates unmodifiable membership not backed by later source changes.
- `unmodifiableList` is a view; mutation through another alias remains visible.

None of these operations makes mutable element objects deeply immutable.

Sorting and conversion basics

JAVA

```
List<Integer> numbers = new ArrayList<>(List.of(4, 1, 3));
numbers.sort(Integer::compare);
Collections.reverse(numbers);

int[] primitiveArray = numbers.stream().mapToInt(Integer::intValue).toArray();
List<Integer> boxed = Arrays.stream(primitiveArray).boxed().toList();
```

`toList()` returns an unmodifiable list in current Java. Use `new ArrayList<>(...)` when a mutable result is required. `Arrays.asList(primitiveArray)` produces a one-element `List<int[]>`, not a list of boxed integers.

Complexity intuition, not folklore

Interfaces do not promise one complexity. State the implementation and qualifier.

Operation	Common implementation	Interview-level cost
indexed get/set	<code>ArrayList</code>	$O(1)$
append	<code>ArrayList</code>	amortized $O(1)$
insert/remove at front	<code>ArrayList</code>	$O(n)$ shifting
indexed get	<code>LinkedList</code>	$O(n)$
membership	<code>HashSet</code>	expected $O(1)$
key lookup	<code>HashMap</code>	expected $O(1)$
ordered lookup	<code>TreeSet/TreeMap</code>	$O(\log n)$
deque end operation	<code>ArrayDeque</code>	amortized $O(1)$
heap offer/poll	<code>PriorityQueue</code>	$O(\log n)$
heap peek	<code>PriorityQueue</code>	$O(1)$

The Time and Space Complexity volume explains how to derive and qualify these costs. The collection-internals volume explains resizing, hashing, trees, node overhead, and implementation behavior.

Interview checklist

Before choosing a collection, state:

- 1 Are duplicates allowed?
- 2 Does encounter or sorted order matter?
- 3 Is the operation positional, membership, key lookup, FIFO, LIFO, or priority?
- 4 Can inputs or returned data be mutated?
- 5 Are nulls valid?
- 6 Which operation dominates and at what scale?
- 7 Does equality or comparator state remain stable?

WATCH OUT | Common mistakes

- Declaring `ArrayList` everywhere instead of the useful interface.
- Forgetting `Map` is not a `Collection`.
- Confusing `remove(index)` and `remove(value)` for `List<Integer>`.
- Depending on `HashMap/HashSet` iteration order.
- Assuming `LinkedList` is automatically faster for insertion.
- Using `Stack` instead of `ArrayDeque`.
- Treating `PriorityQueue` iteration as sorted.
- Writing comparator subtraction that can overflow.
- Modifying a collection directly in enhanced-for.
- Confusing fixed-size, unmodifiable, snapshot, view, and deeply immutable.
- Using mutable equality/hash fields for set elements or map keys.
- Claiming `HashMap` operations are guaranteed $O(1)$.

Practice

- 1 **Foundation:** Use `ArrayList` to insert, replace, remove by index, and remove by value.
- 2 **Foundation:** Deduplicate input while preserving first-seen order.
- 3 **Interview Core:** Build a frequency map, then return the most frequent key with a deterministic tie-breaker.
- 4 **Interview Core:** Implement FIFO and LIFO behavior with two separate `ArrayDeque` instances.
- 5 **Interview Core:** Maintain the largest `k` values with a size-`k` min-heap.
- 6 **Interview Core:** Demonstrate the difference among `List.of`, `Arrays.asList`, `List.copyOf`, and an unmodifiable view.
- 7 **SDE-2 Follow-up:** Define an API returning ordered results without exposing mutable internal membership.
- 8 **SDE-2 Follow-up:** Compare `ArrayList`, `HashSet`, and `TreeSet` for repeated membership queries, including build cost and ordering.

RECAP | Chapter summary

Choose collections from semantics: sequence, uniqueness, association, FIFO/LIFO, or priority. Use ArrayList, HashSet, HashMap, ArrayDeque, and PriorityQueue as the normal interview starting points, then select linked or sorted variants only for a stated contract. Know the everyday APIs, ownership/mutability differences, safe iteration rules, null and equality boundaries, and qualified costs before studying internals.

SDE-2 CORE CHAPTER

Chapter 11 - Exceptions and Resource Management

Learning objectives

By the end of this chapter, you should be able to:

- distinguish checked exceptions, unchecked exceptions, and errors by contract;
- trace matching, propagation, rethrow, `finally`, and suppressed exceptions;
- use try-with-resources with correct ownership and close order;
- translate failures without losing cause or actionable context; and
- design retryable, observable, and non-leaking production error boundaries.

WHY IT WORKS | Why this matters at SDE-2

Failure handling determines whether a service preserves data and recovers predictably. A lost cause obscures incidents, an overbroad catch turns corruption into a fake success, and a leaked connection can exhaust a pool. SDE-2 engineers must design failure semantics, not merely make the compiler accept `throws` clauses.

Interviews commonly ask about checked versus unchecked exceptions, `finally` behavior, close ordering, and what happens when both work and cleanup fail. The strongest answers connect the mechanics to API ownership and operational policy.

WHY IT WORKS | First-principles model

An exception is an object representing abrupt completion. `throw` transfers control up dynamically nested execution until a compatible `catch` is found. If no application frame catches it, the thread terminates after its uncaught-exception handling.

Java divides `Throwable` into `Error` and `Exception`. `RuntimeException` and its subclasses are unchecked, as are `Error` subclasses. Other `Exception` subclasses are checked: a potentially escaping checked exception must be caught or declared.

A resource is something with a lifecycle that must be closed regardless of normal or abrupt completion. Try-with-resources gives that lifecycle structured scope. Ownership answers who closes; it is separate from which variable can access the resource.

Specification boundary: Java specifies exception search, `finally`, try-with-resources translation, close order, and suppression. It does not guarantee that a process can recover from every `Error`, that every termination path runs cleanup, or that stack-trace collection has a fixed cost.

Core terminology

- **Checked exception:** non-`RuntimeException` subclass of `Exception`, enforced by compile-time handling rules.
- **Unchecked exception:** `RuntimeException`, `Error`, or subclass.
- **Cause:** lower-level failure wrapped by another exception.
- **Stack trace:** recorded execution frames associated with a throwable.
- **Suppressed exception:** secondary failure retained when another failure is primary.
- **Try-with-resources:** statement that closes `AutoCloseable` resources automatically.
- **Exception translation:** mapping an implementation failure to a boundary-appropriate abstraction.
- **Precise rethrow:** compiler infers narrower exceptions when rethrowing an effectively final catch parameter.
- **Idempotency:** repeated operation has an effect compatible with safe retry.
- **Ownership:** responsibility for closing or releasing a resource.

Detailed mechanics

Throwing and declaring

`throw` requires a `Throwable` reference; throwing null causes `NullPointerException`. A `throws` clause declares possible checked failures for callers but does not itself throw anything. Unchecked failures may be documented without being declared.

Checked exceptions can communicate a recoverable condition a caller is expected to consider. Unchecked exceptions commonly represent violated preconditions, programming defects, or failures handled at a wider boundary. This is an API design choice, not a guarantee that checked failures are recoverable or unchecked failures are fatal.

JAVA

```
static int parsePort(String text) {
    int port;
    try {
        port = Integer.parseInt(text);
    } catch (NumberFormatException ex) {
        throw new IllegalArgumentException("port is not numeric: " + text, ex);
    }
    if (port < 1 || port > 65_535) {
        throw new IllegalArgumentException("port out of range: " + port);
    }
    return port;
}
```

Translation keeps the original exception as cause and adds domain context. Avoid putting secrets in messages.

Catch selection and propagation

Catch clauses are tested in source order, and the first assignment-compatible type handles the throwable. Therefore a subtype catch must precede a supertype catch or the latter makes it unreachable. After the catch runs normally, execution continues after the whole try statement.

Multi-catch uses alternatives such as `catch (IOException | SQLException ex)`. Alternatives cannot be related by subclassing, and the catch parameter is implicitly final because assigning it would make its type unclear.

Precise rethrow lets this pattern declare the specific possible checked types rather than broad `Exception` when the caught parameter is not reassigned:

JAVA

```
static void invoke() throws java.io.IOException, java.sql.SQLException {
    try {
        callThatThrowsEither();
    } catch (Exception ex) {
        audit(ex);
        throw ex;
    }
}
```

This should not justify catching broad types routinely; it is useful at cross-cutting boundaries that must observe and rethrow unchanged.

Finally and abrupt completion

A `finally` block executes after its associated try/catch completes normally or abruptly, before control continues outward. If `finally` itself returns or throws, it replaces the pending return value or exception. That can silently lose the true failure.

JAVA

```
static int misleading() {
    try {
        return 1;
    } finally {
        return 2; // replaces the pending return; avoid this
    }
}
```

Cleanup may not run if the process halts, is externally killed, the JVM crashes, or execution cannot proceed. Resource safety is scoped, not a substitute for durable recovery protocols.

Try-with-resources

A resource type implements `AutoCloseable`; `Closeable` is a narrower I/O-oriented subtype whose `close` declares `IOException`. Resources initialize left to right and close in reverse order.

JAVA

```
try (var input = java.nio.file.Files.newBufferedReader(path);
    var output = java.nio.file.Files.newBufferedWriter(target)) {
    output.write(input.readLine());
}
```

If initialization of a later resource fails, already initialized earlier resources are closed. If the body throws and closing also throws, the body exception remains primary and close failures are attached through `getSuppressed()`. If the body succeeds but close fails, the close exception propagates. Multiple close failures retain the first propagating close failure as primary and later ones as suppressed according to reverse close order.

Since Java 9, an existing final or effectively final resource variable can appear directly in the resource specification:

JAVA

```
var reader = java.nio.file.Files.newBufferedReader(path);
try (reader) {
    System.out.println(reader.readLine());
}
```

The statement assumes ownership and closes it. The variable remains in scope afterward but denotes a closed resource; do not use it.

The conceptual compiler translation uses nested try/finally logic and `addSuppressed`. Writing that translation manually is error-prone, so use try-with-resources whenever ownership is lexical.

Rethrow, wrapping, and interruption

Use `throw ex;` to rethrow the same object and preserve its trace. Wrapping with `new DomainException(message, ex)` creates an abstraction boundary with a cause. `throw new DomainException(ex.getMessage())` discards the cause and should usually be rejected in review.

Do not catch `Throwable` to continue normal service; it includes serious `Error` conditions. Catching `Exception` at a request or job boundary can be appropriate to log, translate, and isolate one unit of work, provided fatal conditions and cancellation semantics remain intact.

`InterruptedException` communicates cooperative cancellation. Code that cannot propagate it should normally restore the status with `Thread.currentThread().interrupt()` and stop its work. Swallowing it can make shutdown and timeouts fail.

TRACE IT | Worked Java example

This example demonstrates primary and suppressed failures with two resources.

JAVA

```
public class SuppressionDemo {
    static final class Resource implements AutoCloseable {
        private final String name;

        Resource(String name) {
            this.name = name;
            System.out.println("open " + name);
        }

        @Override
        public void close() {
            System.out.println("close " + name);
            throw new IllegalStateException("close failed: " + name);
        }
    }

    public static void main(String[] args) {
        try (Resource first = new Resource("first");
            Resource second = new Resource("second")) {
            throw new IllegalArgumentException("body failed");
        } catch (Exception ex) {
            System.out.println("primary: " + ex.getMessage());
            for (Throwable suppressed : ex.getSuppressed()) {
                System.out.println("suppressed: " + suppressed.getMessage());
            }
        }
    }
}
```

The body failure stays primary, preserving the operation's causal event. Both cleanup failures remain discoverable for diagnosis.

TRACE IT | Execution or memory walkthrough

`first` initializes, then `second`. The body constructs and throws `IllegalArgumentException`. Before catch selection, resources close in reverse order. Closing `second` throws; because a body exception is already pending, this failure is added to the body's suppressed list. Closing `first` also throws and is added next.

The `catch (Exception ex)` matches the primary `IllegalArgumentException`. It prints `body failed`, followed by `close failed: second` and `close failed: first`. The close exceptions are not causes; they are independent secondary failures during cleanup.

Had resource construction for `second` thrown, the body would never begin, `first` would still close, and any close failure would be suppressed under the second-construction failure.

Complexity and performance

Normal try blocks do not inherently change algorithmic complexity. Throwing and capturing a stack trace can be expensive relative to ordinary branches, and exceptions should not represent expected high-volume control flow. Catch matching scales with propagation depth and handlers but is not usually the dominant production concern.

Try-with-resources adds close work that was required anyway. Its safety far outweighs tiny structural overhead. Stack traces and retained causes consume memory, especially if exceptions are accumulated rather than logged and released.

HotSpot note: HotSpot optimizes normal paths through exception regions and may omit or change some diagnostic detail for certain repeated VM-thrown exceptions. Do not depend on such implementation behavior; preserve useful application context explicitly.

WATCH OUT | Edge cases and common mistakes

- Catching a superclass before its subclass is unreachable code.
- Throwing from `finally` masks a pending exception; returning from `finally` masks both exceptions and returns.
- A `throws Exception` declaration exports implementation uncertainty and burdens every caller.
- Logging and rethrowing at every layer produces duplicate noise; log where context and ownership meet.
- Wrapping without a cause destroys the diagnostic chain.
- Catching and ignoring `InterruptedException` breaks cancellation.
- Retrying after an unknown partial side effect can duplicate writes.
- `AutoCloseable.close` is allowed to throw broad `Exception` and is not universally idempotent.
- Resource variables close in reverse declaration order.
- A caller-supplied stream should not be closed by a callee unless ownership transfer is explicit.
- `getCause()` and `getSuppressed()` represent different relationships.
- An exception message can leak tokens, SQL, customer data, or filesystem paths.

Production engineering notes

Define exceptions at architectural boundaries. Translate driver errors into repository failures and domain failures into stable API responses, while retaining the original cause internally. Give clients machine-readable error codes rather than parsing messages.

Retries require classification, idempotency, deadlines, bounded attempts, backoff, and jitter. Never classify only by broad exception type when the provider exposes more specific status. Preserve interruption and cancellation.

Use try-with-resources for files, JDBC statements/results, streams, locks represented by closeable guards, and telemetry scopes. Declare ownership in method names or documentation. At top-level request and worker boundaries, record exception class, safe context, trace or correlation ID, and suppressed failures without logging secrets.

TRY IT | Interview questions and model answers

What is the difference between checked and unchecked exceptions?

Checked exceptions are `Exception` subclasses other than `RuntimeException`; Java requires callers to catch or declare them. Runtime exceptions and errors are unchecked. The hierarchy affects compile-time handling, not an absolute recoverability classification.

What happens if both a try body and close throw?

In try-with-resources, the body exception remains primary. Close exceptions are attached as suppressed throwables in reverse close order. This preserves both the causal operation failure and cleanup evidence.

Why should you avoid return in finally?

It replaces a pending return or exception, silently changing behavior and losing diagnostics. `finally` should perform limited cleanup that does not determine the method outcome.

When should an exception be translated?

At an abstraction boundary where the lower-level type is not part of the caller's contract. The translated exception should add safe, actionable context and retain the original as its cause.

How should `InterruptedException` be handled?

Prefer propagating it so an owning layer can cancel. If an API cannot declare it, restore interrupt status, stop the current operation, and translate only if the boundary requires it. Do not swallow and continue.

TRY IT | Exercises

- 1 Predict primary and suppressed exceptions for three resources whose body and close methods fail.
- 2 Refactor manual file cleanup into try-with-resources and test initialization failure.
- 3 Design repository exception translation that preserves SQL cause without exposing SQL to clients.
- 4 Find all ways a return or throw in `finally` can alter a pending outcome.
- 5 Implement a retry loop that stops on interruption, deadline, permanent failure, and maximum attempts.
- 6 Document ownership for a method accepting an `InputStream`, then offer borrowing and ownership-transfer variants.

RECAP | Chapter summary

Exceptions represent abrupt completion and propagate to the first compatible handler. Checked status is a compile-time contract choice. `finally` participates in abrupt completion and can dangerously replace outcomes. Try-with-resources initializes left to right, closes right to left, and preserves secondary failures through suppression. Production failure handling retains causes, respects cancellation and ownership, translates at abstraction boundaries, and retries only with bounded, idempotent policy.

TRY IT | Revision checklist

- ☐ I can classify checked, runtime, and error types.
- ☐ I can trace catch selection and propagation.
- ☐ I distinguish causes from suppressed exceptions.
- ☐ I know try-with-resources initialization and reverse close order.
- ☐ I never return from `finally` and avoid throwing from cleanup.
- ☐ I preserve causes when translating failures.
- ☐ I propagate or restore interruption.
- ☐ I make resource ownership, retries, and client error contracts explicit.

SDE-2 CORE CHAPTER

Chapter 12 - Collections Framework Architecture

Learning objectives

By the end of this chapter, you should be able to:

- distinguish collection interfaces, implementations, algorithms, and views;
- choose between `List`, `Set`, `Queue`, `Deque`, and `Map` from behavioral requirements;
- explain optional operations, iteration order, mutability, and fail-fast iteration;
- reason about structural modification, backed views, equality, and bulk operations;
- state complexity as an implementation property, not an interface promise; and
- design collection-facing APIs that preserve invariants and ownership boundaries.

WHY IT WORKS | Why this matters at SDE-2

Collections sit in nearly every backend path: request aggregation, caches, indexes, batching, deduplication, scheduling, and persistence mapping. At SDE-2, naming a familiar class is not enough. You are expected to identify the required semantics first, defend complexity claims, understand aliasing, and notice when an innocent view or mutable key creates a correctness defect.

An interview question such as "Which collection would you use?" is usually testing a decision process. Does order matter? Are duplicates allowed? Is lookup by key required? Are nulls valid? Is mutation concurrent? Is a sorted range query needed? The framework gives a vocabulary for those constraints.

WHY IT WORKS | First-principles model

A collection is an object that owns or exposes a group of references. The Java Collections Framework separates four ideas:

- 1 Interfaces define behavioral contracts, such as uniqueness or positional access.
- 2 Implementations choose a data structure, memory layout, and performance profile.
- 3 Algorithms operate through interfaces, such as sorting or binary search.
- 4 Views expose a live projection of another object, such as a map's key set.

The principal hierarchy is:

TEXT

```
Iterable
  Collection
    List
    Set
      SortedSet
      NavigableSet
    Queue
    Deque

Map                                (not a Collection)
  SortedMap
  NavigableMap
```

Map is separate because it models key-value associations rather than individual elements. Its `keySet()`, `values()`, and `entrySet()` methods bridge into the **Collection** hierarchy through views.

Specification boundary: Interface contracts specify observable behavior. They generally do not promise array storage, linked nodes, hashing strategy, tree shape, amortized cost, or fail-fast detection. Complexity claims belong to the documentation of a concrete implementation.

Core terminology

- **Element:** A reference stored by a collection. Collections do not contain primitive values directly; boxing creates wrapper objects when needed.
- **Structural modification:** A change that can alter iteration structure, usually adding or removing elements. Replacing a value may or may not be structural for a particular implementation.
- **Encounter order:** The order in which an iteration mechanism presents elements. Some collections define it; others do not.
- **Natural ordering:** Ordering defined by `Comparable`.
- **Optional operation:** An interface method that an implementation may reject with `UnsupportedOperationException`.
- **View:** A projection backed by another object. Changes may be visible in both directions.
- **Snapshot:** An independent representation of state at a point in time.
- **Unmodifiable:** Mutation through that reference is rejected. It does not necessarily mean the underlying data or elements are immutable.
- **Immutable:** State cannot change after construction, including through aliases permitted by the abstraction.
- **Fail-fast iterator:** An iterator that attempts to detect unsupported concurrent structural modification and throw `ConcurrentModificationException`.

Detailed mechanics

Selecting the abstraction

Use a `List` when position, encounter order, or duplicates are meaningful. Use a `Set` for membership and uniqueness. Use a `Queue` when processing order matters and elements enter and leave through queue operations. Use a `Deque` for both ends or stack behavior. Use a `Map` when a unique key identifies a value.

Program parameters and return types to the narrowest useful interface. A method that only iterates can accept `Iterable<T>` or `Collection<T>` rather than `ArrayList<T>`. Do not hide an important semantic property, however: accepting `Set<T>` communicates uniqueness, while accepting `Collection<T>` does not.

Optional operations and capability

`Collection` includes methods such as `add`, `remove`, and `clear`, but implementations can reject them. `List.of(...)` produces an unmodifiable list. `Arrays.asList(array)` is fixed-size: `set` works, but size-changing methods do not. This design lets algorithms use a common interface, but it means the static type alone does not prove mutability.

Java has no standard type-level distinction between mutable and read-only collections. Document ownership and capability explicitly. Prefer immutable copies at public boundaries when callers should not mutate state:

JAVA

```
final class RoutingTable {
    private final List<String> backends;

    RoutingTable(Collection<String> backends) {
        this.backends = List.copyOf(backends);
    }

    List<String> backends() {
        return backends;
    }
}
```

`List.copyOf` also rejects null elements. That is useful only if null rejection matches the domain contract.

Iterators and structural modification

An `Iterator` is a cursor with `hasNext`, `next`, and optional `remove`. Enhanced `for` normally uses it. Removing directly from a typical collection during iteration can invalidate cursor state:

JAVA

```
for (String id : ids) {
    if (id.isBlank()) {
        ids.remove(id); // usually wrong
    }
}
```

Use `Iterator.remove`, `removeIf`, or a separate result collection. The iterator's own removal operation updates both the structure and its bookkeeping.

HotSpot note: Common OpenJDK collections maintain a modification counter and compare it with an iterator's expected value. This is implementation detail and detection is best-effort. `ConcurrentModificationException` is a bug signal, not a synchronization mechanism. Exact fields and detection points are version-sensitive.

Views and aliasing

`map.keySet()`, `map.values()`, and `map.entrySet()` are normally backed views. Removing a key from the key set removes the mapping. An entry's `setValue` can update the map when supported. `list.subList(from, to)` is also a backed view. A structural change to the parent outside the sublist can make later sublist operations fail.

Wrappers from `Collections.unmodifiableList(source)` are read-only views, not copies. If another alias mutates `source`, the wrapper reflects it. By contrast, `List.copyOf(source)` returns an unmodifiable list whose membership is not backed by later source changes. Neither operation freezes mutable element objects.

Equality and bulk operations

`List.equals` is order-sensitive. `Set.equals` compares membership independent of order. `Map.equals` compares mappings. These contracts enable equality across implementations: an `ArrayList` can equal a `LinkedList` with the same sequence.

Bulk methods include `addAll`, `removeAll`, `retainAll`, `containsAll`, `removeIf`, and `replaceAll`. Their apparent single-call form does not imply constant time. Cost depends on both receiver and argument. For example, removing every element found in an `ArrayList` argument can be quadratic when membership checks are linear. Converting the lookup side to a `HashSet` can improve the expected bound.

Null policy and element validity

Null support varies. Some general-purpose collections allow null; many queues, concurrent collections, sorted structures with natural ordering, and factory-created immutable collections reject it. A robust API validates domain values before selecting an implementation rather than relying on an incidental null policy.

TRACE IT | Worked Java example

The following method groups unique order IDs by customer while preserving first-seen customer order and first-seen order ID order:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

record Order(String id, String customerId) {}

final class OrderIndex {
    static Map<String, List<String>> build(Collection<Order> orders) {
        Map<String, Set<String>> working = new LinkedHashMap<>();

        for (Order order : orders) {
            if (order == null || order.id() == null || order.customerId() == null) {
                throw new IllegalArgumentException("order fields must be non-null");
            }
            working.computeIfAbsent(
                order.customerId(), ignored -> new LinkedHashSet<>())
```

JAVA (continued 2/2)

```
                .add(order.id());
        }

        Map<String, List<String>> result = new LinkedHashMap<>();
        working.forEach((customer, ids) ->
            result.put(customer, List.copyOf(ids)));
        return Map.copyOf(result);
    }

    public static void main(String[] args) {
        List<Order> input = new ArrayList<>();
        input.add(new Order("o-1", "c-7"));
        input.add(new Order("o-2", "c-8"));
        input.add(new Order("o-1", "c-7"));
        input.add(new Order("o-3", "c-7"));

        System.out.println(build(input));
    }
}
```

One subtlety: `Map.copyOf` guarantees an unmodifiable map, but its iteration order is not specified to preserve the insertion order of the supplied `LinkedHashMap`. If public iteration order is part of the result contract, return an unmodifiable copy with an order-preserving implementation, for example `Collections.unmodifiableMap(new LinkedHashMap<>(result))`, and document the guarantee.

TRACE IT | Execution or memory walkthrough

For the four inputs above:

- 1 `o-1/c-7` creates the first map entry and a set containing `o-1`.
- 2 `o-2/c-8` creates a second entry. The map's encounter order is now `c-7`, `c-8`.
- 3 The duplicate `o-1/c-7` reaches the existing set. `Set.add` returns `false`; membership remains unchanged.
- 4 `o-3/c-7` appends a second unique ID to the first customer's insertion-ordered set.
- 5 The conversion creates new lists, so clients cannot add or remove IDs through the result.

The working representation uses a map object, map entries, set objects, set entries, and references to existing strings. The result adds list storage and map entries. Peak memory includes both representations until the method returns and the working map becomes unreachable. The code copies collection structure, not `String` content; strings are immutable, so sharing them is safe.

Complexity and performance

Let n be the number of orders and u the number of unique customer-order pairs. With typical `LinkedHashMap` and `LinkedHashSet` behavior, building has expected $O(n)$ time and $O(u)$ space; producing the result costs $O(u)$. Hash collisions, resizing, and key methods affect constants and worst cases.

Interface-only complexity should be stated cautiously:

Operation	Interface guarantee	Common implementation example
<code>List.get(i)</code>	No bound	<code>ArrayList</code> : $O(1)$; <code>LinkedList</code> : $O(n)$
<code>Set.contains(x)</code>	No bound	<code>HashSet</code> : expected $O(1)$; <code>TreeSet</code> : $O(\log n)$
<code>Map.get(k)</code>	No bound	<code>HashMap</code> : expected $O(1)$; <code>TreeMap</code> : $O(\log n)$
iteration	Semantic order may vary	Usually $O(n)$, but hash-table capacity can affect traversal
<code>containsAll</code>	No bound	Depends on sizes and membership cost

Memory is often as important as asymptotic time. Node-based structures add per-element objects and pointers. Array-based structures may reserve unused capacity but provide locality. Measure representative workloads rather than extrapolating from Big-O alone.

WATCH OUT | Edge cases and common mistakes

- Choosing a concrete type before defining uniqueness, ordering, and access semantics.
- Returning a mutable internal list and unintentionally granting write access.
- Assuming an unmodifiable wrapper is a snapshot or that immutable membership makes elements immutable.
- Depending on `HashMap` or `HashSet` iteration order.
- Modifying a collection directly while iterating over it.
- Treating `ConcurrentModificationException` as guaranteed detection of a race.
- Forgetting that `subList` and map collection views are backed by their owner.
- Passing a collection to itself in a bulk operation whose behavior is undefined or surprising.
- Using mutable objects whose `equals`, `hashCode`, or ordering fields change while stored.
- Assuming every implementation permits null or mutation.
- Calling `size()` repeatedly on a nonstandard collection without checking whether it is cheap.
- Confusing `Collection` with `Collections`; the latter is an algorithm and wrapper utility class.

Production engineering notes

Define collection contracts in API documentation: order, duplicates, null policy, mutability, snapshot versus live view, thread safety, and expected scale. A return type of `List<T>` communicates sequence but not ownership.

Defensively copy inputs when the component must own stable membership. For very large inputs, copying can be costly; an explicit ownership-transfer convention or immutable persistent data structure may be more appropriate. Never expose a lazily changing view across layers unless that behavior is deliberate.

Avoid sharing ordinary mutable collections across threads without a synchronization policy. A synchronized wrapper makes individual calls mutually exclusive, but compound actions such as "check then add" still need one lock scope. Concurrent collections have different iteration and atomicity contracts and should be selected explicitly.

Prefer domain operations over raw collection exposure. `routingTable.chooseBackend()` protects invariants better than `routingTable.backends().remove(0)`. Validate capacity and input size where untrusted requests can cause large allocations. Instrument unusually large collection sizes and expensive conversions in latency-sensitive paths.

TRY IT | Interview questions and model answers

Why is `Map` not a subtype of `Collection`?

A collection models individual elements, while a map models key-value mappings with key uniqueness. `Map` exposes collection views for keys, values, and entries, but operations such as adding one arbitrary element do not fit its contract.

What does fail-fast mean?

It means an iterator may detect an unsupported structural modification and throw `ConcurrentModificationException` promptly. It is best-effort behavior in common implementations, not a thread-safety guarantee or a correctness mechanism.

What is the difference between an unmodifiable view and an immutable copy?

An unmodifiable view rejects mutation through that reference but can reflect mutations through another alias. An immutable copy has independent, unchangeable membership. In both cases, referenced element objects can still be mutable unless separately constrained.

Why accept an interface rather than `ArrayList`?

It minimizes coupling and admits any implementation satisfying the required behavior. The chosen interface should still express semantics: `Set` is preferable to `Collection` when uniqueness is required.

Can interface type tell you complexity?

Usually no. `List.get` can be constant or linear depending on implementation. State the concrete type and assumptions whenever making a complexity claim.

How do backed views affect correctness?

They create aliases. Mutating the owner can change the view, and supported mutations through the view can change the owner. They are useful for efficient projections but dangerous if callers assume snapshots.

TRY IT | Exercises

- 1 Design a method signature for returning active feature flags in deterministic order without allowing callers to mutate membership. State every contract not captured by the type.
- 2 Predict the result of removing an entry through `map.entrySet().iterator().remove()` and explain why it differs from mutating the map directly during iteration.
- 3 Compare `Collections.unmodifiableList(source)`, `List.copyOf(source)`, and `new ArrayList<>(source)` for mutability, null handling, and aliasing.
- 4 Rewrite a quadratic `removeAll` workflow so membership tests use a set. Give time and space bounds.
- 5 Create a small program that demonstrates a `subList` backed view. Then structurally modify the parent outside the view and record the observed behavior without treating it as a portable synchronization technique.

RECAP | Chapter summary

The Collections Framework separates behavioral interfaces from data-structure implementations, reusable algorithms, and backed views. A sound choice begins with semantics: order, uniqueness, lookup, mutation, concurrency, and ownership. Complexity belongs to concrete implementations. Iterators, views, and optional operations make the framework flexible, but they also create important capability and aliasing boundaries. Production APIs should state those boundaries explicitly and copy or wrap data according to an intentional ownership model.

TRY IT | Revision checklist

- ☐ I can sketch the **Collection** hierarchy and explain why **Map** is separate.
- ☐ I can choose among **List**, **Set**, **Queue**, **Deque**, and **Map** from requirements.
- ☐ I distinguish interface contracts from implementation complexity.
- ☐ I understand optional operations and common fixed-size or unmodifiable collections.
- ☐ I can explain structural modification and correct iterator removal.
- ☐ I can distinguish a view, an unmodifiable wrapper, a shallow copy, and deep immutability.
- ☐ I know the equality semantics of lists, sets, and maps.
- ☐ I document order, nulls, ownership, thread safety, and expected scale in APIs.

SDE-2 CORE CHAPTER

Chapter 13 - Comparable, Comparator, Sorting, and Selection

Learning objectives

By the end of this chapter, you should be able to:

- define natural and external orderings with valid comparison contracts;
- compose null-safe, deterministic comparators without overflow;
- explain stable sorting, in-place sorting, and comparison lower bounds;
- choose among object sort, primitive sort, parallel sort, heap selection, and quickselect;
- use binary search only under its ordering precondition; and
- discuss current OpenJDK algorithms without confusing them with API guarantees.

WHY IT WORKS | Why this matters at SDE-2

Sorting is often the boundary between raw data and useful output: ranked results, merge pipelines, pagination, reconciliation, and deduplication all depend on ordering. Comparison defects can be intermittent and severe. A non-transitive comparator may make sorting fail at runtime, corrupt a tree collection's model, or create unstable pagination.

At SDE-2, you should translate a product rule into a total order, handle ties explicitly, and select an algorithm based on whether all values or only the top *k* are needed. You should distinguish specification promises, such as stability for an object sort, from OpenJDK's current TimSort or primitive-array algorithms.

WHY IT WORKS | First-principles model

An ordering comparator maps two values to a negative number, zero, or a positive number:

TEXT

```
compare(a, b) < 0  means a precedes b
compare(a, b) = 0  means a and b are ordering-equivalent
compare(a, b) > 0  means a follows b
```

The magnitude is irrelevant. A comparator must behave like a total order over the accepted domain. Important laws are:

- sign symmetry: `sign(compare(a,b)) == -sign(compare(b,a));`

- transitivity: if $a > b$ and $b > c$, then $a > c$;
- zero consistency: if a compares equal to b , comparisons of each against c have the same sign; and
- repeatability while participating fields remain unchanged.

Comparison sorting uses pairwise questions to distinguish possible permutations. There are $n!$ permutations, so a decision tree requires height $\Omega(\log(n!))$, which is $\Omega(n \log n)$. General comparison sorting cannot asymptotically beat that bound. Algorithms using restricted keys, such as counting sort, operate under different assumptions.

Specification boundary: `Comparable` and `Comparator` define ordering contracts. Sorting APIs document properties such as stability and permitted mutation. Their exact algorithm, run detection, pivot selection, temporary storage, and thresholds can vary by JDK implementation and version.

Core terminology

- **Natural order:** A type's canonical order implemented by `Comparable`.
- **External order:** A purpose-specific `Comparator` supplied by a client.
- **Stable sort:** Equal-order elements retain input relative order.
- **In-place:** Uses only bounded or logarithmic auxiliary storage under a stated model; common library documentation may use looser wording.
- **Adaptive sort:** Exploits existing order or runs in the input.
- **Total order:** Consistent ordering for every accepted pair.
- **Partial selection:** Find a rank or top subset without fully sorting.
- **Quickselect:** Partition-based expected linear-time selection.
- **Partition:** Rearrange elements around a pivot by comparison.
- **Tie-breaker:** Additional field distinguishing otherwise equal rankings.
- **Schwartzian transform/decorate-sort-undecorate:** Precompute expensive sort keys, sort decorated values, then extract originals.

Detailed mechanics

Comparable versus Comparator

Implement `Comparable<T>` when a type has one unsurprising natural order used broadly:

JAVA

```
record Version(int major, int minor, int patch)
    implements Comparable<Version> {
    @Override
    public int compareTo(Version other) {
        int result = Integer.compare(major, other.major);
        if (result != 0) return result;
        result = Integer.compare(minor, other.minor);
        if (result != 0) return result;
        return Integer.compare(patch, other.patch);
    }
}
```

Use comparators for alternative views: orders by creation time, priority, customer, or amount. A natural order becomes part of a public type's long-lived meaning, so do not add one merely for a single screen.

Natural ordering should usually be consistent with `equals`, especially when values enter sorted sets or maps. `BigDecimal` is a notable counterexample: values such as `1.0` and `1.00` compare as zero but are not equal because scale affects `equals`. A `TreeSet<BigDecimal>` can therefore have different uniqueness behavior from a `HashSet<BigDecimal>`.

Comparator composition

Comparator factories make intent explicit:

JAVA

```
Comparator<Order> order = Comparator
    .comparingInt(Order::priority).reversed()
    .thenComparing(Order::createdAt)
    .thenComparing(Order::id);
```

Apply reversal carefully. Calling `reversed()` at the end reverses the entire chain. Calling it after the primary comparator reverses only that comparator before adding subsequent ascending tie-breakers.

Use primitive factories to avoid boxing in comparison: `comparingInt`, `comparingLong`, and `comparingDouble`. Use `Comparator.nullsFirst` or `nullsLast` only if null is valid domain data. Do not compare integers with `a - b`; overflow can violate the sign rule. Floats and doubles also require library comparison because NaN and signed zero need a consistent total order.

Stability and tie-breaking

Stable sort preserves input order when comparator returns zero. This permits multi-pass sorting: stable-sort by a secondary key, then by primary key. A single comparator chain is usually clearer and performs fewer sorts.

Stability is not a substitute for a deterministic total presentation order. If input arrives from a hash map or concurrent source with unspecified encounter order, stable sorting equal ranks preserves an unspecified order. Add a unique immutable ID as a tie-breaker for reproducible pagination and tests.

Library sorting APIs

`List.sort(comparator)` mutates the list and is specified as stable. `Collections.sort` delegates to the list's sorting facility in modern APIs. The list must support the required replacement operation; an unmodifiable list rejects sorting.

`Arrays.sort(Object[])` is stable. Primitive overloads are not required to be stable because primitive values have no separately observable identity when equal. Primitive sorts avoid boxing and are normally much more memory-efficient.

`Arrays.parallelSort` may use the common fork-join pool and temporary storage. Parallel overhead can outweigh gains for small arrays or contended services. Measure on the deployment shape, and remember that comparator work must be thread-safe and side-effect-free.

HotSpot note: Current OpenJDK releases commonly use TimSort-derived logic for object arrays and lists, dual-pivot quicksort and specialized paths for several primitive types, and parallel merge/sort strategies above thresholds. Algorithms and thresholds are version-sensitive.

TimSort intuition and comparator failures

TimSort is adaptive: it discovers ascending or descending runs, extends short runs, and merges runs while maintaining stack invariants. Nearly sorted input can therefore be efficient. It needs temporary storage for merging.

Inconsistent comparators can trigger exceptions such as "comparison method violates its general contract" in some sort paths, but detection is not guaranteed. A sort that appears to complete does not prove comparator correctness.

Selection instead of full sorting

If only the smallest or largest k items are needed:

- sort all: $O(n \log n)$ time, straightforward, produces complete order;
- maintain a heap of size k : $O(n \log k)$ time and $O(k)$ extra space;
- quickselect: expected $O(n)$ to place the k th rank, then optionally sort the selected k ; worst case $O(n^2)$ without robust pivot strategy;
- counting/bucket techniques: potentially $O(n + \text{range})$ when integer key range is small and bounded.

Quickselect partitions around a pivot. After partition, if the pivot lands at rank k , selection is complete; otherwise recurse or iterate only on the side containing rank k . It mutates the array unless implemented on a copy.

Binary search

`Collections.binarySearch` and `Arrays.binarySearch` require data sorted according to the same ordering. A nonnegative result is a matching index. A negative result encodes insertion point $-(\text{result}) - 1$. When duplicates exist, no promise should be inferred about which equal occurrence is returned unless the API states one. Use explicit lower-bound and upper-bound searches for ranges of duplicates.

TRACE IT | Worked Java example

This method returns the top k candidates while bounding intermediate storage:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

record Candidate(String id, int score, long completedAtEpochMillis) {}

final class Ranking {
    // Best first: higher score, earlier completion, lexicographically smaller ID.
    private static final Comparator<Candidate> BEST_FIRST = Comparator
        .comparingInt(Candidate::score).reversed()
        .thenComparingLong(Candidate::completedAtEpochMillis)
        .thenComparing(Candidate::id);

    static List<Candidate> topK(Collection<Candidate> input, int k) {
        if (k < 0) throw new IllegalArgumentException("k must be non-negative");
        if (k == 0) return List.of();
    }
}
```

JAVA (continued 2/2)

```
// Worst retained candidate is at the head.
PriorityQueue<Candidate> retained =
    new PriorityQueue<>(k, BEST_FIRST.reversed());

for (Candidate candidate : input) {
    if (candidate == null) throw new IllegalArgumentException("null candidate");
    if (retained.size() < k) {
        retained.offer(candidate);
    } else if (BEST_FIRST.compare(candidate, retained.peek()) < 0) {
        retained.poll();
        retained.offer(candidate);
    }
}

ArrayList<Candidate> result = new ArrayList<>(retained);
result.sort(BEST_FIRST);
return List.copyOf(result);
}
```

The ID tie-breaker gives a deterministic total ranking. If IDs are unique, no two distinct candidates compare as zero. The heap uses reverse order so its head is the worst candidate currently retained; a better incoming candidate replaces it.

TRACE IT | Execution or memory walkthrough

For $k = 3$ and candidates with scores 70/A, 90/B, 80/C, 85/D, and 80/E, assume times and IDs do not change the score comparisons:

- 1 Retain A, B, C. The heap head is A, the worst of those three.
- 2 D outranks A, so poll A and add D. Retained set is B, C, D; C is now worst.
- 3 E ties C on score. Completion time and then ID decide whether E is better. Replace only if the full comparator ranks E before C.
- 4 Copy the heap. Its iteration order is not ranked.
- 5 Sort the copy best-first and publish an unmodifiable result.

At most k candidate references reside in the heap, plus the final list of at most k references. Candidate records are shared rather than copied. Comparator calls extract primitive fields without boxing for score and time.

Quickselect would store the entire mutable input or a copy but could reduce asymptotic selection time. The heap supports one-pass input and is appropriate when the collection is streamed or k is small.

Complexity and performance

For **topK**, each of n inputs performs constant comparison plus at most one heap replacement costing $O(\log k)$. Time is $O(n \log k + k \log k)$ and auxiliary reference storage is $O(k)$. When $k \geq n$, the behavior approaches $O(n \log n)$ and sorting all may be simpler and faster.

Common sorting bounds:

Technique	Time	Extra space	Stable	Best use
comparison sort	$O(n \log n)$ typical/guaranteed by chosen algorithm	varies	API-dependent	full order
insertion sort	$O(n^2)$, near $O(n)$ when nearly sorted	$O(1)$	yes when implemented conventionally	tiny/nearly sorted ranges
merge sort	$O(n \log n)$	$O(n)$	yes	stable predictable sorting
heap sort	$O(n \log n)$	$O(1)$ array model	no	worst-case bound, low extra space
quicksort	average $O(n \log n)$, worst $O(n^2)$	recursion/stack varies	usually no	fast primitive/in-place-style sorting
heap top-k	$O(n \log k)$	$O(k)$	only with explicit tie order	small top subset
quickselect	expected $O(n)$, worst $O(n^2)$	often $O(1)$ iterative	no	one rank or unsorted partition

Comparator cost can dominate $n \log n$. Avoid network calls, database access, mutable clocks, locale creation, or repeated expensive parsing inside comparison. Precompute keys when profiling justifies it.

WATCH OUT | Edge cases and common mistakes

- Implementing compare with subtraction and overflowing.
- Omitting a stable tie-breaker for pagination or distributed merge results.
- Assuming stable sort makes unspecified input order deterministic.
- Using a comparator that changes based on mutable state, current time, or side effects.
- Mixing natural and external ordering between sort and binary search.
- Expecting binary search to return the first duplicate.
- Sorting an unmodifiable or fixed-capability list without understanding supported operations.
- Mutating comparator fields while objects reside in a sorted collection.
- Assuming primitive and object sort stability are identical.
- Parallelizing a small sort or using a non-thread-safe comparator with parallel sort.
- Fully sorting millions of items to return a tiny top-k result.
- Forgetting that `reversed()` placement can reverse an entire comparator chain.
- Treating current OpenJDK algorithm names or thresholds as portable guarantees.

Production engineering notes

Define ordering once and reuse it across database queries, in-memory merge, API pagination, and tests. Any mismatch can cause duplicates or gaps between pages. Cursor pagination needs a unique final tie-breaker included in both ordering and cursor encoding.

Comparator functions should be pure, cheap, null-explicit, and tested with property-style checks for symmetry and transitivity. Normalize text before sorting if case, Unicode, or locale rules matter. Human-language collation is not equivalent to `String` code-unit order and may depend on locale/version; isolate it from identity ordering.

Avoid sorting shared mutable lists in place. Copy, sort, and publish when readers need snapshots. For large results, push sorting and limiting toward an indexed data source when possible. Enforce input caps for user-controlled sorts to prevent CPU and memory abuse.

Measure sequential versus parallel sorting in realistic service contention. A common pool is shared process capacity, not free compute. Primitive arrays avoid wrapper allocation and should be preferred for numerical kernels when the surrounding design permits them.

TRY IT | Interview questions and model answers

When should a class implement `Comparable`?

When it has one canonical, unsurprising natural order that is meaningful across uses. Purpose-specific orders belong in named comparators.

What properties must a comparator satisfy?

It must provide consistent sign symmetry, transitivity, and zero behavior over its accepted domain. It should be repeatable while compared state is unchanged and preferably consistent with equals for collection use.

What does stable sort mean?

Elements that compare as equal retain their original relative order. It does not create determinism if original encounter order is unspecified.

How would you find the top 100 of ten million values?

Use a size-100 min-heap while scanning: $O(n \log 100)$ time and $O(100)$ memory, then sort the retained items for final output. Discuss database pushdown or distributed merge if data is external.

Why can quickselect be faster than sorting?

It explores only the partition containing the target rank, giving expected linear work. It does not fully order the data and has quadratic worst cases without pivot safeguards.

Can I rely on TimSort for every Java object sort?

Rely on the documented stability and behavior of the API, not an algorithm name. TimSort is a common current OpenJDK implementation and can change.

TRY IT | Exercises

- 1 Write a comparator for nullable invoices ordered by status, descending amount, due date, and unique ID. State null policy.
- 2 Produce a three-value counterexample showing how a bad cyclic comparator violates transitivity.
- 3 Implement lower-bound binary search that returns the first index whose element is not less than a target.
- 4 Compare full sort, heap top-k, and quickselect for $n = 1,000,000$ and $k = 10$, then for $k = 900,000$.
- 5 Repair a comparator written as $(a, b) \rightarrow a.timestamp() > b.timestamp() ? 1 : -1$.
- 6 Explain why stable sorting entries from a `HashMap` by value alone can still produce changing output.

RECAP | Chapter summary

Ordering is a correctness contract before it is an algorithm. `Comparable` defines a canonical natural order; `Comparator` defines external orders that can be composed with safe tie-breakers. Stable full sorting solves complete ordering in $O(n \log n)$, while heaps and selection algorithms avoid unnecessary work for small subsets. Java APIs specify observable properties, whereas TimSort, primitive quicksort variants, and thresholds are current implementation choices. Production ordering must align across layers and remain deterministic, pure, and bounded.

TRY IT | Revision checklist

- ☐ I can state comparator laws and consistency-with-equals concerns.
- ☐ I use safe primitive comparisons rather than subtraction.
- ☐ I understand reversal placement and comparator chaining.
- ☐ I distinguish stability from deterministic total ordering.
- ☐ I know object, primitive, and parallel sort trade-offs.
- ☐ I can choose full sort, heap top-k, or quickselect from requirements.
- ☐ I use binary search only with the same ordering used for sorting.
- ☐ I label OpenJDK sorting algorithms and thresholds as version-sensitive.

SDE-2 CORE CHAPTER

Chapter 14 - Practice Lab C - Wrappers, Generics, Collections, Exceptions, I/O

Start here: select an abstraction from the required behavior before naming an implementation. State ordering, duplicates, null policy, mutation, lookup direction, failure behavior, and resource ownership. Only then discuss expected cost or write code.

Answer K51-K75, predict O39-O57, compile every D39-D57 repair, and implement C39-C57 with one normal and two boundary tests. For exception and I/O tasks, mark the ownership boundary and decide whether the method catches, translates, or propagates each failure. Use Solution Studio C after attempting the work; a remembered API name without its contract does not count as mastery.

Knowledge check

- K51 [Foundation] Map each primitive to its wrapper class.
- K52 [Foundation] Distinguish parsing, boxing, and unboxing.
- K53 [Interview Core] Why can null unboxing fail far from the source of null?
- K54 [Interview Core] Why is wrapper identity not a value contract?
- K55 [Foundation] What type safety does `List<String>` provide?
- K56 [Foundation] Why can primitives not be generic arguments?
- K57 [Interview Core] What does the diamond operator infer?
- K58 [Interview Core] What operation does a simple upper bound permit?
- K59 [Foundation] Why is an enum safer than magic strings for finite state?
- K60 [Interview Core] Why should enum ordinal rarely be persisted?
- K61 [Foundation] Choose List, Set, Map, Queue, Deque, or PriorityQueue by need.
- K62 [Interview Core] Why is `LinkedList` not always faster for insertion?
- K63 [Interview Core] Qualify the expected-cost claim for `HashMap`.
- K64 [Foundation] Distinguish `offer/poll/peek` from throwing queue methods.
- K65 [Interview Core] Why does PriorityQueue iteration not establish order?
- K66 [Interview Core] State a safe comparator sign contract.
- K67 [Foundation] Distinguish checked exceptions, unchecked exceptions, and errors.
- K68 [Interview Core] Distinguish `throw` from `throws`.
- K69 [Interview Core] What does try-with-resources guarantee about close order?
- K70 [Interview Core] Why must catch blocks go from specific to general?
- K71 [Foundation] When is `Scanner` sufficient?
- K72 [Interview Core] Why can `BufferedReader` plus token parsing be preferable?
- K73 [Interview Core] Who should close a resource?
- K74 [Interview Core] Distinguish fixed-size, unmodifiable, and immutable collections.
- K75 [SDE-2 Follow-up] Why is broad `catch (Exception)` dangerous inside core logic?

Predict the output

- **O39 [Foundation]** Predict `Integer.parseInt("42") + 1`.
- **O40 [Interview Core]** Predict unboxing a null `Integer`.
- **O41 [Interview Core]** Predict `Integer.valueOf(127) == Integer.valueOf(127)`.
- **O42 [Interview Core]** Predict value equality for two boxed 128 values.
- **O43 [Foundation]** Predict `new Box<>("x").value()`.
- **O44 [Foundation]** Predict an enum switch result for `PAID`.
- **O45 [Foundation]** Predict a `HashSet` size after adding 2, 2, and 3.
- **O46 [Foundation]** Predict a frequency map count after merging three equal keys.
- **O47 [Foundation]** Predict FIFO removals from an `ArrayDeque`.
- **O48 [Foundation]** Predict LIFO removals using `push` and `pop`.
- **O49 [Interview Core]** Predict the first value polled from a min-heap containing 9, 3, 5.
- **O50 [Interview Core]** Predict comparator sign from `Integer.compare(MAX_VALUE, -1)`.
- **O51 [Foundation]** Predict `Arrays.asList("a","b").set(0,"x")` contents.
- **O52 [Interview Core]** Predict the result of adding to that fixed-size list.
- **O53 [Interview Core]** Predict the size of `Arrays.asList(new int[]{1,2})`.
- **O54 [Interview Core]** Predict which catch handles `NumberFormatException`.
- **O55 [Interview Core]** Predict resource close order for two declared resources.
- **O56 [Foundation]** Predict `Character.digit('A', 16)`.
- **O57 [SDE-2 Follow-up]** Predict `Math.addExact(Integer.MAX_VALUE, 1)`.

Debug the code

- **D39 [Foundation]** Repair `List<int>`.
- **D40 [Interview Core]** Repair a nullable wrapper used in arithmetic.
- **D41 [Interview Core]** Repair wrapper comparisons that use `==`.
- **D42 [Foundation]** Repair a raw `List` that permits mixed values.
- **D43 [Interview Core]** Repair a generic `first` method that accepts an empty list silently.
- **D44 [Foundation]** Replace string order states with an enum.
- **D45 [Interview Core]** Repair a switch that forgets a new enum state.
- **D46 [Foundation]** Repair a deduplication loop that still appends duplicates.
- **D47 [Interview Core]** Repair a frequency map that treats missing as null arithmetic.
- **D48 [Interview Core]** Replace legacy `Stack` with `ArrayDeque` operations.
- **D49 [Interview Core]** Repair a comparator implemented as subtraction.
- **D50 [Interview Core]** Repair code that assumes `PriorityQueue` iteration is sorted.
- **D51 [Interview Core]** Repair enhanced-for structural removal.
- **D52 [Interview Core]** Repair a mutable hash-map key.
- **D53 [Foundation]** Repair catch blocks ordered superclass before subclass.
- **D54 [Interview Core]** Repair a `finally` block that returns.
- **D55 [Interview Core]** Replace manual resource closing with `try-with-resources`.
- **D56 [Foundation]** Repair `Scanner` code that reads an empty token after mixed line/token calls.
- **D57 [SDE-2 Follow-up]** Repair library code that closes a caller-owned stream.

Small coding tasks

- **C39 [Foundation]** Parse an integer and report invalid input without hiding other defects.
- **C40 [Interview Core]** Convert nullable boxed counts to a defined primitive total safely.
- **C41 [Foundation]** Implement a generic immutable `Box<T>`.
- **C42 [Interview Core]** Implement a generic `first` method with an empty-input contract.
- **C43 [Foundation]** Model order status with an enum method `isTerminal`.
- **C44 [Foundation]** Use `ArrayList` to build a result sequence.
- **C45 [Foundation]** Deduplicate integers with `HashSet`.
- **C46 [Interview Core]** Preserve first-seen order with `LinkedHashSet`.
- **C47 [Interview Core]** Count word frequencies with `HashMap.merge`.
- **C48 [Foundation]** Implement FIFO operations with `ArrayDeque`.
- **C49 [Foundation]** Implement LIFO operations with `ArrayDeque`.
- **C50 [Interview Core]** Find three smallest values with `PriorityQueue`.
- **C51 [Interview Core]** Sort records using `comparingInt` and a tie-breaker.
- **C52 [Interview Core]** Remove matching list elements safely.
- **C53 [Foundation]** Catch and explain `NumberFormatException` at an input boundary.
- **C54 [Interview Core]** Implement a checked file-reading helper with a clear throws contract.
- **C55 [Interview Core]** Use try-with-resources with `BufferedReader`.
- **C56 [Foundation]** Read an integer array with `Scanner`.
- **C57 [SDE-2 Follow-up]** Read and validate a jagged matrix with `BufferedReader` and `StringTokenizer`.

Interview follow-ups

- **F27 [Interview Core]** When does boxing create hidden null or allocation risk?
- **F28 [SDE-2 Follow-up]** Why should a cache range never become business logic?
- **F29 [Interview Core]** What error does generics prevent before runtime?
- **F30 [Interview Core]** When should a generic bound stay out of a beginner API?
- **F31 [Interview Core]** How does enum evolution affect persistence and switches?
- **F32 [SDE-2 Follow-up]** Defend a collection choice using semantics before complexity.
- **F33 [Interview Core]** When is insertion order a correctness requirement?
- **F34 [Interview Core]** Why is ArrayDeque usually preferred to Stack?
- **F35 [SDE-2 Follow-up]** Explain PriorityQueue's head guarantee and iterator non-guarantee.
- **F36 [Interview Core]** When should an unchecked exception represent a violated precondition?
- **F37 [SDE-2 Follow-up]** Preserve a primary exception when resource closing also fails.
- **F38 [SDE-2 Follow-up]** Define resource ownership in a reusable parsing method.

Cumulative assessment 3

A03: In 45 minutes, read tokens, build a frequency map, keep the first-seen order of unique values, and print the top three by frequency using a safe comparator. Define null, malformed-input, tie, and empty-input behavior.

Cumulative assessment 4

A04: Review a 40-line solution that uses raw collections, wrapper identity, comparator subtraction, broad catches, and a caller-owned reader. Produce a behavior-preserving refactor and an evidence-based defect list.

SDE-2 CORE CHAPTER

Chapter 15 - The Java Coding Interview Playbook

Learning objectives

By the end of this chapter, you should be able to:

- run a repeatable control loop from clarification through final complexity;
- convert examples and constraints into a baseline, pattern, invariant, and proof;
- communicate while coding without narrating every keystroke;
- write compact, compilable Java 21 with deliberate numeric and collection choices;
- test with a boundary matrix and recover systematically from defects; and
- handle optimization, follow-ups, and production questions at SDE-2 depth.

WHY IT WORKS | Why this matters at SDE-2

Knowledge does not automatically become interview performance. Time pressure causes candidates to skip the contract, silently assume input properties, start coding a half-remembered pattern, and discover an off-by-one error near the end. A playbook preserves reasoning bandwidth.

The interviewer is evaluating several channels at once: correctness, problem decomposition, complexity, communication, code quality, testing, and response to feedback. A candidate who reaches code slightly later but keeps the solution controlled often performs better than one who types immediately and debugs by instinct.

This chapter is not a script to recite. It is a feedback loop. Each artifact - contract, example, baseline, invariant, dry run, and complexity statement - makes the next decision safer and gives the interviewer a chance to correct a misunderstanding early.

Focused practice path: The 18-stage companion series turns this playbook into shorter topic loops. Begin with the first volume whose completion check is not yet automatic, and use Stage 18G for question-bank, mock-loop, and final revision work.

WHY IT WORKS | First-principles model

A coding interview is a constrained engineering session. The problem statement defines an incomplete contract. Input bounds create a performance budget. The solution is a claim that an implementation satisfies the clarified contract within that budget. Evidence consists of an invariant or recurrence, a dry run, tests, and complexity analysis.

Treat the session as a control system:

TEXT

```
observe requirements -> propose model -> receive feedback -> implement
      ^                               |
      +----- trace and verify <-----+
```

At every stage, maintain a usable fallback. The brute-force baseline is not wasted time: it validates the target behavior, reveals repeated work, and can become a test oracle. If optimization fails, a correct partial solution with honest analysis is better than an unverified sophisticated one.

Specification boundary: Java guarantees defined language and library behavior, but not the judge's memory limit, stack size, default input constraints, or expected method signature. Those belong to the interview contract and must be clarified or stated as assumptions.

Core terminology

- **Clarifying question:** resolves an ambiguity that can change correctness or design.
- **Constraint pressure:** reason the baseline fails at stated input scale.
- **Pattern signal:** structural property suggesting a technique, not proof by keyword.
- **Invariant:** statement connecting variables to the processed and unprocessed state.
- **Oracle:** trusted implementation or property used to verify another result.
- **Boundary matrix:** compact set of tests covering shape, value, and behavioral extremes.
- **Counterexample:** input disproving a proposed rule or invariant.
- **Follow-up:** changed constraint requiring adaptation or trade-off discussion.
- **Recovery loop:** reproduce, localize invariant violation, repair, and retrace.
- **Closeout:** final proof, complexity, mutation, edge cases, and alternative summary.

Detailed mechanics

A 45-minute operating rhythm

Exact interview lengths vary, but a time budget prevents uncontrolled drift:

Phase	Approximate time	Deliverable
Clarify and examples	4-6 minutes	Contract and expected cases
Baseline and optimization	5-8 minutes	Costed alternatives and selected approach
Invariant and pseudocode	3-5 minutes	Proof skeleton
Java implementation	15-20 minutes	Compilable solution
Dry run and tests	5-7 minutes	Corrected boundary behavior
Follow-ups and close	Remaining	Complexity and trade-offs

Do not obey the table mechanically. A familiar problem can move faster; a modeling-heavy graph problem deserves more clarification. The important discipline is noticing when ten silent minutes have passed without a verified artifact.

Step 1: clarify the contract

Ask questions that can change the algorithm:

- Can input be null or empty?
- What are n and value ranges?
- Are values sorted, unique, positive, or immutable?
- Do duplicates represent separate items?
- May the method mutate or reorder input?
- Which output is required when several are valid?
- Is there always a solution? What represents absence?
- Are indexes, values, paths, counts, or only an optimum required?
- What does "character," interval overlap, or distance mean?

Avoid spending time on irrelevant possibilities. If the interviewer says to choose reasonable assumptions, state them aloud and continue: "I will treat input as non-null, allow an empty array, and return -1 when infeasible."

Restate the contract in one sentence. This creates an early synchronization point.

Step 2: construct examples

Use one normal example, one smallest case, and one adversarial case tied to a likely mistake. For a window, include a case that shrinks multiple times. For a BST, include a deep ancestor violation. For Dijkstra, include a stale priority-queue entry. For duplicates, include equal values on both sides of a boundary.

Calculate expected output manually before coding. If that is difficult, the contract is not yet understood. Do not rely exclusively on the sample supplied by the interviewer; samples often avoid exactly the boundary that breaks a memorized template.

Step 3: establish and pressure-test a baseline

Describe the direct correct approach in enough detail to cost it. For example: enumerate every subarray, calculate each sum incrementally, and track the best, $O(n^2)$ time and $O(1)$ space. Then compare with constraints. At $n = 200$, it may be fine. At $n = 200,000$, it is not.

Name eliminated work:

- repeated membership test -> hash set;
- repeated range aggregate -> prefix sum;
- repeated valid-range rescan -> sliding window;
- repeated minimum selection -> heap;
- repeated subproblem -> memoization/DP;
- exhaustive ordered boundary -> binary search;
- repeated connectivity traversal -> DSU.

This explanation is stronger than saying "I know this pattern." It shows why the representation pays for itself.

Step 4: select a pattern and state its preconditions

Use a decision map, then check its proof obligation:

Need	Candidate	Must be true
Complement/frequency/history	Hashing	Equality and memory acceptable
Eliminate ordered candidates	Two pointers	Pointer movement is monotone and safe
Maintain contiguous valid region	Window	State updates and validity permit shrinking
Ordered boundary/answer	Binary search	Predicate is monotone
Nearest in edge count	BFS	Edges have equal effective weight
Next greater/smaller	Monotonic stack	Popped candidate is permanently dominated
Best k/frontier minimum	Heap	Only priority head is required
Repeated optimal states	DP	State is sufficient; transitions complete

If a precondition fails, do not force the pattern. Negative numbers can invalidate a sum window. Negative edges invalidate Dijkstra. A non-monotone feasibility test invalidates binary search.

Step 5: articulate the invariant before code

An interview invariant should be operational, not philosophical:

- "At loop start, indexes below `low` are infeasible and indexes at or above `high` are feasible."
- "The map counts prefix sums strictly before the current prefix."
- "The deque contains unexpired indexes in increasing index and decreasing value order."
- "`dp[a]` is the minimum number of coins for exact amount `a`, or impossible."

Also state progress: right advances, candidate interval shrinks, one node moves from suffix to prefix, or every recursive call reduces remaining work.

Write short pseudocode when it helps validate phase order. For mutation-heavy logic, draw state. For recursion, write the return contract and base case before the method body.

Step 6: implement interview-grade Java

Prefer Java that makes the proof visible:

- semantic names such as `left`, `rightExclusive`, `indegree`, and `prefixFrequency`;
- half-open intervals where they simplify boundaries;
- `long` before potentially overflowing arithmetic;
- `ArrayDeque` for stacks and queues;
- `PriorityQueue` with `Integer.compare`, `Long.compare`, or comparator factories;
- records for small immutable state pairs;
- arrays for dense integer domains and maps for sparse domains;
- helper methods where they isolate a predicate or traversal contract.

Do not build a production framework during a 40-minute exercise. Avoid streams when a stateful loop is easier to trace. Avoid clever one-liners, raw types, mutable static fields, and subtraction comparators. A local nested record is often clearer than parallel arrays, but check what the interview environment supports.

Compilation is a separate correctness layer. Scan imports, generic types, return paths, method names, and static context. If no IDE is available, mentally compile one declaration at a time.

Step 7: dry run by invariant

Do not narrate only values. At each iteration, verify the invariant and boundary updates. Use a small table with the variables that prove correctness. For recursive code, draw the first few frames and show state restoration. For a heap or deque, show contents in logical order, not internal library array order.

Select a case that activates the difficult branch. Testing a sliding window that never shrinks or a graph without a cycle proves little.

Step 8: use a boundary matrix

Cover categories rather than random anecdotes:

Dimension	Cases
Size	empty, one, two, typical, maximum shape
Values	zero, negative, duplicates, min/max numeric
Position	answer at start, middle, end, absent
Structure	sorted/reversed, skewed tree, disconnected graph, cycle
Behavior	impossible, multiple valid answers, all qualify, none qualify

Trace two or three high-yield cases aloud. Mention additional cases you would automate. If time permits, compare optimized output against the baseline on small generated inputs; this property-based oracle catches many pointer and DP defects.

Step 9: recover from a bug

Do not patch randomly. Use this loop:

- 1 Freeze a minimal failing input.
- 2 Identify the first step where actual state violates the invariant.
- 3 Classify the cause: initialization, update order, boundary, stale state, overflow, or contract mismatch.
- 4 Make the smallest coherent correction.
- 5 Restart the trace from initialization.
- 6 Recheck complexity and neighboring boundary cases.

Say what you found. "I am expiring the deque after reading its front, so an old maximum leaks into the answer. I will expire before reporting." This demonstrates control rather than panic.

Step 10: close and handle follow-ups

End with:

- time in named dimensions and any expected/amortized qualifier;
- auxiliary, recursion-stack, and output space;
- whether input is mutated;
- the critical precondition and invariant;
- one alternative and when it is preferable; and
- production constraints if asked.

When a follow-up changes one constraint, identify which proof breaks before changing code. If input becomes streaming, random access disappears. If memory becomes $O(1)$, hashing may no longer fit. If negatives are allowed, window monotonicity can fail. This approach prevents cargo-cult adaptation.

TRACE IT | Worked Java example

Problem: each positive workload takes `ceil(work / rate)` hours at integer processing rate. Workloads are handled sequentially, and each nonempty workload consumes at least one hour. Return the minimum rate that completes all work within `hourLimit`, or -1 when impossible.

JAVA (continued 1/2)

```
public final class MinimumProcessingRate {
    public static int minimumRate(int[] workloads, int hourLimit) {
        if (workloads == null) throw new IllegalArgumentException("null workloads");
        if (workloads.length == 0) return 0;
        if (hourLimit < workloads.length) return -1;

        int high = 0;
        for (int workload : workloads) {
            if (workload <= 0) {
                throw new IllegalArgumentException("workloads must be positive");
            }
            high = Math.max(high, workload);
        }

        int low = 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (finishesWithin(workloads, hourLimit, mid)) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
    }
}
```

JAVA (continued 2/2)

```

    }
    return low;
}

private static boolean finishesWithin(
    int[] workloads, int hourLimit, int rate) {
    long hours = 0;
    for (int workload : workloads) {
        hours += (workload + (long) rate - 1) / rate;
        if (hours > hourLimit) return false;
    }
    return true;
}

public static void main(String[] args) {
    System.out.println(minimumRate(new int[] {3, 6, 7, 11}, 8)); // 4
    System.out.println(minimumRate(new int[] {5}, 1));           // 5
    System.out.println(minimumRate(new int[] {2, 3}, 1));        // -1
    System.out.println(minimumRate(new int[] {}, 0));             // 0
}
}

```

The feasibility predicate is monotone: if rate r finishes in time, every larger rate also finishes in time. The answer is therefore the first true rate in $[1, \text{maxWorkload}]$.

TRACE IT | Execution or memory walkthrough

For workloads $[3, 6, 7, 11]$ and limit 8, initialize candidate range $[1, 11]$ with both endpoints represented by `low` and `high`. Rate 11 is feasible because each workload takes one hour.

Step	low	high	mid	Hours needed	Update
1	1	11	6	6	high = 6
2	1	6	3	10	low = 4
3	4	6	5	8	high = 5
4	4	5	4	8	high = 4
End	4	4	-	-	Return 4

Invariant: every rate below `low` is infeasible, and `high` is feasible; the first feasible rate remains in $[\text{low}, \text{high}]$. A feasible `mid` can still be larger than minimum, so retain it by setting `high = mid`. An infeasible `mid` is excluded with `low = mid + 1`. The interval strictly shrinks.

The ceiling formula promotes before addition to avoid `int` overflow. Early exit in the predicate preserves correctness because hours only increase. The empty-input behavior and impossible case are explicit contract choices rather than accidental loop results.

Complexity and performance

Let n be workload count and M the maximum workload. Feasibility is $O(n)$ time and $O(1)$ space. Binary search performs $O(\log M)$ predicate calls, so total time is $O(n \log M)$ and auxiliary space is $O(1)$. The value-domain logarithm is distinct from $O(\log n)$.

A brute-force scan of rates is $O(nM)$ and is a useful conceptual oracle for small M . Sorting does not help because every feasibility check needs the aggregate work across all workloads. Precomputing cannot remove dependence on rate without a more specialized constraint set.

Interview choice	Benefit	Cost or risk
Baseline first	Correctness anchor and oracle	Uses a few minutes
Helper predicate	Isolates monotonic proof	Extra method boundary
<code>long</code> aggregate	Prevents realistic overflow	Must promote before arithmetic
Early predicate exit	Less work on infeasible rates	Does not change worst-case bound
Half-open/closed convention	Prevents boundary drift	Must remain consistent

HotSpot note: HotSpot may inline the feasibility helper and optimize the primitive loop. Such optimization changes constants, not the $O(n \log M)$ algorithm or the need for overflow-safe arithmetic.

WATCH OUT | Edge cases and common mistakes

- Coding before deciding null, empty, impossible, and mutation behavior.
- Asking many questions that cannot affect the algorithm while missing value ranges.
- Quoting a pattern without stating its preconditions.
- Giving an optimized idea without a correct baseline or proof.
- Using sample input as the only test.
- Mixing `int` and `long` so overflow occurs before widening.
- Writing ceiling division as floating point and introducing precision or conversion issues.
- Failing to establish that the binary-search predicate is monotone.
- Mixing first-true updates with any-match loop conditions.
- Narrating syntax instead of decisions and invariants.
- Going silent after finding a bug and applying unexplained patches.
- Claiming success without tracing the branch that mutates or shrinks state.
- Reporting $O(\log n)$ when the search dimension is a value M .
- Forgetting stack or output space.
- Overengineering interfaces and classes that do not help solve or verify the problem.
- Treating interviewer feedback as a command to patch rather than evidence to re-evaluate the model.

Production engineering notes

Interview code optimizes for a short, isolated demonstration. Production code needs input limits, observability, cancellation, API error types, test suites, documentation, and ownership. Do not paste a coding solution directly into a service boundary.

Numeric models deserve special attention. A workload might exceed `int`, processing might be parallel rather than sequential, or hour limits might use time zones and partial units. Clarify units and use domain types where appropriate. The worked predicate assumes deterministic integer work and no setup cost.

For large inputs, benchmark the actual representation. A map-heavy expected $O(n)$ solution may exceed memory. A recursive DFS may overflow a worker stack. A returned list may be the dominant allocation. State these constraints in an SDE-2 follow-up without derailing the initial interview solution.

Interview practice should be measurable. Track failure categories: contract, recognition, proof, coding, boundary, complexity, and communication. Re-solving ten problems without classifying mistakes is less useful than repairing one repeated invariant failure.

TRY IT | Interview questions and model answers

Should I start coding immediately if I recognize the problem?

No. Spend a short, bounded period confirming the contract, constraints, and expected output. Then state the approach and invariant. Recognition accelerates reasoning but does not replace proof or boundary checks.

How much should I talk while coding?

Explain decisions, invariant-changing updates, and trade-offs. Do not narrate punctuation. Brief silence while implementing a stated plan is fine; resynchronize when entering a tricky branch or after changing the plan.

What if I cannot find the optimal solution?

Present a correct baseline, analyze its bottleneck, and explore one optimization at a time. Ask for a constraint hint if appropriate. A verified suboptimal solution plus clear progress is stronger than speculative code.

What should I do when the interviewer finds a counterexample?

Restate the failing case, find the first invariant violation, explain which assumption failed, repair the model, and retrace. Do not defend the old approach or patch only the observed output.

How do I know binary search on the answer is valid?

Define a totally ordered candidate domain and a feasibility predicate. Prove all candidates on one side are false and all on the other side true. Then search explicitly for the first true or last false boundary.

What distinguishes an SDE-2 closeout?

It includes named dimensions, qualified time, auxiliary/stack/output space, mutation, critical invariant, edge cases, and a credible alternative. It can also discuss scale, overflow, concurrency, or API contracts when prompted.

TRY IT | Exercises

- 1 Run the ten-step playbook on two-sum, recording no more than one sentence per step.
- 2 Solve a variable-window problem and design a test that forces several left-pointer moves in one iteration.
- 3 Use an $O(n^2)$ oracle to property-test an $O(n)$ prefix-hash solution on small random arrays.
- 4 Conduct a 45-minute graph mock and record time spent in each phase.
- 5 Deliberately inject an off-by-one error into lower bound, then practice the recovery loop aloud.
- 6 Adapt `minimumRate` when workloads may be zero and when work can be split across workers; identify which assumptions change.
- 7 Take one recent failed problem and classify the root cause as contract, model, invariant, implementation, or testing.
- 8 Practice three follow-ups: forbid extra space, make input streaming, and require all valid outputs.

RECAP | Chapter summary

A reliable coding interview is a controlled engineering loop. Clarify the contract, compute examples, establish a costed baseline, name eliminated work, verify pattern preconditions, state an invariant and progress measure, implement readable Java, trace a difficult branch, test a boundary matrix, and close with complete complexity. When a defect appears, localize the first invariant violation rather than patching randomly. Follow-ups change constraints; identify which proof breaks before adapting code.

TRY IT | Revision checklist

- ☐ I can restate null, empty, duplicate, mutation, and absence behavior.
- ☐ I create a normal, smallest, and adversarial example before coding.
- ☐ I state a correct baseline and the repeated work being eliminated.
- ☐ I verify pattern preconditions rather than matching keywords.
- ☐ I articulate an operational invariant and progress measure.
- ☐ I use semantic Java names, safe arithmetic, and appropriate collections.
- ☐ I trace the difficult branch and use a boundary matrix.
- ☐ I recover by finding the first invariant violation.
- ☐ I report time, auxiliary space, stack, output, and mutation.
- ☐ I identify which assumption a follow-up changes before editing code.

SDE-2 CORE CHAPTER

Chapter 16 - Forty Java Interview Traps

These are behavior questions, not trivia contests. For each one: predict before running, identify the language or API rule, repair the code, and state the boundary an interviewer could change.

1. == versus equals

JAVA

```
String left = new String("java");
String right = new String("java");
System.out.println(left == right);
System.out.println(left.equals(right));
```

Predicted/actual output: `false`, then `true`. `==` compares reference identity; `equals` compares string content. Correct value test: `Objects.equals(left, right)` when either value may be null. Follow-up: what contract must a custom key's `equals` satisfy?

2. String-pool assumptions

JAVA

```
String first = "sde2";
String second = "sde" + "2";
System.out.println(first == second);
```

Actual output is `true` because this constant expression is folded and the literal may share the pooled instance. Do not infer that all equal strings have one object. Correct comparison: `first.equals(second)`. Follow-up: what changes if part of the concatenation is a variable?

3. new String(...)

JAVA

```
String pooled = "java";
String explicit = new String("java");
System.out.println(pooled == explicit);
```

Actual output: `false`. The constructor creates a distinct `String` object even though it is content-equal to the literal. Correct value test: `pooled.equals(explicit)`. Follow-up: what does `explicit.intern()` return?

4. Java pass-by-value

JAVA

```
static void replace(StringBuilder value) {  
    value = new StringBuilder("new");  
}  
StringBuilder caller = new StringBuilder("old");  
replace(caller);  
System.out.println(caller);
```

Actual output: `old`. The parameter receives a copy of the reference value; reassigning that copy does not reassign `caller`. Mutation such as `value.append("!")` would affect the shared object. Follow-up: draw both variables before and after reassignment.

5. Primitive overflow

JAVA

```
int maximum = Integer.MAX_VALUE;  
System.out.println(maximum + 1);
```

Actual output: `-2147483648`. `int` arithmetic wraps to the defined 32-bit two's-complement result; it does not widen automatically or throw. Correct when overflow is invalid: `Math.addExact(maximum, 1)`. Follow-up: when is modular wraparound intentional?

6. Overflow before assignment to long

JAVA

```
long wrong = 100_000 * 100_000;  
long correct = 100_000L * 100_000;  
System.out.println(wrong + " " + correct);
```

Actual output: `1410065408 100000000000`. Both operands of the first product are `int`, so overflow happens before assignment. Widen an operand before the operation. Follow-up: why is `(long) (a * b)` still too late?

7. Integer division

JAVA

```
System.out.println(5 / 2);  
System.out.println(5 / 2.0);
```

Actual output: `2`, then `2.5`. Integer division truncates toward zero. Correct an average by widening before division: `(double) sum / count`. Follow-up: what is `-5 / 2`?

8. Floating-point equality

JAVA

```
System.out.println(0.1 + 0.2 == 0.3);
```

Actual output: usually **false** by specified binary floating-point evaluation of these literals. Compare within a domain-defined tolerance, or use exact decimal/integer units when the domain requires them. Follow-up: why is one universal epsilon incorrect?

9. Narrowing conversion

JAVA

```
double value = 10.8;
int whole = (int) value;
System.out.println(whole);
```

Actual output: **10**. The cast truncates toward zero; it does not round. Correct rounding may use **Math.round** with explicit range handling. Follow-up: what happens for a value outside the **int** range?

10. Prefix versus postfix increment

JAVA

```
int value = 5;
int first = value++;
int second = ++value;
System.out.println(value + " " + first + " " + second);
```

Actual output: **7 5 7**. Postfix yields the old value; prefix increments before yielding. Correct interview style separates increments from larger expressions. Follow-up: predict **numbers[index++] = index** and explain evaluation order.

11. Short-circuit evaluation

JAVA

```
String text = null;
System.out.println(text != null && !text.isEmpty());
```

Actual output: **false**; the second operand is not evaluated. Replacing **&&** with boolean **&** evaluates both sides and throws. Correct guard: cheapest/safest prerequisite first. Follow-up: how can short-circuiting avoid an out-of-bounds read?

12. Null unboxing

JAVA

```
Integer count = null;  
// int value = count;
```

If reached, the assignment throws `NullPointerException` during unboxing. Correct by rejecting null, defining a default, or keeping absence explicit. Follow-up: why can `map.get(key) == 0` throw?

13. Integer wrapper caching

JAVA

```
Integer a = 127, b = 127;  
Integer c = 128, d = 128;  
System.out.println(a == b);  
System.out.println(c == d);
```

The first is `true` for the required cache; the second identity result must not be used as a value contract and is commonly `false`. Correct: `a.equals(b)` and `c.equals(d)`. Follow-up: why may a larger implementation cache not improve correctness?

14. Math.abs(Integer.MIN_VALUE)

JAVA

```
System.out.println(Math.abs(Integer.MIN_VALUE));
```

Actual output remains `-2147483648`: the positive magnitude is not representable as `int`. Correct: widen first with `Math.abs((long) value)` or use `Math.absExact` when overflow must throw. Follow-up: can `Math.abs` be used safely as a hash bucket index?

15. Math.abs(Long.MIN_VALUE)

JAVA

```
System.out.println(Math.abs(Long.MIN_VALUE));
```

Actual output remains negative for the same asymmetry at 64 bits. `Math.absExact(long)` throws on this input; `BigInteger` handles a larger exact magnitude. Follow-up: design a non-negative modulo-based index without `abs`.

16. Array aliasing

JAVA

```
int[] first = {1, 2};
int[] alias = first;
alias[0] = 9;
System.out.println(first[0]);
```

Actual output: 9. Assignment copies the array reference, not its elements. Correct independent storage: `int[] copy = first.clone()`. Follow-up: does cloning a two-dimensional array copy nested rows?

17. Shallow copying

JAVA

```
int[][] original = {{1}, {2}};
int[][] copy = original.clone();
copy[0][0] = 9;
System.out.println(original[0][0]);
```

Actual output: 9. Only the outer array was copied; row references are shared. Correct deep copy: clone each non-null row. Follow-up: what contract should define behavior for null rows?

18. Arrays.asList fixed-size behavior

JAVA

```
List<String> values = Arrays.asList("a", "b");
values.set(0, "x");
values.add("c");
```

`set` succeeds; `add` throws `UnsupportedOperationException`. The list is fixed-size and backed by an array. Correct resizable copy: `new ArrayList<>(Arrays.asList("a", "b"))`. Follow-up: are changes to the original array visible through the list?

19. Primitive arrays passed to Arrays.asList

JAVA

```
int[] numbers = {1, 2, 3};
List<int> list = Arrays.asList(numbers);
System.out.println(list.size());
```

Actual output: 1. `int[]` is one reference element; generic varargs do not box its contents. Correct: loop into `List<Integer>` or use `Arrays.stream(numbers).boxed().toList()`. Follow-up: is the stream result mutable?

20. String immutability

JAVA

```
String text = "java";
text.toUpperCase();
System.out.println(text);
```

Actual output: java. String methods return results; the original object does not change. Correct: `text = text.toUpperCase(Locale.ROOT)`. Follow-up: why can immutable strings still be referenced by mutable variables?

21. Repeated string concatenation

JAVA

```
String result = "";
for (int i = 0; i < n; i++) result += i;
```

This may repeatedly copy a growing immutable string and become quadratic in produced characters. Correct repeated construction: `StringBuilder builder = new StringBuilder();`
`builder.append(i);`. Follow-up: when can one expression's `+` remain readable and efficient?

22. char arithmetic

JAVA

```
char digit = '7';
System.out.println(digit - '0');
System.out.println(digit + 1);
```

Actual output: 7, then 56; binary numeric promotion produces `int`. Correct next character: `(char)(digit + 1)`. Validate ASCII digits before `digit - '0'`; `Character.digit(digit, 10)` handles broader digit definitions. Follow-up: what type is `'a' + 'b'`?

23. Unicode limitations of char

JAVA

```
String symbol = "\uD83D\uDE00";
System.out.println(symbol.length());
```

Actual output: 2. A Java `char` is a UTF-16 code unit; this supplementary code point uses a surrogate pair. Correct code-point count: `symbol.codePointCount(0, symbol.length())`. Follow-up: why is even a code point not always one user-perceived character?

24. Switch fall-through

JAVA

```
int value = 1;
switch (value) {
    case 1: System.out.print("one ");
    case 2: System.out.print("two");
}
```

Actual output: **one two**. Traditional colon cases fall through without **break**, **return**, or **throw**. Correct with breaks or a version-labeled switch expression. Follow-up: when is deliberate fall-through acceptable?

25. Missing default constructor

JAVA

```
class User { User(String name) {} }
// User user = new User();
```

The commented construction would not compile. Declaring any constructor prevents the compiler from supplying a no-argument constructor. Correct by calling the declared constructor or declaring an explicit no-argument constructor. Follow-up: what if the superclass lacks an accessible no-argument constructor?

26. Constructor chaining rules

JAVA

```
class Size {
    Size() {
        // System.out.println("start");
        // this(1); // illegal after another statement
    }
    Size(int value) {}
}
```

this(...) or **super(...)** must be the constructor's first statement. Correct: put **this(1);** first and move common work to the target constructor. Follow-up: can one constructor invoke both explicitly?

27. Static versus instance access

JAVA

```
class Counter {
    static int total;
    int local;
    static void reset() { total = 0; }
}
```


A static method has no receiver and cannot directly read `local`. Correct: pass a `Counter`, make the method an instance method, or operate only on class state. Follow-up: why is `instance.reset()` legal but misleading?

28. Overriding versus overloading

JAVA

```
class Base { String label(Object value) { return "base"; } }
class Child extends Base { String label(String value) { return "child"; } }
Base item = new Child();
System.out.println(item.label("x"));
```

Actual output: `base.label(String)` overloads rather than overrides `label(Object)`; compile-time selection uses `Base`'s visible signature. Correct override uses the identical parameter types and `@Override`. Follow-up: can return type alone overload a method?

29. Covariant return types

JAVA

```
class Parent { Number value() { return 1; } }
class Child extends Parent { @Override Integer value() { return 2; } }
```

This compiles: an overriding reference return type may be a subtype of the original return type. Primitive return types are not covariant. Follow-up: what return type is visible at a call site through a `Parent` reference?

30. Field hiding versus method overriding

JAVA

```
class Parent { int value = 1; int value() { return 1; } }
class Child extends Parent { int value = 2; @Override int value() { return 2; } }
Parent item = new Child();
System.out.println(item.value + " " + item.value());
```

Actual output: `1 2`. Fields use the declared reference type; instance methods dispatch to the runtime object. Correct design avoids same-name instance fields across a hierarchy. Follow-up: how do static methods behave?

31. Unsafe downcasting

JAVA

```
Object value = "java";  
// Integer number = (Integer) value;
```

If reached, the cast throws `ClassCastException`. Correct: keep a stronger type or guard with `instanceof`. Follow-up: why can a cast compile yet fail at runtime?

32. Exception catch ordering

JAVA

```
try {  
    Integer.parseInt("x");  
} catch (RuntimeException exception) {  
    System.out.println("runtime");  
} catch (NumberFormatException exception) { } // unreachable  
}
```

A catch for a subtype after its supertype is a compile-time error because the earlier catch already handles it. Correct: most specific catches first. Follow-up: when is one broad boundary catch appropriate?

33. finally behavior

JAVA

```
static int result() {  
    try { return 1; }  
    finally { return 2; }  
}
```

Actual result: 2; the `finally` return suppresses the pending return and is dangerous. Correct: never return from `finally`; reserve it for cleanup that does not replace the primary outcome. Also, `finally` is not guaranteed after forced process termination or fatal host failure. Follow-up: how does try-with-resources preserve suppressed close failures?

34. Mutable keys in HashMap

JAVA

```
List<Integer> key = new ArrayList<>(List.of(1));  
Map<List<Integer>, String> map = new HashMap<>();  
map.put(key, "value");  
key.add(2);  
System.out.println(map.get(key));
```

The lookup is commonly `null` because the key's hash code changed while the entry stayed in its old bucket. Correct: use an immutable key whose equality fields never change. Follow-up: why may iterating still reveal the entry?

35. Comparator subtraction overflow

JAVA

```
Comparator<Integer> broken = (left, right) -> left - right;  
System.out.println(broken.compare(Integer.MAX_VALUE, -1));
```

Subtraction overflows and can report the wrong sign. Correct: `Integer.compare(left, right)`. Follow-up: how do you add a deterministic tie-breaker?

36. PriorityQueue iteration is not sorted

JAVA

```
PriorityQueue<Integer> queue = new PriorityQueue<>(List.of(4, 1, 3, 2));  
for (int value : queue) System.out.print(value + " ");
```

Only `peek/poll` guarantee the head according to priority; iterator order is unspecified. Correct sorted consumption repeatedly polls a copy if the original must be preserved. Follow-up: what is the cost and mutation trade-off?

37. Modifying during enhanced-for

JAVA

```
List<Integer> values = new ArrayList<>(List.of(1, 2, 3));  
for (int value : values) {  
    if (value == 2) values.remove(Integer.valueOf(value));  
}
```

This can throw `ConcurrentModificationException`; fail-fast behavior is a bug detector, not a synchronization guarantee. Correct: use the iterator's `remove`, `removeIf`, or collect changes separately. Follow-up: why does the exact moment of failure not form an API guarantee?

38. Null handling

JAVA

```
String value = null;  
// System.out.println(value.equals("x"));
```

The call would throw. Correct null-safe choices depend on contract: reject with `requireNonNull`, compare as `"x".equals(value)`, or use `Objects.equals`. Follow-up: why can silently treating null as empty lose information?

39. Off-by-one array errors

JAVA

```
int[] values = {2, 4, 6};
for (int index = 0; index <= values.length; index++) {
    System.out.println(values[index]);
}
```

The fourth access throws `ArrayIndexOutOfBoundsException`; valid indexes are 0 through `length - 1`. Correct half-open loop: `index < values.length`. Follow-up: state its loop invariant.

40. Incorrect loop termination

JAVA

```
int value = 10;
while (value != 0) {
    value -= 3;
}
```

This never reaches zero and continues through negative values until eventual integer wraparound. Correct the contract and progress rule, for example `while (value > 0) value -= 3;`. Follow-up: how would you prove termination for arbitrary step sizes?

Rapid trap checklist

Before submitting, inspect equality, null, arithmetic type, overflow order, division, casts, aliasing, copy depth, loop bounds, comparator sign, collection mutation, constructor availability, dispatch kind, exception order, and resource ownership.

Cross-book boundary

Numeric traps continue in Number Systems and Bit Manipulation. Loop termination continues in Loop Mastery. Aliasing and copying continue in Arrays. Hash keys continue in Hashing. Priority ordering continues in Heaps. Deep exception and JVM failure behavior continue in Advanced Java and JVM.

RECAP | Chapter summary

Each trap has the same repair method: state the type and object contract, predict evaluation order, identify shared state, label the failure stage, and test a boundary that distinguishes the wrong mental model from the correct one.

SDE-2 CORE CHAPTER

Chapter 17 - Practice Lab D - Traps, Refactoring, and Readiness

Start here: this lab is a retrieval test across the full volume. Do not run output questions until a prediction and governing rule are written. For debugging items, preserve the intended contract, make the smallest correct change, and add a test that fails before the repair. For coding tasks, state time and auxiliary space after the code works.

Use three sessions: K76-K100 plus O58-O75, D58-D75 plus C58-C66, then C67-C75 plus F39-F50 and the assessments. Finish with the readiness rubric at the end of this chapter. If a miss comes from an earlier concept, return to that chapter instead of memorizing this lab's answer.

Knowledge check

- **K76 [Foundation]** Distinguish reference identity from semantic equality.
- **K77 [Interview Core]** State what string interning does and does not guarantee.
- **K78 [Interview Core]** Why does explicit String construction affect identity but not content?
- **K79 [Interview Core]** Separate mutation through a copied reference from parameter reassignment.
- **K80 [Interview Core]** Why can overflow happen before a wider assignment?
- **K81 [Interview Core]** When does floating comparison require a domain tolerance?
- **K82 [Foundation]** Why does narrowing truncate rather than round?
- **K83 [Interview Core]** What is special about primitive minimum magnitudes?
- **K84 [Interview Core]** What is copied by an array clone?
- **K85 [Interview Core]** Why is `Arrays.asList` different for object and primitive arrays?
- **K86 [Foundation]** Why must String method results usually be captured?
- **K87 [Interview Core]** When is `Character.digit` safer than subtracting `'0'`?
- **K88 [Interview Core]** How does fall-through differ from multi-label switch cases?
- **K89 [Foundation]** Why can a constructor not declare a return type?
- **K90 [Interview Core]** What is the first-statement rule for constructor chaining?
- **K91 [Interview Core]** Which members are dynamically dispatched?
- **K92 [Interview Core]** Why is downcasting evidence of a possible contract gap?
- **K93 [Interview Core]** Why does catch ordering affect compilability?
- **K94 [SDE-2 Follow-up]** Name conditions under which `finally` may not complete normally.
- **K95 [Interview Core]** Why must hash keys remain equality-stable?
- **K96 [Interview Core]** Why must comparator logic avoid subtraction?
- **K97 [Interview Core]** What does fail-fast collection iteration guarantee and not guarantee?
- **K98 [Foundation]** State the half-open array-index interval.
- **K99 [Interview Core]** What progress measure proves a loop terminates?
- **K100 [SDE-2 Follow-up]** List the final review order for contract, type, state, control flow, API, and complexity.

Predict the output

- **O58 [Interview Core]** Predict two equal new Strings under `==` and `equals`.
- **O59 [Interview Core]** Predict literal-plus-constant literal identity.
- **O60 [Interview Core]** Predict caller state after object mutation and parameter reassignment in one method.
- **O61 [Interview Core]** Predict `100_000 * 100_000` assigned to long.
- **O62 [Foundation]** Predict `-5 / 2` and `-5 % 2`.
- **O63 [Interview Core]** Predict `0.1 + 0.2 == 0.3`.
- **O64 [Foundation]** Predict `(int) -10.8`.
- **O65 [Interview Core]** Predict `Math.abs(Integer.MIN_VALUE)`.
- **O66 [Interview Core]** Predict a nested array mutation through a shallow copy.
- **O67 [Interview Core]** Predict add versus set on `Arrays.asList`.
- **O68 [Foundation]** Predict ignored `String.replace` result.
- **O69 [Interview Core]** Predict code-point count versus String length for a surrogate pair.
- **O70 [Foundation]** Predict traditional switch without breaks.
- **O71 [Interview Core]** Predict field and method access through parent reference/child object.
- **O72 [Interview Core]** Predict which overload runs for a base-typed argument.
- **O73 [Interview Core]** Predict a return in try replaced by return in finally.
- **O74 [Interview Core]** Predict map lookup after mutable-key mutation.
- **O75 [SDE-2 Follow-up]** Predict whether heap iteration order is guaranteed after seeing one sample run.

Debug the code

- **D58 [Foundation]** Replace object `==` with the correct null-safe value comparison.
- **D59 [Interview Core]** Repair code whose correctness depends on string interning.
- **D60 [Interview Core]** Repair a swap method that tries to swap caller references.
- **D61 [Interview Core]** Repair multiplication widened only after overflow.
- **D62 [Interview Core]** Repair floating equality for a currency-domain value.
- **D63 [Foundation]** Repair narrowing code that intended rounding.
- **D64 [Interview Core]** Repair a hash index derived with `Math.abs(hash)`.
- **D65 [Interview Core]** Repair a matrix deep-copy routine.
- **D66 [Interview Core]** Repair `Arrays.asList(primitiveArray)` under a `List<Integer>` requirement.
- **D67 [Foundation]** Capture a String transformation result.
- **D68 [Interview Core]** Repair character logic under a Unicode code-point contract.
- **D69 [Foundation]** Repair accidental switch fall-through.
- **D70 [Foundation]** Add a missing no-argument constructor deliberately.
- **D71 [Interview Core]** Repair a constructor that calls `this` after validation code.
- **D72 [Interview Core]** Repair code expecting static method overriding.
- **D73 [Interview Core]** Repair an unsafe cast by changing the base contract.
- **D74 [Interview Core]** Repair a catch hierarchy and preserve specific handling.
- **D75 [SDE-2 Follow-up]** Repair a loop whose termination depends on eventually hitting exactly zero.

Small coding tasks

- **C58 [Foundation]** Build a null-safe string equality helper.
- **C59 [Interview Core]** Demonstrate pool, constructor, and `intern` behavior without using identity for business logic.
- **C60 [Interview Core]** Build a pass-by-value diagram generator for one mutable object scenario.
- **C61 [Interview Core]** Implement safe average and safe product helpers.
- **C62 [Interview Core]** Implement domain-tolerant floating comparison with documented scale.
- **C63 [Interview Core]** Compute a non-negative bucket index without `Math.abs` overflow.
- **C64 [Interview Core]** Deep-copy a jagged object array under a stated element-copy contract.
- **C65 [Interview Core]** Convert a primitive array to a boxed resizable list.
- **C66 [Foundation]** Build text with `StringBuilder` using `append`, `insert`, `delete`, `setCharAt`, and `reverse`.
- **C67 [Interview Core]** Count Unicode code points without splitting surrogate pairs.
- **C68 [Foundation]** Rewrite a fall-through switch as an arrow switch expression.
- **C69 [Interview Core]** Demonstrate constructor chaining and initialization order.
- **C70 [Interview Core]** Demonstrate overload, override, field hiding, and static hiding separately.
- **C71 [Interview Core]** Implement safe polymorphic behavior without downcasting.
- **C72 [Interview Core]** Preserve a primary exception and inspect suppressed close failures.
- **C73 [Interview Core]** Create an immutable hash-map key and prove stable lookup.
- **C74 [Interview Core]** Poll a copy of a `PriorityQueue` into sorted order.
- **C75 [SDE-2 Follow-up]** Refactor a weak array-sum solution into interview-quality Java with contract, type choice, tests, and complexity.

Interview follow-ups

- **F39 [Interview Core]** Which equality operation belongs to primitives, strings, arrays, and custom keys?
- **F40 [SDE-2 Follow-up]** How can a seemingly harmless refactor change compile-time overload selection?
- **F41 [Interview Core]** Which boundary tests expose overflow-before-widening?
- **F42 [SDE-2 Follow-up]** How do you choose integer units versus decimal arithmetic versus tolerance?
- **F43 [Interview Core]** When is a shallow copy the intended contract?
- **F44 [SDE-2 Follow-up]** How would you document Unicode assumptions in a string algorithm?
- **F45 [Interview Core]** When is traditional switch fall-through useful enough to keep?
- **F46 [SDE-2 Follow-up]** Why are fields not polymorphic even when methods are?
- **F47 [Interview Core]** What should a boundary catch log or translate without hiding?
- **F48 [SDE-2 Follow-up]** Explain how mutable-key failure follows from the equality/hash contract.
- **F49 [Interview Core]** What ordering does PriorityQueue promise at each operation?
- **F50 [SDE-2 Follow-up]** Give a five-minute verbal review of an interview solution before coding.

Cumulative assessment 5

A05: In 60 minutes, diagnose a mixed program containing eight traps from this chapter, repair it, and explain why every corrected line follows Java's language or library contract. Add tests for null, empty, singleton, extremes, duplicates, and malformed input.

Final readiness assessment

Complete without notes:

- 1 Explain source-to-execution, primitive/reference values, promotion, pass-by-value, equality, construction, dispatch, generics, collection choice, exception ownership, and I/O boundaries.
- 2 Implement one array/string task and one map/queue task in Java 21.
- 3 Predict ten outputs chosen randomly from O01-O75 with at least eight correct.
- 4 Repair ten defects chosen randomly from D01-D75 with the governing rule stated.
- 5 Complete five coding tasks chosen randomly from C01-C75 and explain time and auxiliary space.
- 6 Answer ten follow-ups chosen randomly from F01-F50 without relying on folklore.

Readiness standard: at least 80 percent on prediction/debugging, compiling solutions for all five coding tasks, no incorrect pass-by-reference or equality explanation, and explicit boundary tests. If numeric overflow is weak, return to Number Systems. If loop invariants are weak, continue with Loop Mastery after Volume 02. Otherwise proceed to Bit Manipulation, Arrays, Strings, and Hashing.

SDE-2 CORE CHAPTER

Chapter 18 - Solution Studio A - Language and Execution

Use these after attempting Practice Lab A. Each answer names the rule, because an unexplained answer is not interview-ready.

How to review: compare your rule before comparing your final value. Mark a response correct only if both match. For each miss, retype the smallest compiling example and change one operand, boundary, or control-flow condition to prove that the rule transfers. The grouped coding guidance is intentionally not a paste-ready answer: implement it, compile it, and defend the contract yourself.

Knowledge answers K01-K25

- **K01:** Source is human-authored `.java`; `javac` emits class-file bytecode; a compatible JVM executes the class-file operations on the host.
- **K02:** JDK is the development kit; a runtime/JRE is JVM plus runtime libraries/support; JVM executes class files. Modern packaging may not expose a separate JRE product.
- **K03:** `public` accessible, `static` no receiver required, `void` no result, `main` launcher entry name, `String[]` text argument array, `args` parameter name.
- **K04:** A public top-level class must normally use a same-named source file so the compiler/source lookup contract is satisfied.
- **K05:** Declaration introduces a variable; initialization gives its first value; assignment stores a value; reassignment replaces a prior value when permitted.
- **K06:** Java's definite-assignment analysis rejects a local read unless every reachable path assigns it; locals do not receive field defaults.
- **K07:** Integral: byte, short, int, long, char. Floating: float, double. boolean is logical.
- **K08:** The language fixes primitive value sets and numeric behavior, while field padding, object headers, alignment, and optimized representation are JVM/layout concerns.
- **K09:** A primitive variable stores its primitive value; a reference variable stores a reference value that may designate an object or be null.
- **K10:** Every argument expression produces a value copied into a parameter. For objects that value is a reference value, so mutation can be shared but parameter reassignment cannot escape.
- **K11:** `char` is an unsigned 16-bit UTF-16 code unit from 0 through 65535, not necessarily a full code point or glyph.
- **K12:** Expression type is decided before assignment; two int operands produce an int product, which may overflow before conversion to long.
- **K13:** Widening moves to a type whose range/representation is accepted implicitly; narrowing needs a cast and can discard range or precision.
- **K14:** Each byte promotes to int, so byte addition produces int.
- **K15:** Integer division truncates toward zero; remainder satisfies `a == (a / b) * b + a % b` and follows the dividend's sign when nonzero.
- **K16:** `&&` short-circuits after false; boolean `&` evaluates both operands.
- **K17:** `x += y` behaves like `x = (typeOfX) (x + y)` with one evaluation of the left side, so a narrowing conversion may be implicit.
- **K18:** Prefix updates then yields the new value; postfix yields the old value then updates.
- **K19:** A guard clause is clearer when invalid/special cases can exit early and keep the main path shallow.

- **K20:** Colon cases can fall through; arrow switch expression arms do not fall through and produce a value or complete an action.
- **K21:** `for` for explicit counter/update, `while` for condition-driven repetition, `do-while` for at least one pass, enhanced-for for value traversal without index mutation.
- **K22:** Unlabeled break exits only the nearest loop or switch; use a label or helper return deliberately for outer exit.
- **K23:** Name plus parameter types distinguish overloads. Return type is not part of overload selection at a call expression.
- **K24:** Base case stops recursion; recursive case reduces the problem toward it. Without progress, frames grow until `StackOverflowError`.
- **K25:** Class files are portable across compatible JVMs, but native libraries, OS services, paths, charsets, resources, timing, and environment remain host-dependent.

Output answers O01-O19

- **O01:** `2`; both operands are `int`, so the fractional part is discarded.
- **O02:** `-1`; remainder follows the dividend and preserves the division identity.
- **O03:** `-126`; 130 modulo 256 maps to that signed byte value.
- **O04:** `-2147483648`; overflow occurs as `int` before widening to `long`.
- **O05:** `3 4`; postfix yields 3, then `x` becomes 4.
- **O06:** `4 4`; prefix updates before yielding.
- **O07:** `15`; compound assignment includes the required narrowing cast.
- **O08:** `7`; digit code units are consecutive.
- **O09:** `66`; `char` participates in `int` arithmetic.
- **O10:** `false`; `fail()` is skipped by short-circuiting.
- **O11:** `0`; true on the left skips `++x`.
- **O12:** `2 1 0`; the condition includes zero and the decrement then reaches -1.
- **O13:** `1`; do-while runs once before testing.
- **O14:** `AB`; case 1 falls into case 2 without break.
- **O15:** `5`; arguments 2 and 3 are added and returned.
- **O16:** Caller primitive unchanged; only the copied parameter value increments.
- **O17:** Field changed; both reference values designate the same mutable object.
- **O18:** Caller reference unchanged; only the local parameter is reassigned.
- **O19:** Value `1.0`, type `double`; numeric conditional typing promotes the `int` arm.

Debug answers D01-D19

- **D01:** Add `;`; Java statements require termination.
- **D02:** Initialize `total` on every path before read, usually `int total = 0;` for an accumulator.
- **D03:** Rename the file `Main.java` or make the top-level class non-public/rename it to `Program`.
- **D04:** `long area = (long) width * height;;` widen before multiplication.
- **D05:** Use `value++`, `value += 1`, or an explicit checked/narrowing conversion; plain addition produces `int`.
- **D06:** Use `(double) sum / count` after checking `count != 0`.
- **D07:** Use `&&`; it skips the dereference when text is null.
- **D08:** Use `==`; assignment is not a boolean expression for `int`.
- **D09:** Use `i < array.length`; array indexes form a half-open interval.
- **D10:** Start at `array.length - 1` and continue while `i >= 0`.
- **D11:** Change the update or condition so a well-founded measure approaches the exit.
- **D12:** Add `break`/return/throw or use arrow switch syntax if the target Java version supports it.
- **D13:** Use an indexed loop or maintain an explicit, correctly updated counter.
- **D14:** Change parameter lists or method names; return type alone cannot overload.
- **D15:** Cast null to the intended parameter type or redesign ambiguous overloads.
- **D16:** Return on every reachable path or declare `void` if no value is part of the contract.
- **D17:** Add a reachable base case and make every recursive call reduce toward it.
- **D18:** `long magnitude = Math.abs((long) value);;` widening before `abs` makes 2147483648 representable.
- **D19:** Validate each failure with an early return/throw, then leave one shallow success path; verify original exception/result behavior with tests.

Coding guidance C01-C19

- **C01-C04:** Define null/empty/negative contracts first, then use one clear branch chain and test each boundary.
- **C05-C08:** Write the loop interval and invariant before code; test empty, singleton, first, and last positions.
- **C09:** Cast an operand before multiplication. **C10:** compare against `Integer.MIN_VALUE/MAX_VALUE` or use `Math.toIntExact`.
- **C11-C12:** The colon switch needs explicit completion; the arrow switch should return/assign one value per arm and label its Java version.
- **C13-C14:** Validate dimensions/bounds; overloading requires distinct parameter types and should preserve one conceptual contract.
- **C15-C17:** Assert caller state after primitive copy, shared mutation, and reference reassignment; the distinction is the copied value plus aliasing.
- **C18:** Check `character >= '0' && character <= '9'` or use `Character.digit` and reject `-1`.
- **C19:** Extract named predicates/helpers only when they clarify the contract; preserve failure order and output with tests.

Follow-up guidance F01-F13

- **F01-F02:** Separate source compatibility, class-file level, runtime linkage, dependency compatibility, and behavior; `--release` constrains language/API signature set, not third-party or environment behavior.
- **F03-F05:** Make numeric and evaluation policy explicit; parentheses communicate intent and short-circuiting prevents invalid/expensive work.
- **F06-F07:** Name a non-negative measure that strictly decreases. Prefer a helper return when a label would obscure ownership of the exit.
- **F08-F09:** Recompilation can select a new overload or become ambiguous; `varargs` is an array parameter with call-site packing syntax.
- **F10:** Recursion depth depends on frames, JVM configuration, and execution; the language promises no fixed safe depth.
- **F11:** Test odd sums, negative values, zero count, and large totals.
- **F12-F13:** Use compiler diagnostics, thrown stack traces, wrong outputs, and environment evidence rather than guesses.

A01 model: A strong answer widens before arithmetic, rejects invalid tokens at the input boundary, never catches unrelated runtime defects, uses a single pass, and tests no args, one value, negatives, zero, extreme ints, malformed text, and product overflow policy.

SDE-2 CORE CHAPTER

Chapter 19 - Solution Studio B - Arrays, Strings, and Objects

Use this studio only after Practice Lab B. Redraw every alias diagram before reading an answer, and distinguish object identity, value equality, copy depth, mutation, and runtime dispatch. A solution is complete when the code compiles, the object/reference picture matches the code, and one boundary test would fail under the original misconception.

The coding guidance gives design constraints rather than a single canonical implementation. Preserve those constraints while choosing clear names and the smallest useful API. Add missed rules to the same error log used in Practice Lab A.

Knowledge answers K26-K50

- **K26:** Declaration introduces an array reference; creation allocates fixed-length storage; initialization supplies initial element values or defaults.
- **K27:** Numeric zero, false, `\u0000` for char, and null for references.
- **K28:** Assignment copies the same array reference, so element mutation is shared.
- **K29:** Shallow copy duplicates one container while nested references remain shared; deep copy recursively duplicates the state required by the contract.
- **K30:** Arrays inherit identity-oriented `Object.toString`; use `Arrays.toString` or `deepToString`.
- **K31:** `equals` compares one-dimensional element values/references appropriately; `deepEquals` descends into nested arrays.
- **K32:** A two-dimensional array is an array of row references, and each row can have a different length or be null.
- **K33:** Its public contract exposes no mutation; operations return new results, enabling safe sharing and stable hashing.
- **K34:** Identity asks same object, content asks same characters, pooling may share selected strings but is not the value contract.
- **K35:** Empty has length zero; blank contains only whitespace under the chosen API; null means no String reference.
- **K36:** Use a builder when repeated updates would copy a growing immutable result, especially in loops.
- **K37:** One UTF-16 code unit.
- **K38:** Code unit is one char, code point is a Unicode scalar value represented by one/two units, grapheme is a user-perceived cluster that may include multiple code points.
- **K39:** Class defines type; object is instance; field stores state; method defines behavior.
- **K40:** A reference value designating an object or null, not the full object.
- **K41:** Only when the class declares no constructor, subject to superclass construction accessibility.
- **K42:** Receiver access/disambiguation, constructor chaining with `this(...)`, and briefly passing/returning the receiver.
- **K43:** It is shared, order-dependent state that leaks across tests and requires concurrency reasoning.
- **K44:** Same-package access is broad; cross-package subclass access is through inherited context, not arbitrary base/sibling instances.
- **K45:** It enables a short source name; it does not copy, load, or repack the type.
- **K46:** Encapsulation protects representation and invariants through domain operations; indiscriminate setters may destroy both.

- **K47:** Overloading is compile-time parameter-list selection; overriding is runtime instance dispatch; fields/static methods hide by declared type.
- **K48:** Interface is a role/contract, abstract class may share state/construction/implementation, concrete class is instantiable and complete.
- **K49:** For HAS-A, configurable collaboration, false subtype relationships, localized invariants, and test seams.
- **K50:** If two keys are equal, their hash codes must be equal, and equality/hash-relevant state must remain stable while keyed.

Output answers O20-O38

- **O20:** `0`; int array elements receive default zero.
- **O21:** The original shows the mutation because both variables alias one array.
- **O22:** Original nested element changes because outer clone shares row references.
- **O23:** `[1, 2]`; the utility formats contents.
- **O24:** Both rows are null; only the outer array was created.
- **O25:** `av`; start is inclusive and end exclusive.
- **O26:** Usually/defined for identical interned literal occurrence as true, but value code must use equals because nonliteral construction/runtime concatenation need not share identity.
- **O27:** `false`; explicit construction creates another object.
- **O28:** Original lowercase value; the returned String was ignored.
- **O29:** Apply operations in source order; the builder mutates in place, unlike String.
- **O30:** `2`; the supplementary code point occupies a surrogate pair.
- **O31:** Only the renamed object's field changes; objects have independent state.
- **O32:** Both aliases observe the new field value.
- **O33:** Target constructor body prints before the delegating constructor continues.
- **O34:** Three if each constructor increments once; static state is shared.
- **O35:** Parent field, child method; fields are hidden and instance methods dispatch.
- **O36:** Compile-time overload chosen from the declared type and argument type.
- **O37:** Implementing object's override runs through the interface reference.
- **O38:** Lookup can fail after key mutation because hash/equality no longer match original bucket placement.

Debug answers D20-D38

- **D20:** Use `< length`. **D21:** copy before mutation when preservation is the contract.
- **D22:** Allocate outer storage and clone/copy each non-null row. **D23:** use `Arrays.equals` or `deepEquals`.
- **D24:** use `Arrays.toString/deepToString`. **D25:** check each row and use `matrix[row].length`.
- **D26:** use `equals` or `Objects.equals`. **D27:** reject null, `Objects.equals`, or constant-first equality according to contract.
- **D28:** use a builder. **D29:** validate ASCII range or use `Character.digit`.
- **D30:** traverse code points with `codePoints()/codePointAt` and define grapheme limitations.
- **D31:** remove `void`; otherwise it is a method. **D32:** declare the no-argument constructor explicitly and delegate to validation.
- **D33:** pass a receiver or make it an instance method. **D34:** expose validated domain operations instead of raw assignment.
- **D35:** match parameters and add `@Override`. **D36:** guard with `instanceof` or move needed behavior to the base contract.
- **D37:** implement an interface's public method as public. **D38:** defensively copy on input and return an unmodifiable copy/view that cannot expose mutation.

Coding guidance C20-C38

- **C20-C25:** State shape, null-row, mutation, and copy-depth contracts; test empty, singleton, jagged, and null rows.
- **C26-C29:** Use equals for value, builder for repeated construction, and code-point/digit APIs when the contract exceeds ASCII code units.
- **C30-C34:** Validate constructors, keep fields private, make class/static ownership explicit, and implement equality from stable value state.
- **C35-C37:** Demonstrate one actual override with `@Override`; call through base/interface references to prove runtime dispatch. Abstract classes may construct shared state.
- **C38:** Inject the collaborator through the constructor, delegate through its interface, and test with a deterministic fake; no subtype claim is needed.

Follow-up guidance F14-F26

- **F14-F16:** Mutation is an API decision; copy depth and sorted-input prerequisites must be stated and tested.
- **F17-F18:** `char` is correct for an explicitly UTF-16-code-unit/ASCII contract; complexity units must match traversal.
- **F19-F21:** Final stops reassignment, not nested mutation; constructor dispatch can expose default subclass state; protected cross-package access is inheritance-scoped.
- **F22-F24:** Abstract class for shared state/template, interface for role/open implementations, composition for replaceable HAS-A behavior.
- **F25-F26:** Hash tables rely on equal hash codes; immutable owners copy mutable input and avoid exposing mutable internal aliases.

A02 model: A strong design uses immutable value identifiers, one guarded aggregate for state transitions, an enum rather than strings, an injected policy interface, no public mutable collections, and tests illegal transitions, duplicate identifiers, null collaborators, and alias attempts.

SDE-2 CORE CHAPTER

Chapter 20 - Solution Studio C - Wrappers, Generics, Collections, Exceptions, I/O

Use this studio after completing Practice Lab C. Review answers in this order: collection semantics, type/null behavior, ordering, failure contract, and cost. For I/O and exceptions, confirm who owns the resource and whether a catch can genuinely recover or translate at that boundary.

Retype every missed comparator, unboxing, fixed-size-list, heap-order, or try-with-resources example. Change the input so a folklore-based implementation fails; the distinguishing test is more valuable than memorizing the fixed line.

Knowledge answers K51-K75

- **K51:** Byte, Short, Integer, Long, Float, Double, Character, Boolean.
- **K52:** Parsing converts text to a value; boxing wraps a primitive; unboxing extracts its primitive value.
- **K53:** Collections/maps may carry a null wrapper through several calls before arithmetic or assignment triggers implicit unboxing.
- **K54:** Caching and allocation are implementation/language optimizations; equals defines wrapper value equality.
- **K55:** The compiler rejects non-String additions and returns String from get without an unchecked caller cast.
- **K56:** Generic type arguments are reference types under Java's generic type system; use wrappers for primitives.
- **K57:** It infers type arguments from target and constructor context, not runtime element inspection.
- **K58:** An upper bound exposes members of the bound, for example compareTo for a Comparable bound.
- **K59:** Enum constants are compiler-checked finite values and support typed behavior/switching.
- **K60:** Ordinals depend on declaration order and change when constants are inserted/reordered.
- **K61:** Sequence/List, uniqueness/Set, association/Map, FIFO/Queue, double-ended or stack/Deque, priority head/PriorityQueue.
- **K62:** Locating a position can be linear, nodes add allocation/cache costs, and iterator-position insertion is a narrower scenario.
- **K63:** Expected constant-time wording assumes suitable hashing/load and ordinary implementation behavior; worst cases and adversarial keys require qualification.
- **K64:** Offer/poll/peek signal failure with false/null; add/remove/element may throw. Check each implementation's null policy.
- **K65:** Heap structure guarantees only the head; its backing traversal is not globally sorted.
- **K66:** Return negative/zero/positive consistently, use compare helpers, and add deterministic tie-breakers where total order matters.
- **K67:** Checked exceptions need catch/declare; unchecked exceptions often signal programming/precondition failures; Errors indicate serious runtime conditions and are not ordinary recovery flow.
- **K68:** `throw` raises one exception object; `throws` declares possible propagation in a method signature.
- **K69:** Resources close in reverse declaration order; close failures may be suppressed behind a primary failure.
- **K70:** A general catch first would make a subtype catch unreachable.

- **K71:** Small, convenience-first interactive/token input where its overhead and token/line semantics are acceptable.
- **K72:** It offers explicit line/token control and commonly lower overhead for large input.
- **K73:** The component that creates/owns the resource usually closes it; borrowed streams remain open unless the API says otherwise.
- **K74:** Fixed-size forbids size changes, unmodifiable forbids mutation through that view/reference, immutable promises no observable content change.
- **K75:** It can hide null, index, cast, and invariant defects, blur failure contracts, and continue with corrupt state.

Output answers O39-O57

- **O39:** 43; text parses to primitive 42 then adds 1.
- **O40:** NullPointerException during unboxing.
- **O41:** `true` for the required small wrapper cache, not a general value-comparison pattern.
- **O42:** `equals` is true regardless of identity.
- **O43:** `x`; generic return type is String.
- **O44:** The matching PAID arm's result; enum matching is typed.
- **O45:** 2; duplicate 2 is not added.
- **O46:** 3; each merge increments the same key.
- **O47:** Values leave in insertion order from the opposite end.
- **O48:** Most recently pushed leaves first.
- **O49:** 3; natural-order min-heap exposes minimum.
- **O50:** Positive 1; compare avoids overflow.
- **O51:** `[x, b]`; element replacement is allowed.
- **O52:** UnsupportedOperationException; size is fixed.
- **O53:** 1; the primitive array is one reference element.
- **O54:** The first compatible catch in source order; specific-before-general is required.
- **O55:** Reverse declaration order.
- **O56:** 10; hexadecimal A represents ten.
- **O57:** ArithmeticException; exact arithmetic rejects overflow.

Debug answers D39-D57

- **D39:** `List<Integer>`. **D40:** reject/default/check before unboxing.
- **D41:** use `equals/Objects.equals`. **D42:** supply a type argument and remove unchecked mixed usage.
- **D43:** reject empty or return Optional under an explicit contract. **D44:** declare enum constants and parse external text at the boundary.
- **D45:** cover all states or use a deliberate default with evolution trade-off. **D46:** let `Set.add`'s boolean decide whether to append.
- **D47:** use `merge`, `getOrDefault`, or `compute`. **D48:** map push/pop/peek to `ArrayDeque`.
- **D49:** use primitive compare helpers and `thenComparing`. **D50:** poll a copy or sort a snapshot.
- **D51:** use `iterator.remove/removeIf/separate` changes. **D52:** use an immutable, equality-stable key.
- **D53:** order specific to general. **D54:** remove return/throw that masks the primary completion from finally.
- **D55:** declare owned resources in try parentheses. **D56:** consume the pending newline deliberately or use one tokenization model consistently.
- **D57:** document borrowed ownership and do not close it; wrap only resources the method owns.

Coding guidance C39-C57

- **C39-C43:** Keep conversion at boundaries, define null/empty behavior, retain generic type safety, and avoid enum ordinals in external contracts.
- **C44-C49:** Select by required semantics and assert result order/membership/frequency, including empty and duplicates.
- **C50-C52:** Use heap head operations, safe comparators, and supported iterator mutation; state whether input is preserved.
- **C53-C55:** Catch only exceptions you can translate/recover from, propagate checked boundaries honestly, and use try-with-resources for owned resources.
- **C56-C57:** Validate declared sizes, token counts, missing lines, extra tokens, and jagged row lengths; separate parsing from algorithm logic.

Follow-up guidance F27-F38

- **F27-F30:** Boxing can hide allocation and nullable state; cache identity is irrelevant; bounds belong only when operations require them.
- **F31:** Persist stable external codes, not ordinal; decide whether switches should fail compilation or use an explicit unknown policy.
- **F32-F35:** Start with ordering, uniqueness, key lookup, FIFO/LIFO/priority contracts, then qualify costs; heaps promise head order only.
- **F36-F38:** Illegal preconditions can be unchecked; try-with-resources records close failures as suppressed; resource ownership is part of the method contract.

A03 model: Use `HashMap.merge`, `LinkedHashSet` when encounter order matters, a small immutable entry for heap candidates, `Integer.compare/comparingInt` plus a stable tie-breaker, and bounded heap logic if only top three are needed.

A04 model: Parameterize every collection, replace identity with value equality, replace subtraction with compare helpers, narrow catches to the boundary, and leave caller-owned resources open. Re-run behavior tests after each refactor.

SDE-2 CORE CHAPTER

Chapter 21 - Solution Studio D - Traps and Readiness

This final studio is a calibration tool, not a shortcut. Check the rule, the predicted behavior, the repair, and the follow-up separately. If one layer is weak, return to the originating chapter and rerun its companion example before retrying the item.

Use the readiness interpretation at the end to decide what to study next. A high score with incorrect pass-by-value, equality, overflow, aliasing, or dispatch explanations is not a pass, because those misconceptions contaminate later array, hashing, recursion, and graph solutions.

Knowledge answers K76-K100

- **K76:** Identity asks whether two references designate the same object; semantic equality asks whether values satisfy a type's equals contract.
- **K77:** Interning provides a canonical pooled reference for selected string content; it does not mean every equal String is created only once or justify identity comparison.
- **K78:** The constructor allocates a distinct String object while preserving the same character content.
- **K79:** Both parameters and callers may designate one object, so mutation is visible; assignment changes only the local reference copy.
- **K80:** Expression operand types determine arithmetic before assignment conversion.
- **K81:** When representation error is acceptable and the domain defines scale-aware absolute/relative tolerance; exact financial/identifier quantities often need another representation.
- **K82:** Java's floating-to-integral cast discards the fractional part toward zero by rule.
- **K83:** Two's-complement signed ranges contain one more negative value, so the minimum's positive magnitude is unrepresentable in the same type.
- **K84:** One array object and its element values; reference elements still designate the same nested objects.
- **K85:** Object arrays can serve as the generic varargs array; a primitive array is itself one Object/reference element.
- **K86:** Strings are immutable, so transformations normally return a new or existing result rather than mutate the receiver.
- **K87:** It validates and converts according to a radix and can recognize more than ASCII digits; **-1** signals failure.
- **K88:** Fall-through executes subsequent statements; multi-label arrow cases select one body for several labels without fall-through.
- **K89:** Constructor syntax is distinguished by class name and absence of a return type; adding one declares a method.
- **K90:** An explicit **this(...)** or **super(...)** must be first, and only one is explicit in a constructor.
- **K91:** Overridable instance methods. Fields and static methods are selected/hide by declared type.
- **K92:** Repeated downcasts can mean needed behavior is missing from the base interface or the model exposes the wrong abstraction.
- **K93:** A supertype catch first consumes all subtype instances, making later subtype handling unreachable.
- **K94:** It may return/throw abruptly itself, or not run/completely execute under forced termination, host/process failure, or nontermination.
- **K95:** Hash lookup uses the insertion-time hash/equality placement; changing those values breaks the lookup path.

- **K96:** Subtraction can overflow and reverse comparator sign; compare helpers preserve ordering laws.
- **K97:** It may detect structural interference and throw, but timing is not guaranteed and it provides no thread-safety contract.
- **K98:** `[0, length)`.
- **K99:** A well-founded non-negative quantity that strictly decreases (or bounded quantity that increases) on every continuing iteration.
- **K100:** Contract/input, types/arithmetic, aliases/mutation, bounds/termination, API guarantees/failures, then time/auxiliary space and tests.

Output answers O58-O75

- **O58:** `false` then `true`; distinct identity, equal content.
- **O59:** Often/defined constant folding yields the pooled literal reference, but use equals for correctness.
- **O60:** Mutation is visible; parameter reassignment is not.
- **O61:** `1410065408`; int overflow precedes long assignment.
- **O62:** `-2` and `-1`; division truncates toward zero and remainder preserves the identity.
- **O63:** `false`; these binary floating values do not equal the literal 0.3 exactly.
- **O64:** `-10`; truncation is toward zero.
- **O65:** `Integer.MIN_VALUE`, still negative.
- **O66:** Original nested content changes because row references are shared.
- **O67:** set works; add throws `UnsupportedOperationException`.
- **O68:** Original String unchanged because result was ignored.
- **O69:** String length 2 and code-point count 1 for the example.
- **O70:** All reached later colon cases print until a terminating statement.
- **O71:** Parent field and child override result.
- **O72:** Compile-time overload applicable to declared argument/reference types.
- **O73:** Finally's return replaces the pending return; avoid this code.
- **O74:** Lookup may return null because the key moved logically but not structurally.
- **O75:** No sorted-iteration conclusion is valid; only head operations carry the priority guarantee.

Debug answers D58-D75

- **D58:** primitives use operators; nullable objects use `Objects.equals`; arrays use `Arrays.equals/deepEquals`; custom keys need `equals/hashCode`.
- **D59:** replace identity checks with content checks; intern only for an explicitly designed canonicalization mechanism.
- **D60:** return the swapped pair/container or mutate caller-owned container elements; Java cannot swap caller variables by parameter reassignment.
- **D61:** widen an operand before multiplication or use `Math.multiplyExact` in the chosen width.
- **D62:** use integer minor units or `BigDecimal` for exact currency policy; do not patch with arbitrary epsilon.
- **D63:** use `Math.round/floor/ceil` according to contract and validate range.
- **D64:** use `Math.floorMod(hash, capacity)` with positive capacity, or bit masking only under a justified power-of-two contract.
- **D65:** copy each row/nested mutable element to the depth promised and handle null explicitly.
- **D66:** loop/stream-box into a new `ArrayList<Integer>`.
- **D67:** assign or return the transformation result.
- **D68:** traverse code points and state whether grapheme-cluster behavior is required.
- **D69:** add termination or use arrow syntax.
- **D70:** declare one and delegate to the validated constructor.
- **D71:** delegate first, then validate in the target constructor/factory.
- **D72:** call static methods through class names and do not expect runtime dispatch.
- **D73:** put needed behavior on the polymorphic contract or use a checked pattern at a true type boundary.
- **D74:** specific catches first; translate only at an ownership boundary and preserve cause.
- **D75:** use an inequality or modular reachability/progress proof; do not rely on hitting an exact sentinel accidentally.

Coding guidance C58-C75

- **C58-C60:** Define null and identity contracts, use `Objects.equals`, and illustrate copied reference values explicitly.
- **C61-C63:** Widen before arithmetic, define division/rounding, and use `floorMod` for non-negative indexing.
- **C64-C67:** State copy depth, box explicitly, use builder for mutable construction, and traverse code points under Unicode contracts.
- **C68-C71:** Label switch version, keep constructor invariants centralized, separate all dispatch kinds, and prefer base behavior over casts.
- **C72-C74:** Inspect `getSuppressed`, use immutable keys, and poll a defensive heap copy when sorted output plus preservation are required.
- **C75:** A strong refactor names `calculateSum`, decides `int` versus `long` from constraints, handles null by contract, uses enhanced-for where index is irrelevant, tests empty/singleton/extremes, and states $O(n)$ time/ $O(1)$ auxiliary space.

Follow-up guidance F39-F50

- **F39-F41:** Match equality to representation, remember overload selection is compile-time, and test products/sums around primitive limits.
- **F42-F44:** Numeric and Unicode representations are domain contracts, not one-size-fits-all implementation details.
- **F45-F46:** Keep deliberate fall-through only when grouped behavior is unmistakable; fields are resolved from declared type while instance methods dispatch.
- **F47-F49:** Boundary catches translate with cause/context; mutable keys violate stable lookup; heaps promise the best head after each mutation.
- **F50:** Restate input/output, constraints, examples, edge cases, type/structure choice, invariant, algorithm, complexity, and tests before implementation.

A05 model: Find failures by stage, repair the narrowest violated contract, then prove with distinguishing boundary tests. Avoid behavior-changing cleanup until the original defect is covered.

Final readiness interpretation

- Below 80 percent on K/O/D: revisit the named chapter and re-run the examples.
- Compiling but poorly explained C tasks: practice verbal invariants and contracts.
- Incorrect equality/pass-by-value/type-promotion explanation: do not advance; these faults spread into every DSA book.
- Strong fundamentals but slow implementation: move to Loop Mastery and Arrays while keeping this book as a reference.

SDE-2 CORE CHAPTER

Chapter 22 - Java Syntax and Language Quick Reference

This appendix is a recall sheet, not a substitute for the language chapters. Every compact rule hides edge cases. When correctness depends on an exact compile-time rule, use the Java Language Specification linked in Appendix G.

After Java Foundations, continue to [Java-SDE2-DSA-02-Time-and-Space-Complexity.pdf](#), then use [Java-SDE2-DSA-01-Number-Systems-and-Math-Foundations.pdf](#) for a printable numeric quick-reference with worked algorithms. For the full focused sequence, open [series/dist/Java-SDE2-Interview-Preparation-Series-Index.pdf](#).

Source files, packages, and launch

A conventional source file begins with an optional package declaration, followed by imports and type declarations:

JAVA

```
package com.example.orders;

import java.time.Instant;
import java.util.List;

public final class OrderBatch {
    private final List<String> ids;
    private final Instant createdAt;

    public OrderBatch(List<String> ids, Instant createdAt) {
        this.ids = List.copyOf(ids);
        this.createdAt = createdAt;
    }

    public static void main(String[] args) {
        var batch = new OrderBatch(List.of("demo"), Instant.now());
        System.out.println("orders=" + batch.ids.size());
    }
}
```

At most one top-level public type appears in a source file, and its name normally matches the file name. A package name maps to a directory layout by convention and by most build tools. The unnamed package is unsuitable for maintained applications because named packages cannot import from it.

Common commands:

TEXT

```
javac --release 21 -d out src/com/example/orders/OrderBatch.java
java -cp out com.example.orders.OrderBatch
java --enable-preview --source 21 PreviewExample.java
javap -classpath out -c -v com.example.orders.OrderBatch
```

`--release 21` selects a supported language level and matching documented Java SE APIs. It is safer for cross-compilation than setting only `-source` and `-target`. Java 21 features covered in this book do not require preview mode when they were final in that release.

Primitive types, references, and default values

Java has eight primitive types:

Type	Width or model	Representative range or values	Default field value
<code>byte</code>	8-bit signed	-128 through 127	<code>0</code>
<code>short</code>	16-bit signed	-32,768 through 32,767	<code>0</code>
<code>int</code>	32-bit signed	-2^{31} through $2^{31} - 1$	<code>0</code>
<code>long</code>	64-bit signed	-2^{63} through $2^{63} - 1$	<code>0L</code>
<code>char</code>	16-bit unsigned UTF-16 code unit	<code>\u0000</code> through <code>\uffff</code>	<code>\u0000</code>
<code>float</code>	IEEE 754 binary32	finite values, infinities, NaN	<code>0.0f</code>
<code>double</code>	IEEE 754 binary64	finite values, infinities, NaN	<code>0.0d</code>
<code>boolean</code>	logical value	<code>true</code> , <code>false</code>	<code>false</code>

Reference variables hold reference values or `null`; they do not contain an object inline as a language guarantee. Fields and array elements receive defaults. Local variables must be definitely assigned before use.

Integer arithmetic on `byte`, `short`, and `char` is normally promoted to `int`. Integer overflow wraps using two's-complement arithmetic; it does not throw. Use `Math.addExact`, `subtractExact`, and related methods when overflow must be rejected. Floating-point NaN is unequal to every value, including itself; use `Double.isNaN` rather than equality.

Useful literals:

JAVA

```
int decimal = 1_000_000;
int hexadecimal = 0xFF;
int binary = 0b1010_0110;
long longValue = 9_000_000_000L;
float ratio = 0.25F;
char newline = '\n';
String textBlock = """
    line one
    line two
    """;
```

Conversions, boxing, and numeric comparison

Widening primitive conversions such as `int` to `long` are implicit, although a large integer can lose precision when converted to floating point. Narrowing conversions require a cast and can discard high bits or truncate a fractional component.

Boxing converts a primitive to a wrapper, and unboxing extracts the primitive. Unboxing `null` throws `NullPointerException`. Wrapper identity is not a numeric comparison:

JAVA

```
Integer a = 1_000;
Integer b = 1_000;
boolean wrongQuestion = a == b;           // compares references
boolean valueEquality = a.equals(b);       // compares int values
```

Do not rely on wrapper caches. Prefer primitives in hot numeric loops unless nullability, generic APIs, or object semantics are required.

For money, binary floating point is usually the wrong domain model. Use an integer minor-unit representation when scale is fixed, or use `BigDecimal` with an explicit scale and rounding policy. `new BigDecimal("0.1")` is predictable; `new BigDecimal(0.1)` captures the binary floating-point approximation.

Operators and precedence

From high to low, a practical precedence summary is:

Family	Operators
postfix	<code>expr++, expr--</code>
unary	<code>++, --, +, -, ~, !, cast</code>
multiplicative	<code>*, /, %</code>
additive	<code>+, -</code>
shift	<code><<, >>, >>></code>
relational	<code><, <=, >, >=, instanceof</code>
equality	<code>==, !=</code>
bitwise	<code>&, then ^, then `</code>
conditional logical	<code>&&, then `</code>
conditional	<code>condition ? yes : no</code>
assignment	<code>=, compound assignments</code>

Use parentheses when a reviewer would otherwise have to recall the table. `&&` and `||` short-circuit; `&` and `|` evaluate both operands even for booleans. Compound assignment includes an implicit conversion: `byteValue += 1` compiles where `byteValue = byteValue + 1` needs a cast.

`==` compares primitive values or reference identity. Semantic object equality is normally expressed by `equals`. Strings must not be compared with `==`.

Control flow and pattern matching

Basic forms:

JAVA

```
if (ready) {
    start();
} else {
    defer();
}

for (int i = 0; i < values.length; i++) {
    consume(values[i]);
}

for (String value : values) {
    consume(value);
}

while (condition()) {
    step();
}

do {
    step();
} while (condition());
```

A switch expression returns a value and should be exhaustive:

JAVA

```
enum State { NEW, RUNNING, DONE }

String label = switch (state) {
    case NEW -> "queued";
    case RUNNING -> "active";
    case DONE -> "complete";
};
```

The colon form retains fall-through semantics. Prefer arrow labels unless fall-through is deliberate and obvious.

Pattern variables are flow-scoped:

JAVA

```
if (candidate instanceof String text && !text.isBlank()) {
    System.out.println(text.length());
}
```

Record patterns and pattern matching for switch are final in Java 21. A sealed hierarchy lets the compiler check exhaustiveness when all permitted direct subtypes are known in the relevant compilation context.

Methods, overloads, and parameter passing

A method signature for overloading consists of the method name and parameter types. Return type alone cannot distinguish overloads. Resolution occurs at compile time using the declared argument types, applicability, conversions, and most-specific rules. Overriding is dynamic dispatch based on the runtime receiver type.

JAVA

```
static long sum(int left, long right) {
    return left + right;
}

static int sum(int... values) {
    int total = 0;
    for (int value : values) total += value;
    return total;
}
```

Varargs is an array parameter with call-site convenience. It can allocate an array and can create overload ambiguity. A caller may pass `null` as the entire array, so a public varargs method still needs a null policy.

Java is always pass-by-value. For an object argument, the copied value is a reference. The callee can use that copied reference to mutate the same mutable object, but assigning a different object to the parameter does not change the caller's variable.

Arrays, strings, and text

Arrays are reified, covariant objects with a fixed length:

JAVA

```
String[] names = new String[3];
int[][] matrix = new int[4][5];
int[] copy = java.util.Arrays.copyOf(values, values.length);
```

Covariance can fail at runtime: assigning a `String[]` to `Object[]` is legal, but storing a non-string throws `ArrayStoreException`. Generic collections are invariant and catch the analogous mismatch at compile time.

`String` is immutable. Concatenation in one expression is compiled using platform mechanisms selected by the compiler; repeated concatenation in a loop should normally use `StringBuilder`. `String.length()` counts UTF-16 code units, not user-perceived characters. A supplementary Unicode code point uses a surrogate pair. Use `codePoints()` when code points are the needed abstraction and a text library when grapheme clusters matter.

Common equality and emptiness checks:

JAVA

```
boolean same = java.util.Objects.equals(left, right);
boolean empty = text.isEmpty();
boolean blank = text.isBlank();
```

Classes, records, interfaces, and sealed hierarchies

Class skeleton:

JAVA

```
public final class Account {
    private final String id;
    private long balance;

    public Account(String id) {
        this.id = java.util.Objects.requireNonNull(id);
    }

    public synchronized void credit(long cents) {
        balance = Math.addExact(balance, cents);
    }
}
```

Access levels:

Modifier	Same class	Same package	Subclass in another package	Everywhere
private	yes	no	no	no
package-private	yes	yes	no	no
protected	yes	yes	qualified rules apply	no
public	yes	yes	yes	yes

Constructors are not inherited. A subclass constructor begins with an explicit or implicit invocation of another constructor. Static methods are hidden, not overridden. Private methods are not overridden. An overriding method may use a covariant return type and cannot broaden checked exceptions.

A record declares a nominal data carrier:

JAVA

```
public record Range(int start, int end) {
    public Range {
        if (start > end) throw new IllegalArgumentException("reversed");
    }
}
```

Records are shallowly immutable: component fields are final, but a referenced component can still be mutable. Make defensive copies when value semantics require them.

An interface defines a contract and can contain abstract, default, static, and private methods. An abstract class can hold instance state and protected implementation machinery. Prefer composition when the relationship is behavioral reuse rather than a stable subtype relationship.

Sealed types restrict direct subtypes:

JAVA

```
sealed interface Result permits Success, Failure {}
record Success(String value) implements Result {}
record Failure(String message) implements Result {}
```

Each permitted subtype must be `final`, `sealed`, or `non-sealed` unless its form, such as a record, is implicitly final.

Equality, hashing, and ordering

The `equals` contract requires reflexivity, symmetry, transitivity, consistency, and a false result for null. Equal objects must have equal hash codes. Unequal objects may collide. Fields used by `equals` and `hashCode` should not change while the object is a key in a hash-based collection.

`Comparable<T>` defines a natural ordering. `Comparator<T>` defines an external ordering and supports composition:

JAVA

```
Comparator<Person> order = Comparator
    .comparing(Person::lastName)
    .thenComparing(Person::firstName)
    .thenComparingInt(Person::age);
```

An ordering inconsistent with equality is legal for some APIs but can make a sorted set or map treat unequal values as the same key. Document the choice.

Exceptions and resource management

Checked exceptions are subclasses of `Exception` excluding `RuntimeException`; the compiler enforces catch-or-declare. Unchecked exceptions include `RuntimeException` and `Error`. Do not catch `Throwable` as routine application control flow.

JAVA

```
try (var input = java.nio.file.Files.newBufferedReader(path)) {
    return input.readLine();
} catch (java.io.IOException failure) {
    throw new UncheckedIOException("cannot read " + path, failure);
}
```

Try-with-resources closes initialized resources in reverse order. If both the body and close fail, the body failure is primary and close failures are suppressed. Inspect `getSuppressed()` when diagnosing layered resource failures.

Catch the narrowest useful type, preserve the cause when translating, and attach context without secrets. An interruption caught as `InterruptedException` should normally be propagated or restored with `Thread.currentThread().interrupt()`.

Generics and variance

Generic types are invariant: `List<Integer>` is not a subtype of `List<Number>`. Use bounded wildcards at API boundaries:

JAVA

```
static double total(java.util.List<? extends Number> source) {  
    return source.stream().mapToDouble(Number::doubleValue).sum();  
}  
static void addDefaults(java.util.List<? super Integer> sink) { /* write */ }
```

Mnemonic: producer extends, consumer super. A wildcard is not a declaration-site type parameter and may need capture by a helper method.

Type erasure means most type arguments are not reified at runtime. Consequently, Java prohibits `new T()`, `new T[]`, `instanceof List<String>`, and overloading methods whose signatures erase to the same form. Heap pollution occurs when a variable of a parameterized type refers to an incompatible value, often through raw types, unchecked casts, or generic varargs.

Prefer a generic method when the type relationship is local to one operation. Prefer a generic class when the type relationship belongs to the object's persistent state or contract.

Lambdas, method references, and streams

A lambda implements a functional interface, an interface with one abstract method after inheritance rules:

JAVA

```
java.util.function.Predicate<String> useful =  
    value -> value != null && !value.isBlank();  
java.util.function.Function<String, Integer> length = String::length;
```

Captured local variables must be final or effectively final. Capturing mutable state does not make access thread-safe.

Streams describe a lazy traversal pipeline. Intermediate operations are lazy; a terminal operation drives evaluation:

JAVA

```
Map<String, Long> counts = words.stream()  
    .filter(useful)  
    .map(String::toLowerCase)  
    .collect(java.util.stream.Collectors.groupingBy(  
        java.util.function.Function.identity(),  
        java.util.stream.Collectors.counting()));
```

Do not reuse a consumed stream. Avoid side effects in behavioral parameters. Parallel streams use shared runtime machinery and are not automatically faster; measure a representative, sufficiently large, CPU-oriented workload and account for blocking, ordering, splitting, and contention.

Concurrency recall sheet

Key rules:

- Starting a thread happens-before actions in that thread.
- All actions in a thread happen-before another thread detects its termination, including an untimed `join()` that returns.
- An unlock on a monitor happens-before a later lock on the same monitor.
- A volatile write happens-before a later read of that same field in synchronization order.
- Final fields receive special visibility guarantees when construction is correct and `this` does not escape.
- `volatile` supplies visibility and ordering for the field; it does not turn `count++` into one atomic action.
- Higher-level concurrent collection and executor methods have their own documented memory-consistency effects.

Prefer ownership, immutability, message passing, and well-defined concurrent components over scattered low-level synchronization. Every blocking operation needs a cancellation and shutdown story.

Java 21 virtual threads make thread-per-task practical for many blocking workloads. They do not remove resource limits, make CPU work parallel beyond available processors, or repair data races. Bound downstream resources such as connections and request quotas rather than pooling virtual threads merely to limit their count.

Common interview traps

- Java has no pass-by-reference parameter mode.
- `String` immutability does not make every object containing a string immutable.
- `final` freezes a variable binding or prevents overriding/inheritance in context; it is not a universal deep-immutability or thread-safety keyword.
- `volatile` is not mutual exclusion.
- `HashMap` has no general iteration-order contract.
- `PriorityQueue` exposes only its head ordering; iteration is not sorted.
- `Arrays.asList` returns a fixed-size list backed by the array.
- `List.subList` is a backed view whose structural validity depends on disciplined modification.
- `Stream.toList()` returns an unmodifiable list by contract; do not assume a concrete implementation.
- `Optional` is primarily a return-type tool, not a universal field, parameter, or serialization model.
- `Thread.sleep` does not release a held monitor.
- `wait` must be called while owning the monitor, releases that monitor while waiting, and belongs in a condition loop.
- A successful test run cannot prove a racy program correct.
- Big-O does not describe allocation, locality, constant costs, warm-up, contention, or tail latency.

Use this appendix for rapid recall, then return to the relevant chapter to recover the mechanism, proof obligation, trade-off, and production implication.

SDE-2 CORE CHAPTER

Chapter 23 - Seventy Compiling Java Fundamentals Examples

This dependency-free Java 21 companion contains seventy executable checks spanning the mandatory fundamentals and core collection usage. Compile with `javac -Xlint:all -Werror JavaFundamentalsExamples.java` and run with `java JavaFundamentalsExamples`. A successful run prints exactly PASS 70 Java Fundamentals examples.

JAVA (continued 1/21)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.StringReader;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Scanner;
import java.util.Set;
import java.util.TreeSet;

/**
 * Seventy dependency-free Java 21 fundamentals examples.
 *
 * Each example is a small executable check. A successful run prints exactly:
 * PASS 70 Java Fundamentals examples
 */
public final class JavaFundamentalsExamples {
    private JavaFundamentalsExamples() {}

    public static void main(String[] args) {
        List<Runnable> examples = List.of(
            JavaFundamentalsExamples::example01FirstProgram,
            JavaFundamentalsExamples::example02VariableDeclaration,
            JavaFundamentalsExamples::example03PrimitiveRanges,
```

JAVA (continued 2/21)

```
JavaFundamentalsExamples::example04NumericPromotion,  
JavaFundamentalsExamples::example05SafeLongMultiplication,  
JavaFundamentalsExamples::example06WideningConversion,  
JavaFundamentalsExamples::example07NarrowingConversion,  
JavaFundamentalsExamples::example08IntegerDivision,  
JavaFundamentalsExamples::example09ShortCircuitEvaluation,  
JavaFundamentalsExamples::example10PrefixAndPostfix,  
JavaFundamentalsExamples::example11ConditionalStatements,  
JavaFundamentalsExamples::example12TraditionalSwitch,  
JavaFundamentalsExamples::example13SwitchExpression,  
JavaFundamentalsExamples::example14ForwardLoop,  
JavaFundamentalsExamples::example15ReverseLoop,  
JavaFundamentalsExamples::example16EnhancedForLoop,  
JavaFundamentalsExamples::example17NestedLoop,  
JavaFundamentalsExamples::example18MethodParametersAndReturn,  
JavaFundamentalsExamples::example19MethodOverloading,  
JavaFundamentalsExamples::example20PrimitivePassByValue,  
JavaFundamentalsExamples::example21MutableObjectPassByValue,  
JavaFundamentalsExamples::example22ReferenceReassignment,  
JavaFundamentalsExamples::example23OneDimensionalArray,  
JavaFundamentalsExamples::example24TwoDimensionalArray,  
JavaFundamentalsExamples::example25JaggedArray,  
JavaFundamentalsExamples::example26ArrayAliasing,  
JavaFundamentalsExamples::example27ArrayCopy,  
JavaFundamentalsExamples::example28StringEquality,  
JavaFundamentalsExamples::example29StringPool,  
JavaFundamentalsExamples::example30StringBuilder,  
JavaFundamentalsExamples::example31CharacterToDigit,  
JavaFundamentalsExamples::example32ClassAndObject,  
JavaFundamentalsExamples::example33ConstructorOverloading,  
JavaFundamentalsExamples::example34ConstructorChaining,  
JavaFundamentalsExamples::example35StaticFieldAndMethod,  
JavaFundamentalsExamples::example36Encapsulation,  
JavaFundamentalsExamples::example37ImmutableClass,  
JavaFundamentalsExamples::example38Inheritance,
```

JAVA (continued 3/21)

```
JavaFundamentalsExamples::example39MethodOverriding,  
JavaFundamentalsExamples::example40RuntimePolymorphism,  
JavaFundamentalsExamples::example41AbstractClass,  
JavaFundamentalsExamples::example42Interface,  
JavaFundamentalsExamples::example43Composition,  
JavaFundamentalsExamples::example44WrapperParsing,  
JavaFundamentalsExamples::example45AutoboxingAndUnboxing,  
JavaFundamentalsExamples::example46IntegerCaching,  
JavaFundamentalsExamples::example47GenericClassAndMethod,  
JavaFundamentalsExamples::example48ArrayList,  
JavaFundamentalsExamples::example49HashSet,  
JavaFundamentalsExamples::example50HashMapFrequency,  
JavaFundamentalsExamples::example51QueueWithArrayDeque,  
JavaFundamentalsExamples::example52StackWithArrayDeque,  
JavaFundamentalsExamples::example53PriorityQueue,  
JavaFundamentalsExamples::example54Enum,  
JavaFundamentalsExamples::example55CheckedException,  
JavaFundamentalsExamples::example56UncheckedException,  
JavaFundamentalsExamples::example57TryWithResources,  
JavaFundamentalsExamples::example58ScannerInput,  
JavaFundamentalsExamples::example59BufferedReaderInput,  
JavaFundamentalsExamples::example60InterviewQualityRefactoring,  
JavaFundamentalsExamples::example61ListOperations,  
JavaFundamentalsExamples::example62ListRemovalOverloads,  
JavaFundamentalsExamples::example63SetVariants,  
JavaFundamentalsExamples::example64MapUpdateApis,  
JavaFundamentalsExamples::example65MapIteration,  
JavaFundamentalsExamples::example66DequeConventions,  
JavaFundamentalsExamples::example67PriorityQueueComparator,  
JavaFundamentalsExamples::example68CollectionFactories,  
JavaFundamentalsExamples::example69SafeIteratorRemoval,  
JavaFundamentalsExamples::example70CopiesAndViews);  
  
examples.forEach(Runnable::run);  
System.out.println("PASS " + examples.size() + " Java Fundamentals examples");
```

JAVA (continued 4/21)

```
}

private static void check(boolean condition, String message) {
    if (!condition) {
        throw new AssertionError(message);
    }
}

static void example01FirstProgram() {
    String output = "Ready for Java";
    check(output.equals("Ready for Java"), "first program");
}

static void example02VariableDeclaration() {
    int solvedProblems = 10;
    solvedProblems = 11;
    check(solvedProblems == 11, "assignment");
}

static void example03PrimitiveRanges() {
    check(Byte.MIN_VALUE == -128 && Byte.MAX_VALUE == 127, "byte range");
    check(Integer.MIN_VALUE == -2_147_483_648, "int range");
}

static void example04NumericPromotion() {
    byte left = 100;
    byte right = 20;
    int sum = left + right;
    check(sum == 120, "byte operands promote to int");
}

static void example05SafeLongMultiplication() {
    long product = 100_000L * 100_000;
    check(product == 10_000_000_000L, "widen before multiply");
}
```

JAVA (continued 5/21)

```
static void example06WideningConversion() {
    int count = 10;
    long total = count;
    check(total == 10L, "widening");
}

static void example07NarrowingConversion() {
    double value = 10.8;
    int truncated = (int) value;
    check(truncated == 10, "narrowing truncates");
}

static void example08IntegerDivision() {
    check(5 / 2 == 2, "integer division");
    check(5 / 2.0 == 2.5, "floating division");
}

static void example09ShortCircuitEvaluation() {
    String text = null;
    boolean present = text != null && !text.isEmpty();
    check(!present, "short circuit null guard");
}

static void example10PrefixAndPostfix() {
    int value = 5;
    int before = value++;
    int after = ++value;
    check(value == 7 && before == 5 && after == 7, "increments");
}

static void example11ConditionalStatements() {
    int score = 72;
    String grade;
    if (score >= 70) grade = "pass";
}
```


JAVA (continued 6/21)

```
        else grade = "retry";
        check(grade.equals("pass"), "if else");
    }

    static String traditionalSwitch(int day) {
        String kind;
        switch (day) {
            case 6:
            case 7:
                kind = "weekend";
                break;
            default:
                kind = "weekday";
        }
        return kind;
    }

    static void example12TraditionalSwitch() {
        check(traditionalSwitch(7).equals("weekend"), "traditional switch");
    }

    static String switchExpression(int day) {
        return switch (day) {
            case 6, 7 -> "weekend";
            default -> "weekday";
        };
    }

    static void example13SwitchExpression() {
        check(switchExpression(1).equals("weekday"), "switch expression");
    }

    static void example14ForwardLoop() {
        int sum = 0;
        for (int value = 1; value <= 3; value++) sum += value;
    }
}
```

JAVA (continued 7/21)

```
        check(sum == 6, "forward loop");
    }

    static void example15ReverseLoop() {
        StringBuilder order = new StringBuilder();
        for (int index = 2; index >= 0; index--) order.append(index);
        check(order.toString().equals("210"), "reverse loop");
    }

    static void example16EnhancedForLoop() {
        int sum = 0;
        for (int number : new int[] {2, 4, 6}) sum += number;
        check(sum == 12, "enhanced for");
    }

    static void example17NestedLoop() {
        int cells = 0;
        for (int row = 0; row < 2; row++) {
            for (int column = 0; column < 3; column++) cells++;
        }
        check(cells == 6, "nested loop");
    }

    static int add(int left, int right) {
        return left + right;
    }

    static void example18MethodParametersAndReturn() {
        check(add(4, 5) == 9, "method result");
    }

    static int clamp(int value, int minimum, int maximum) {
        return Math.max(minimum, Math.min(value, maximum));
    }
}
```

JAVA (continued 8/21)

```
static long clamp(long value, long minimum, long maximum) {
    return Math.max(minimum, Math.min(value, maximum));
}

static void example19MethodOverloading() {
    check(clamp(12, 0, 10) == 10, "int overload");
    check(clamp(12L, 0L, 10L) == 10L, "long overload");
}

static void increment(int number) {
    number++;
    check(number == 8, "local primitive changed");
}

static void example20PrimitivePassByValue() {
    int caller = 7;
    increment(caller);
    check(caller == 7, "caller primitive unchanged");
}

static final class Person {
    String name;
    Person(String name) { this.name = name; }
}

static void rename(Person person) {
    person.name = "Updated";
}

static void example21MutableObjectPassByValue() {
    Person person = new Person("Original");
    rename(person);
    check(person.name.equals("Updated"), "shared mutation");
}
```

JAVA (continued 9/21)

```
static void replace(Person person) {
    person = new Person("New");
    check(person.name.equals("New"), "local reference reassigned");
}

static void example22ReferenceReassignment() {
    Person person = new Person("Caller");
    replace(person);
    check(person.name.equals("Caller"), "caller reference unchanged");
}

static void example23OneDimensionalArray() {
    int[] values = {2, 4, 6};
    check(values.length == 3 && values[1] == 4, "one dimensional array");
}

static void example24TwoDimensionalArray() {
    int[][] matrix = {{1, 2}, {3, 4}};
    check(matrix[1][0] == 3, "matrix");
}

static void example25JaggedArray() {
    int[][] rows = {new int[1], new int[3]};
    check(rows[0].length == 1 && rows[1].length == 3, "jagged array");
}

static void example26ArrayAliasing() {
    int[] first = {1, 2};
    int[] alias = first;
    alias[0] = 9;
    check(first[0] == 9, "array aliases share mutation");
}

static void example27ArrayCopy() {
    int[] original = {1, 2};
```

JAVA (continued 10/21)

```
int[] copy = Arrays.copyOf(original, original.length);
copy[0] = 9;
check(original[0] == 1, "independent primitive array copy");
}

static void example28StringEquality() {
    String left = new String("java");
    String right = new String("java");
    check(left != right && left.equals(right), "identity versus content");
}

static void example29StringPool() {
    String first = "pool";
    String second = "pool";
    String explicit = new String("pool");
    check(first == second && first != explicit, "pool and explicit construction");
}

static void example30StringBuilder() {
    String result = new StringBuilder().append("java").append('-').append(21).toString();
    check(result.equals("java-21"), "builder");
}

static void example31CharacterToDigit() {
    char character = '7';
    int asciiDigit = character - '0';
    int generalDigit = Character.digit(character, 10);
    check(asciiDigit == 7 && generalDigit == 7, "digit conversion");
}

static final class Student {
    final int id;
    String name;
    Student(int id, String name) { this.id = id; this.name = name; }
}
```

JAVA (continued 11/21)

```
static void example32ClassAndObject() {
    Student student = new Student(1, "Ada");
    check(student.id == 1 && student.name.equals("Ada"), "class and object");
}

static final class Rectangle {
    final int width;
    final int height;
    Rectangle() { this(1, 1); }
    Rectangle(int width, int height) { this.width = width; this.height = height; }
    int area() { return width * height; }
}

static void example33ConstructorOverloading() {
    check(new Rectangle().area() == 1, "no argument constructor");
    check(new Rectangle(3, 4).area() == 12, "parameter constructor");
}

static void example34ConstructorChaining() {
    Rectangle unit = new Rectangle();
    check(unit.width == 1 && unit.height == 1, "this constructor chaining");
}

static final class Counter {
    private static int created;
    Counter() { created++; }
    static int created() { return created; }
}

static void example35StaticFieldAndMethod() {
    int before = Counter.created();
    new Counter();
    new Counter();
}
```

JAVA (continued 12/21)

```
        check(Counter.created() == before + 2, "static shared count");
    }

    static final class BoundedCounter {
        private final int maximum;
        private int value;
        BoundedCounter(int maximum) { this.maximum = maximum; }
        void increment() { if (value == maximum) throw new IllegalStateException(); value++; }
        int value() { return value; }
    }

    static void example36Encapsulation() {
        BoundedCounter counter = new BoundedCounter(2);
        counter.increment();
        check(counter.value() == 1, "encapsulated invariant");
    }

    static final class Coordinate {
        private final int row;
        private final int column;
        Coordinate(int row, int column) { this.row = row; this.column = column; }
        int row() { return row; }
        int column() { return column; }
    }

    static void example37ImmutableClass() {
        Coordinate coordinate = new Coordinate(2, 3);
        check(coordinate.row() == 2 && coordinate.column() == 3, "immutable state");
    }

    static class Animal {
        String category() { return "animal"; }
    }
```

JAVA (continued 13/21)

```
static final class Dog extends Animal {
    @Override String category() { return "dog"; }
}

static void example38Inheritance() {
    Dog dog = new Dog();
    check(dog instanceof Animal, "is a relationship");
}

static void example39MethodOverriding() {
    Dog dog = new Dog();
    check(dog.category().equals("dog"), "override");
}

static void example40RuntimePolymorphism() {
    Animal animal = new Dog();
    check(animal.category().equals("dog"), "dynamic dispatch");
}

abstract static class Parser {
    abstract int parse(String text);
    int parsePositive(String text) {
        int value = parse(text);
        if (value <= 0) throw new IllegalArgumentException();
        return value;
    }
}

static final class DecimalParser extends Parser {
    @Override int parse(String text) { return Integer.parseInt(text); }
}

static void example41AbstractClass() {
    check(new DecimalParser().parsePositive("7") == 7, "abstract class");
}
```


JAVA (continued 14/21)

```
}

interface Formatter {
    String format(int value);
    default String label(int value) { return "value=" + format(value); }
}

static void example42Interface() {
    Formatter decimal = Integer::toString;
    check(decimal.label(9).equals("value=9"), "interface default method");
}

interface Clock {
    long nowMillis();
}

static final class ExpiringToken {
    private final Clock clock;
    private final long expiresAt;
    ExpiringToken(Clock clock, long expiresAt) { this.clock = clock; this.expiresAt = expiresAt; }
    boolean expired() { return clock.nowMillis() >= expiresAt; }
}

static void example43Composition() {
    Clock fixed = () -> 100L;
    check(new ExpiringToken(fixed, 90L).expired(), "composed clock");
}

static void example44WrapperParsing() {
    int value = Integer.parseInt("42");
    check(value == 42, "wrapper parsing");
}

static void example45AutoboxingAndUnboxing() {
```

JAVA (continued 15/21)

```
    Integer boxed = 7;
    int primitive = boxed;
    check(primitive == 7, "boxing and unboxing");
}

static void example46IntegerCaching() {
    Integer first = 127;
    Integer second = 127;
    Integer highFirst = 128;
    Integer highSecond = 128;
    check(first == second, "required small cache");
    check(highFirst.equals(highSecond), "outside cache compare by value");
}

static final class Box<T> {
    private final T value;
    Box(T value) { this.value = value; }
    T value() { return value; }
}

static <T> T first(List<T> values) {
    if (values.isEmpty()) throw new IllegalArgumentException("empty");
    return values.get(0);
}

static void example47GenericClassAndMethod() {
    Box<String> box = new Box<>("java");
    check(box.value().equals("java") && first(List.of(4, 5)) == 4, "generics");
}

static void example48ArrayList() {
    List<String> values = new ArrayList<>();
    values.add("a");
    values.add("b");
}
```

JAVA (continued 16/21)

```
        check(values.get(1).equals("b"), "array list");
    }

    static void example49HashSet() {
        Set<Integer> values = new HashSet<>(List.of(2, 2, 3));
        check(values.size() == 2, "set deduplication");
    }

    static void example50HashMapFrequency() {
        Map<String, Integer> frequency = new HashMap<>();
        for (String word : List.of("java", "dsa", "java")) {
            frequency.merge(word, 1, Integer::sum);
        }
        check(frequency.get("java") == 2, "frequency map");
    }

    static void example51QueueWithArrayDeque() {
        Deque<Integer> queue = new ArrayDeque<>();
        queue.offerLast(1);
        queue.offerLast(2);
        check(queue.pollFirst() == 1, "fifo queue");
    }

    static void example52StackWithArrayDeque() {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(1);
        stack.push(2);
        check(stack.pop() == 2, "lifo stack");
    }

    static void example53PriorityQueue() {
        PriorityQueue<Integer> heap = new PriorityQueue<>(List.of(9, 3, 5));
        check(heap.poll() == 3, "min heap");
    }
}
```

JAVA (continued 17/21)

```
enum OrderStatus {
    CREATED, PAID, SHIPPED, DELIVERED, CANCELLED;
    boolean terminal() { return this == DELIVERED || this == CANCELLED; }
}

static void example54Enum() {
    check(OrderStatus.DELIVERED.terminal(), "enum behavior");
}

static int parsePositive(String text) throws IOException {
    int value;
    try {
        value = Integer.parseInt(text);
    } catch (NumberFormatException exception) {
        throw new IOException("invalid integer", exception);
    }
    if (value <= 0) throw new IOException("not positive");
    return value;
}

static void example55CheckedException() {
    try {
        check(parsePositive("4") == 4, "checked exception result");
    } catch (IOException exception) {
        throw new AssertionError(exception);
    }
}

static void requirePositive(int value) {
    if (value <= 0) throw new IllegalArgumentException("positive required");
}

static void example56UncheckedException() {
```

JAVA (continued 18/21)

```
        boolean thrown = false;
        try {
            requirePositive(0);
        } catch (IllegalArgumentException exception) {
            thrown = true;
        }
        check(thrown, "unchecked precondition");
    }

    static final class FlagResource implements AutoCloseable {
        private final boolean[] closed;
        FlagResource(boolean[] closed) { this.closed = closed; }
        @Override public void close() { closed[0] = true; }
    }

    static void example57TryWithResources() {
        boolean[] closed = {false};
        try (FlagResource ignored = new FlagResource(closed)) {
            check(ignored != null && !closed[0], "open in body");
        }
        check(closed[0], "closed after body");
    }

    static void example58ScannerInput() {
        try (Scanner scanner = new Scanner("3 4 5 6")) {
            int length = scanner.nextInt();
            int sum = 0;
            for (int index = 0; index < length; index++) sum += scanner.nextInt();
            check(sum == 15, "scanner input");
        }
    }

    static void example59BufferedReaderInput() {
        try (BufferedReader reader = new BufferedReader(new StringReader("7 8\n"))) {
```

JAVA (continued 19/21)

```
        String[] tokens = reader.readLine().split("\\s+");
        int sum = Integer.parseInt(tokens[0]) + Integer.parseInt(tokens[1]);
        check(sum == 15, "buffered reader input");
    } catch (IOException exception) {
        throw new AssertionError(exception);
    }
}

static long calculateSum(int[] numbers) {
    long sum = 0L;
    for (int number : numbers) sum += number;
    return sum;
}

static void example60InterviewQualityRefactoring() {
    int[] numbers = {Integer.MAX_VALUE, 1};
    check(calculateSum(numbers) == 2_147_483_648L, "clear name and safe accumulator");
}

static void example61ListOperations() {
    List<Integer> values = new ArrayList<>(List.of(10, 30));
    values.add(1, 20);
    values.set(0, 5);
    check(values.equals(List.of(5, 20, 30)), "list operations");
}

static void example62ListRemovalOverloads() {
    List<Integer> values = new ArrayList<>(List.of(10, 20, 30));
    values.remove(1);
    values.remove(Integer.valueOf(30));
    check(values.equals(List.of(10)), "index and value removal");
}

static void example63SetVariants() {
```

JAVA (continued 20/21)

```
Set<Integer> insertionOrdered = new LinkedHashSet<>(List.of(3, 1, 3, 2));
Set<Integer> sorted = new TreeSet<>(insertionOrdered);
check(insertionOrdered.toString().equals("[3, 1, 2]"), "linked set order");
check(sorted.toString().equals("[1, 2, 3]"), "tree set order");
}

static void example64MapUpdateApis() {
    Map<String, List<Integer>> groups = new HashMap<>();
    groups.computeIfAbsent("odd", ignored -> new ArrayList<>()).add(3);
    Map<String, Integer> counts = new HashMap<>();
    counts.merge("java", 1, Integer::sum);
    counts.merge("java", 1, Integer::sum);
    check(groups.get("odd").equals(List.of(3)) && counts.get("java") == 2,
           "map update APIs");
}

static void example65MapIteration() {
    Map<String, Integer> ordered = new LinkedHashMap<>();
    ordered.put("a", 1);
    ordered.put("b", 2);
    int sum = 0;
    for (Map.Entry<String, Integer> entry : ordered.entrySet()) {
        check(!entry.getKey().isEmpty(), "entry key");
        sum += entry.getValue();
    }
    check(sum == 3, "entry iteration");
}

static void example66DequeConventions() {
    Deque<Integer> deque = new ArrayDeque<>();
    deque.offerLast(1);
    deque.offerLast(2);
    check(deque.pollFirst() == 1, "queue convention");
    deque.push(3);
}
```

JAVA (continued 21/21)

```
        check(deque.pop() == 3, "stack convention");
    }

    static void example67PriorityQueueComparator() {
        PriorityQueue<Integer> maximums = new PriorityQueue<>(Comparator.reverseOrder());
        maximums.addAll(List.of(Integer.MIN_VALUE, 0, Integer.MAX_VALUE));
        check(maximums.poll() == Integer.MAX_VALUE, "safe max heap comparator");
    }

    static void example68CollectionFactories() {
        List<String> fixedSize = Arrays.asList("a", "b");
        fixedSize.set(0, "x");
        List<String> unmodifiable = List.of("a", "b");
        check(fixedSize.equals(List.of("x", "b")) && unmodifiable.size() == 2,
            "factory capabilities");
    }

    static void example69SafeIteratorRemoval() {
        List<Integer> values = new ArrayList<>(List.of(-1, 0, 2));
        for (Iterator<Integer> iterator = values.iterator(); iterator.hasNext();) {
            if (iterator.next() < 0) iterator.remove();
        }
        check(values.equals(List.of(0, 2)), "iterator removal");
    }

    static void example70CopiesAndViews() {
        List<String> source = new ArrayList<>(List.of("a"));
        List<String> snapshot = List.copyOf(source);
        List<String> view = Collections.unmodifiableList(source);
        source.add("b");
        check(snapshot.equals(List.of("a")), "copy is independent");
        check(view.equals(List.of("a", "b")), "view reflects source");
    }
}
```


Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: trace types, expressions, methods, arrays, strings, objects, and collection operations
- Interview Core: compile the 70-example companion and complete output/debugging drills
- SDE-2 Follow-up: expose contracts, aliases, failure behavior, and collection trade-offs
- Challenge: pass the cumulative and final readiness assessments without notes

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

This is the first volume in the sequence.

[Series index](#)

[Next volume: 02 - Time and Space Complexity for Java Interviews](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Learning Step 1 - Volume 3 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.